

Scalable Secure Multiparty Computation with Perfect Security from Preprocessing

Yifan Song¹ and Xiayi Ye²

¹ Tsinghua University and Shanghai Qi Zhi Institute
yfsong@mail.tsinghua.edu.cn

² Institute for Network Sciences and Cyberspace, Tsinghua University
yexx23@mails.tsinghua.edu.cn

Abstract. In this work, we study the communication complexity of MPC achieving perfect security with optimal resilience ($t < n/3$). We ask the question: “Is it possible to build a perfectly secure MPC for arithmetic circuits of size $|C|$ with optimal resilience with communication of $o(|C| \cdot n)$ field elements?” On the positive side, we construct a perfectly secure MPC protocol for SIMD circuits with a communication complexity of $O(|C|)$ elements assuming preprocessing data of size $O(|C|)$, where the preprocessing data consists of packed Beaver triples over bivariate polynomials. Furthermore, we show that packed Beaver triples over bivariate polynomials can be prepared at an amortized cost of $O(1)$ elements plus $O(1)$ three-party Beaver triples per secret.

On the negative side, we establish a communication lower bound proving that preparing packed Beaver triples over bivariate polynomials requires at least $\Omega(n)$ elements of communication per secret. This lower bound is derived by first proving a communication lower bound for verifying the correctness of packed Beaver triples with perfect security with abort, and then efficiently reducing the task of verifying packed Beaver triples to preparing packed Beaver triples over bivariate polynomials. To match this bound, we give a concrete construction for packed Beaver triples over bivariate polynomials with $O(n)$ elements per secret, demonstrating the tightness of our lower bound.

Our proof technique also extends to show that for the task of computing the inner-product of two length- $|C|$ vectors, any MPC protocol that achieves perfect security with abort requires either $\Omega(|C| \cdot n)$ elements of communication or $\Omega(|C|)$ elements of preprocessing data.

Table of Contents

1	Introduction	3
1.1	Our Contribution	3
2	Technical Overview	5
2.1	Starting Point: Perfect Security with Abort in the Preprocessing Model	5
2.2	Perfect Security with GOD from Bivariate Polynomials	6
2.3	Negative Result of Preparing Packed Beaver Triples	7
3	Preliminary	10
3.1	Security Definition	10
3.2	Packed Secret Sharing via Bivariate Polynomials	10
3.3	Byzantine Agreement	12
3.4	Party Elimination Framework	13
3.5	Building Blocks	13
4	Perfectly Secure MPC in the Preprocessing Model	16
4.1	Preprocessing Data	16
4.2	Computation Phase	16
4.3	Verification Phase	18
4.4	Main Protocol	20
5	Lower Bounds	26
5.1	Warm-up: Impossibility Result of Computing AND of Two Bits	27
5.2	Communication Complexity of Verifying Packed Triples	28
5.3	Communication Complexity of Preparing Packed Triples over Bivariate Polynomials	37
5.4	Communication Complexity of Inner Product Functions	42
5.5	Preparing Packed Beaver Triples	48
6	Preparing Packed Beaver Triples in the 3-Party Beaver Triple-Hybrid Model	49

1 Introduction

Secure multiparty computation (MPC) allows a set of mutually distrustful parties to jointly compute a function on their private inputs. Informally, an MPC protocol guarantees that each party learns only his own input and output, and nothing else. Since the concept was introduced by [Yao82], initial feasibility results were obtained in the computational setting, where adversaries are assumed to have bounded computational power [Yao82,GMW87]. Later works [BGW88,CCD88] extended these results to the unconditional (information-theoretic) setting, showing that MPC is achievable against up to $t < n/3$ corrupted parties, assuming secure point-to-point communication channels.

In this work, we focus on the communication complexity of perfectly secure MPC protocols with optimal resilience $t < n/3$. The first feasibility result was given by [BGW88] in 1988. Since then, a line of works focuses on improving the communication complexity of perfectly secure MPC protocols. In the literature, it is known from [BH08,GLS19,AAPP23] that linear communication of $O(|C| \cdot n)$ field elements can be achieved to compute a circuit of size $|C|$ among n parties. The recent work [SY25b] further improves the online communication complexity to $O(|C|)$ field elements while maintaining the overall linear communication complexity, inspired by Turbopack [EGPS22] that achieves a similar result in the honest majority setting. Despite the great improvement made by the previous works, there is no construction that achieves $o(|C| \cdot n)$ field elements and we still do not know whether a linear communication complexity is necessary for perfectly secure MPC protocols. The only evidence is from [DLS21] which proves a lower bound of $\Theta(|C| \cdot n)$ field elements of communication for certain types³ of perfectly secure MPC protocols targeting for a specific function. This leads to our following question.

“Can we build a perfectly secure MPC protocol with optimal resilience ($t < n/3$) that achieves $o(|C| \cdot n)$ field elements of communication complexity when computing a circuit of size $|C|$?”

1.1 Our Contribution

In this work we obtain the following two results. On the positive side, we show that in the preprocessing model, there is a perfectly secure MPC protocol with optimal resilience that requires both $O(|C|)$ field elements of preprocessing data and communication among all parties.

Theorem 1 (informal). *Let n be the number of parties and $t = (n - 1)/3$. Let $\mathbb{F}$ be a finite field of size $|\mathbb{F}| \geq 2n$. For a SIMD circuit C over $\mathbb{F}$, there exists a perfectly secure MPC protocol in the circuit-independent preprocessing model computing C in the presence of a fully malicious adversary controlling at most t parties. The cost is $O(|C| + C_O \cdot n + n^5)$ elements of communication along with preprocessing data of size $O(|C|)$, where $|C|$ and C_O denote the circuit size and the number of output gates, respectively.*

The formal theorem is stated as Theorem 6. Our result can be upgraded to handle a general circuit by incorporating the techniques of network routing in [GPS21,GPS22]. At a first glance, the readers may feel that a similar result can be obtained from [SY25b]: If we view the output in the offline phase of [SY25b] as preprocessing data, we directly obtain a construction in the preprocessing model with $O(|C|)$ field elements of communication. However, the size of the output in the offline phase of [SY25b] is $\Theta(|C| \cdot n)$ field elements. This is because their construction requires to prepare a random degree- t Shamir sharing for each wire in the circuit. From the technique side, these random degree- t Shamir sharings allow them to compute a degree- t Shamir sharing for each wire value in the online phase and the error-correction property allows them to always reconstruct the whole degree- t Shamir sharing. In our case, we have to always use packed Shamir sharings to keep $O(|C|)$ field elements of preprocessing data and communication, in which case parties can only detect inconsistency but cannot reconstruct the secrets with guarantee. This brings difficulties when the reconstruction fails. We will elaborate our techniques in Section 2.

Going beyond, we show that the required preprocessing data in Theorem 1 can be prepared from three-party Beaver triples.

³ The lower bounds in [DLS21] apply to the so-called two-phase protocols, in which the inputs are committed during the input phase, before any information about the output is revealed.

Theorem 2 (informal). *Let n be the number of parties and $t = (n - 1)/3$. There exists a protocol that, assuming three-party Beaver triples, generates the preprocessing data of size $O(|C|)$ used in Theorem 1 with perfect security against a fully malicious adversary corrupting at most t corrupted parties. The total cost is $O(|C|)$ elements of communication plus $O(|C|)$ three-party Beaver triples.*

The formal statement appears in Lemma 6.

On the negative side, we show that the preprocessing data we need in Theorem 1 cannot be prepared with $o(|C| \cdot n)$ communication in the plain model. To be more concrete, the preprocessing data we need is a variant of packed Beaver triples based on packed Shamir sharings. Our negative result shows that no perfectly secure MPC protocol can prepare such packed Beaver triples with sub-linear communication.

Theorem 3 (informal). *Let $n = 3t + 1$. Any protocol generating packed Beaver triples over bivariate polynomials of size $O(|C|)$ with perfect security with abort against a fully malicious adversary controlling at most t corrupted parties communicates at least $\Omega(|C| \cdot n)$ field elements.*

The formal theorem is stated as Theorem 7. To match this lower bound, we give a concrete construction for packed Beaver triples over bivariate polynomials in Section 5.5, demonstrating the tightness of our lower bound.

Theorem 4 (informal). *Let n be the number of parties and $t = (n - 1)/3$. There exists a protocol generating packed Beaver triples over bivariate polynomials of size $O(|C|)$ with perfect security against a fully malicious adversary controlling at most t corrupted parties. The communication cost is $O(|C| \cdot n)$ elements.*

The formal statement is presented in Lemma 5. We further extend our proof technique to show that for the task of computing the inner-product of two vectors of length $|C|$, a perfectly secure MPC protocol requires either $\Omega(|C| \cdot n)$ elements of communication or $\Omega(|C|)$ elements of preprocessing data.

Theorem 5 (informal). *Let $n = 3t + 1$. Any protocol that computes the inner-product of two vectors of length $|C|$ with perfect security with abort in the preprocessing model, against a fully malicious adversary controlling at most t corrupted parties, either communicates $\Omega(|C| \cdot n)$ elements or requires preprocessing data of size $\Omega(|C|)$.*

The formal theorem is stated as Theorem 8 and Corollary 3. This result implies that there exists a circuit $\tilde{C}$ with input size $|\tilde{C}|$ and output size $O(1)$ for which a linear communication complexity of $\Theta(|\tilde{C}| \cdot n)$ field elements is necessary for perfectly secure MPC protocols. It also suggests that there may not exist a perfectly secure MPC protocol with $o(|C| \cdot n)$ communication for a general circuit C .

Comparison with [DPPR25]. We note that a concurrent work [DPPR25] obtains a similar communication lower bound. To be more concrete, they show that for the task of computing a variant of oblivious transfer functionality (OT) with perfect security with abort, there is no MPC protocol with $o(I \cdot n)$ communication, where I is the message length. Note that such a string OT functionality can be computed with a circuit of size $\Theta(I)$. In comparison, our work targets for the task of preparing packed Beaver triples, which leads to a perfectly secure MPC protocol with both $O(|C|)$ communication and $O(|C|)$ preprocessing data, and the task of computing the inner-product of two vectors.

From the technique side, the authors in [DPPR25] begin with a four-party protocol that computes the string OT functionality with perfect security with abort. They then derive a three-party protocol for the same functionality by randomly emulating the fourth party, while preserving the communication complexity. This three-party protocol is shown to imply a secure two-party protocol for private message transfer over authenticated and insecure channels in the correlated randomness model, where the amount of correlated randomness required corresponds to the communication complexity of the three-party protocol. Then they show a lower bound that, in any such two-party private message transfer protocol, the correlated randomness must be long enough to one-time pad the entire private message. This in turn implies a communication lower bound for the original four-party protocol that securely realizes the string OT functionality. Finally, the result is extended to the general n -party setting with optimal resilience ($t < n/3$).

In contrast, our technique starts from a negative result in the 3-party setting for perfect security with abort. Very informally, we show that there is a strategy for the corrupted party which would cause honest parties ending up with incorrect output for the AND function with non-zero probability. Then in the 4-party setting, we manage to show that if the communication of a party is too small, the same attack can be applied which violates the perfect correctness. This implies a communication lower bound for each party. Finally, we show that this result naturally extends to the n -party setting, which gives the lower bound we want.

2 Technical Overview

In this section, we give a high-level overview of our main techniques. Let $\mathbb{F}$ be a finite field such that $|\mathbb{F}| \geq 2n$. Recall that in this work we focus on the optimal resilience where $n = 3t + 1$. We will use $[\mathbf{x}]_d$ to denote a degree- d packed Shamir sharing of the secret vector $\mathbf{x} \in \mathbb{F}^k$, where k is the number of secrets packed in a single sharing. To achieve privacy against t corrupted parties, we will use a degree- $(t + k - 1)$ packed Shamir sharing to store k secrets. In addition, we set $k = \lceil \frac{n+5}{6} \rceil$ so that $t + 2k - 2 = 2t$. Jumping ahead, we will use the technique of packed Beaver triple [GPS22] to compute a group of k multiplications where parties need to reconstruct a degree- $(t + 2k - 2)$ packed Shamir sharing. Setting k in the above way ensures that the whole sharing is determined by shares held by all honest parties.

In the following, we start with our positive result. For simplicity, we focus on a SIMD circuit.

2.1 Starting Point: Perfect Security with Abort in the Preprocessing Model

Our first step is to understand what kind of preprocessing data is sufficient for us to build a perfectly secure MPC protocol with $O(|C|)$ communication. As we have mentioned in Section 1.1, we may interpret the construction in [SY25b] as a protocol in the preprocessing model, by viewing the output in the offline phase as preprocessing data. This gives us a perfectly secure MPC protocol with $O(|C|n)$ field elements of preprocessing data, and $O(|C|)$ field elements of communication.

When focusing on a SIMD circuit, the preprocessing data in [SY25b] just contains Beaver triples over standard degree- t (non-packed) Shamir sharings. These standard Beaver triples are packed to degree- $(t + k - 1)$ packed Beaver triples and then used in the online phase. If the online phase fails due to an inconsistent degree- $(t + k - 1)$ packed Shamir sharing, parties may use the degree- t Shamir sharings in the preprocessing data to recover the correct sharing and locate a corrupted party.

We notice that if we only focus on perfect security with abort, it is sufficient to only assume packed Beaver triples. This is because during the online computation, parties only need to reconstruct degree- $(t + k - 1)$ and degree- $(t + 2k - 2)$ packed Shamir sharings. In both cases, the whole sharing is fully determined by honest parties' shares. Thus corrupted parties cannot insert error without being detected. To be more concrete, suppose all parties want to compute a group of k multiplication gates. Parties start with input sharings $[\mathbf{x}]_{t+k-1}, [\mathbf{y}]_{t+k-1}$ and hold a random packed Beaver triple $[\mathbf{a}]_{t+k-1}, [\mathbf{b}]_{t+k-1}, [\mathbf{c}]_{t+k-1}$ satisfying that $\mathbf{c} = \mathbf{a} * \mathbf{b}$, where $*$ denotes the coordinate-wise multiplication operation. All parties in addition prepare a pair of random packed Shamir sharings $([\mathbf{r}]_{t+k-1}, [\mathbf{r}]_{t+2k-2})$, which can be prepared with perfect security [SY25b].

1. All parties locally compute $[\mathbf{x} + \mathbf{a}]_{t+k-1}, [\mathbf{y} + \mathbf{b}]_{t+k-1}$.
2. A designated party P_{king} receives $[\mathbf{x} + \mathbf{a}]_{t+k-1}, [\mathbf{y} + \mathbf{b}]_{t+k-1}$ and reshares $[\mathbf{x} + \mathbf{a}]_{k-1}, [\mathbf{y} + \mathbf{b}]_{k-1}$ to all parties.
3. All parties locally compute

$$[\mathbf{z} + \mathbf{r}]_{t+2k-2} = [\mathbf{x} + \mathbf{a}]_{k-1} \cdot [\mathbf{y} + \mathbf{b}]_{k-1} - [\mathbf{x} + \mathbf{a}]_{k-1} \cdot [\mathbf{b}]_{t+k-1} - [\mathbf{a}]_{t+k-1} \cdot [\mathbf{y} + \mathbf{b}]_{k-1} + [\mathbf{c}]_{k-1} + [\mathbf{r}]_{t+2k-2}.$$

4. The party P_{king} receives $[\mathbf{z} + \mathbf{r}]_{t+2k-2}$ and reshares $[\mathbf{z} + \mathbf{r}]_{k-1}$.
5. All parties locally compute $[\mathbf{z}]_{t+k-1} = [\mathbf{z} + \mathbf{r}]_{k-1} - [\mathbf{r}]_{t+k-1}$.

In the above process, an honest P_{king} can detect inconsistency in Step 2 and Step 4. To protect against a corrupted P_{king} , additional verifications are required which we omit here for simplicity.

Thus, from the construction in [SY25b], we may obtain a perfectly secure MPC protocol with abort that achieves $O(|C|)$ elements of both preprocessing data and communication by just providing packed Beaver triples as preprocessing data.

Difficulty of Achieving Guaranteed Output Delivery. To achieve guaranteed output delivery, a generic approach is to use the party-elimination framework [HMP00]. At a high level, when the computation fails, all parties will together find out a corrupted party who deviates from the computation, or a pair of conflicted parties where they do not agree on the message sent from one party to the other. Notice that for the latter case, at least one party is corrupted. Such a pair of parties is also referred to as a disputed pair. This process is referred to as the *fault-localization* process. Then the identified corrupted party or the identified pair of conflicted parties are removed. The whole computation restarts with the remaining parties. Note that the corruption threshold of the remaining parties is still bounded by $1/3$. Since each time at least one corrupted party is eliminated, the number of failure attempts is bounded by $t = O(n)$. To avoid the communication blowup due to the failure attempts and retries, the whole computation is separated into $\Omega(n)$ segments that are evaluated sequentially. In case the computation of the current segment fails, all parties will only restart the computation of the current segment.

To make the above idea work, we need to construct a secure protocol that realizes the fault-localization process while protecting the secrecy of their inputs. This is usually not a difficult task in the plain model:

- First, if we do not care about the secrecy of their inputs, such a corrupted party or a disputed pair can be easily located by letting each party broadcast his view. Then each party may locally emulate the whole protocol and verify whether each party follows the protocol. Now because the computation fails, there must be either a corrupted party who does not follow the computation, or a pair of parties who do not agree on the message sent from one party to the other.
- To protect the secrecy of their inputs, we can simply use a generic MPC protocol where each party uses his view as input. To avoid the communication cost of the fault localization process dominating the communication complexity of the final protocol, we may choose a smaller segment size (and thus the number of segments may become $\text{poly}(n)$).

Unfortunately the above generic solution does not work in the preprocessing model even if we do not care about the secrecy of parties' inputs. This is because the preprocessing data is not generated by any party and if corrupted parties lie about their preprocessing data, we may not be able to find out the corrupted party or a disputed pair. For example, after opening all parties' view, we find that the first packed Shamir sharing $[\mathbf{a}]_{t+k-1}$ of some packed Beaver triple is inconsistent. Then we do not know which parties provide wrong shares. This is unavoidable since a degree- $(t+k-1)$ packed Shamir sharing cannot be reconstructed with guarantee when there are t incorrect shares as long as $k \geq 2$.

2.2 Perfect Security with GOD from Bivariate Polynomials

To resolve the above issue, our idea is to extend packed Shamir sharings which are based on *univariate* polynomials to *bivariate* polynomials. To be more concrete, let $\alpha_1, \dots, \alpha_n$ and $\beta_1, \dots, \beta_k$ be $n+k$ different field elements. To share a vector $\mathbf{x} \in \mathbb{F}^k$ using the packed Shamir sharing scheme, the dealer samples a random degree- $(t+k-1)$ polynomial $f(\cdot)$ such that $f(\beta_i) = x_i$ for all $i \in \{1, \dots, k\}$ and sets $[\mathbf{x}]_{t+k-1} = (f(\alpha_1), \dots, f(\alpha_n))$. When extending to bivariate polynomials, the secret $\mathbf{X}$ is a matrix of size $k \times k$. The dealer samples a random degree- $(t+k-1, t+k-1)$ bivariate polynomial $F(\cdot, \cdot)$ such that $F(\beta_i, \beta_j) = X_{i,j}$. Then each party P_i receives $f_i(\cdot) = F(\cdot, \alpha_i)$ and $g_i(\cdot) = F(\alpha_i, \cdot)$ as his shares. Effectively, this can be viewed as a linear packed secret sharing with packing size k^2 . We denote such a packed sharing as $[\mathbf{X}]_{t+k-1}$. Later we show that we can extend the technique of Beaver triple to such a linear packed secret sharing scheme and then use it to evaluate a group of k^2 multiplication gates in parallel.

The main benefit of using bivariate polynomials is the pairwise consistency: every pair of parties (P_i, P_j) hold two common points $F(\alpha_i, \alpha_j)$ and $F(\alpha_j, \alpha_i)$. Now when $[\mathbf{X}]_{t+k-1}$ is inconsistent, each party P_i broadcasts his share $f_i(\cdot), g_i(\cdot)$. Then we must observe at least one of the two cases:

- For some party P_i , $f_i(\cdot)$ or $g_i(\cdot)$ is not a valid degree- $(t+k-1)$ polynomial;
- For some pair of parties (P_i, P_j) , $f_i(\alpha_j) \neq g_j(\alpha_i)$ or $g_i(\alpha_j) \neq f_j(\alpha_i)$.

For the first case, P_i is a corrupted party. For the second case, (P_i, P_j) is a pair of disputed parties. To see why this is the case, first notice that all honest parties' shares are consistent with the underlying bivariate polynomial $F(\cdot, \cdot)$. Now suppose a corrupted party P_i provides an incorrect share, say $f_i(\cdot) \neq F(\cdot, \alpha_i)$. Either $f_i(\cdot)$ is not a valid degree- $(t + k - 1)$ polynomial or $f_i(\alpha_j) = F(\alpha_j, \alpha_i)$ holds for at most $t + k - 1 \leq 2t$ different α_j . In the latter case, there is an honest party P_j such that $f_i(\alpha_j) \neq F(\alpha_j, \alpha_i) = g_j(\alpha_i)$. Thus the pairwise consistency does not hold for (P_i, P_j) , which will be identified as a disputed pair.

Thus, although there is still no guarantee that $\llbracket \mathbf{X} \rrbracket_{t+k-1}$ can be always reconstructed when there are t incorrect shares, we are able to identify either a corrupted party or a disputed pair when the reconstruction fails *without the help from the dealer*.

Other Difficulties and Our Solutions. Now we can sketch our construction for a SIMD circuit based on packed sharings from bivariate polynomials. At a high level, we follow the party elimination framework and utilize the pairwise consistency of packed sharings based on bivariate polynomials to locate disputed pairs.

In the beginning, all parties receive packed Beaver triples in the form of $(\llbracket \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{B} \rrbracket_{t+k-1}, \llbracket \mathbf{C} \rrbracket_{t+k-1})$ where $\mathbf{A}, \mathbf{B} \in \mathbb{F}^{k \times k}$ are random matrices and $\mathbf{C} = \mathbf{A} * \mathbf{B}$. For each group of k^2 multiplication gates, all parties also prepare $(\llbracket \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{R} \rrbracket_{t+2k-2})$ which can be done with an amortized communication of $O(n^2)$ field elements following the technique in [BH08].

In the online phase, for each group of k^2 addition gates with input sharings $\llbracket \mathbf{X} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} \rrbracket_{t+k-1}$, all parties locally add up their shares to obtain $\llbracket \mathbf{X} + \mathbf{Y} \rrbracket_{t+k-1} = \llbracket \mathbf{X} \rrbracket_{t+k-1} + \llbracket \mathbf{Y} \rrbracket_{t+k-1}$. For each group of k^2 multiplication gates with input sharings $\llbracket \mathbf{X} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} \rrbracket_{t+k-1}$, a fresh Beaver triple is consumed.

1. All parties locally compute $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{B} + \mathbf{Y} \rrbracket_{t+k-1}$.
2. A designated party P_{king} receives $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{B} + \mathbf{Y} \rrbracket_{t+k-1}$ and reshapes $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1}, \llbracket \mathbf{B} + \mathbf{Y} \rrbracket_{k-1}$ to all parties.
3. All parties locally compute

$$\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2} = \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} - \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{B} \rrbracket_{t+k-1} - \llbracket \mathbf{A} \rrbracket_{t+k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} + \llbracket \mathbf{C} \rrbracket_{k-1} + \llbracket \mathbf{R} \rrbracket_{t+2k-2}.$$

4. The party P_{king} receives $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2}$ and reshapes $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$.
5. All parties locally compute $\llbracket \mathbf{Z} \rrbracket_{t+k-1} = \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1} - \llbracket \mathbf{R} \rrbracket_{t+k-1}$.

For each sharing P_{king} received, if the sharing is inconsistent, P_{king} verifies the pairwise consistency and finds (P_i, P_j) whose common shares do not match. Then P_{king} broadcasts the identities of these two parties together with the received shares from P_i and P_j , and P_i, P_j broadcast the shares they sent to P_{king} . The two parties who broadcast different values are identified as a disputed pair. (To protect against a corrupted P_{king} , additional verifications are required for sharings distributed by P_{king} , which we omit here for simplicity and refer the readers to Section 4 for more details.) If a disputed pair of parties is identified, all parties eliminate these two parties and re-evaluate the current segment following the party-elimination framework.

However, to make the above fault localization process work, we need the number of remaining honest parties to be at least $t + 2k - 1$ so that $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2}$ is fully determined by honest parties' shares. Recall that $t + 2k - 1 = 2t + 1$. This means that after we remove a disputed pair of parties (which may contain one honest party), we cannot rely on the above process to locate a disputed pair.

To resolve this issue, in our construction, we use a smaller packing parameter $k = n/12 = \Theta(n)$, which still preserves $O(|C|)$ field elements of preprocessing data and online communication. On one hand, the above sketch still works as long as the number of eliminated honest parties is bounded by $(2t+1) - (t+2k-1) \approx n/6$. On the other hand, after eliminating $n/6$ pairs of disputed parties, the corruption threshold of the remaining parties is reduced to $1/4$. In this case, the remaining parties may restart the whole computation using a perfectly secure MPC protocol against $1/4$ corruption, which can achieve $O(|C|)$ communication using packed Shamir sharings. We refer the readers to Section 4 for more details.

2.3 Negative Result of Preparing Packed Beaver Triples

Our positive result sheds light on achieve $o(|C|n)$ communication for perfectly secure MPC as it is sufficient to focus on preparing packed Beaver triples over bivariate polynomials with $O(n^2)$ field elements of communication per triple (so that the amortized communication per secret is $O(1)$ field elements).

Unfortunately, in this section we show that the task of preparing packed Beaver triples over bivariate polynomials cannot be achieved with an amortized communication complexity of $o(n^3)$ field elements, even just for perfect security with abort. Our negative result shows that

- there exists a concrete computation task C that cannot be realized with $o(|C|n)$ field elements of communication,
- and for a given circuit $\tilde{C}$, computing $\tilde{C}$ with perfect security may inherently require $O(|C|n)$ a linear communication.

Warm-up: Impossibility Result of 3PC AND. To better explain our idea, we start with a three-party computation task $f_{\text{AND}}(\perp, x_2, x_3) \rightarrow (\perp, x_2 \cdot x_3, x_2 \cdot x_3)$. We show that there is no 3-party protocol that computes f_{AND} with perfect security with abort. Later, we will extend the proof technique and use it to show our negative result.

For the sake of contradiction, suppose Π is a 3-party protocol that computes f_{AND} with perfect security with abort. The parties are denoted by P_1, P_2, P_3 . At a high level, we want to construct an execution where $x_2 = x_3 = 1$ and P_1 is corrupted. In this case, both P_2 and P_3 should take either 1 or $\perp$ as the output. We will construct a corrupted P_1 so that P_2 and P_3 both take 0 as output, thus leading to a contradiction. The same technique idea is also used in [PP25] for network-agnostic MPC with perfect security.

We start with an honest execution with input $x_2 = x_3 = 0$ and each party's randomness r_1, r_2, r_3 , which we refer to as exec_0 . Now consider the perfect privacy against a corrupted P_2 . When $x_2 = 0$, we always have $x_2 \cdot x_3 = 0$. Thus a semi-honest P_2 should not distinguish whether x_3 is 0 or 1. This implies that there should exist an honest execution with input $x_2 = 0, x_3 = 1$ and each party's randomness r'_1, r_2, r'_3 , denoted by exec_1 , such that P_2 's view is identical to that in exec_0 . Similarly, from the perfect privacy against a corrupted P_3 , there exists an honest execution with input $x_2 = 1, x_3 = 0$ and each party's randomness r''_1, r'_2, r_3 , denoted by exec_2 , such that P_3 's view is identical to that in exec_0 . Note that both P_2 's view and P_3 's view contain the communication transcript between P_2 and P_3 . This shows that in all three executions, the communication transcripts between P_2 and P_3 are identical.

Now consider the following execution where P_1 is corrupted: P_2 uses $(x_2 = 1, r'_2)$ as his input and randomness while P_3 uses $(x_3 = 1, r'_3)$ as his input and randomness. The corrupted P_1 first emulates exec_1 and exec_2 and obtains the messages he would send to P_2 and P_3 in both executions. Now in the actual execution, P_1 sends the messages he would send in exec_2 when interacting with P_2 and sends the messages he would send in exec_1 when interacting with P_3 . By induction, we can show that in each round, P_2 would see the same messages from P_1 and P_3 as those in exec_2 , and P_3 would see the same messages from P_1 and P_2 as those in exec_1 . Then P_2 would obtain the same output as that in exec_2 and P_3 would obtain the same output as that in exec_1 , which are both 0. This leads to a contradiction.

Intuition of Our Communication Lower Bound. Now we attempt to extend the above proof to the 4-party setting for the computation task $f_{\text{AND}}(\perp, \perp, x_2, x_3) \rightarrow (\perp, \perp, x_2 \cdot x_3, x_2 \cdot x_3)$ by adding a party P_0 . Notice that such an attempt must fail due to the positive result from [BGW88]. However, we try to obtain some intuition from this attempt about the communication required by each party.

Following the same analysis, we may obtain $\text{exec}_0, \text{exec}_1, \text{exec}_2$ that use inputs $(0, 0), (0, 1), (1, 0)$ respectively such that P_2 's view is identical in exec_0 and exec_1 while P_3 's view is identical in exec_0 and exec_2 , and then build the last execution exec_3 as above (where P_1 is corrupted). The major difference here is that we have another honest party P_0 . Recall that our strategy for P_1 is to use the same messages as those in exec_2 when interacting with P_2 so that P_2 cannot distinguish exec_2 and exec_3 . But the messages from P_0 to P_2 may be different in these two executions, which makes the same argument no longer work. On the other hand, if the messages P_0 receives and sends in all three executions $\text{exec}_0, \text{exec}_1, \text{exec}_2$ are identical, we can show that the messages P_0 receives and sends in exec_3 would be identical as well when P_1 follows our strategy.

Thus, given any fixed P_0 's received and sent messages, there should be no $\text{exec}_0, \text{exec}_1, \text{exec}_2$ that satisfy the above conditions, or otherwise we may construct exec_3 that breaks the perfect correctness. To derive a communication lower bound for perfectly secure MPC, our idea is to classify all possible honest executions

based on P_0 's received and sent messages. Intuitively, if P_0 's message size is small, then for some assignment of P_0 's messages, there will be sufficiently many executions that allow us to find $\text{exec}_0, \text{exec}_1, \text{exec}_2$.

Negative Result of Verifying Packed Beaver Triples. Instead of focusing on the AND function, our negative result targets for the verification of packed Beaver triples over packed Shamir sharings for two reasons.

- First, it is easy to extend to the general n -party setting and allows us to derive a communication lower bound for preparing packed Beaver triples over bivariate polynomials.
- Second, the packed Shamir sharings reveal no information about the secrets to any t parties. Thus, given that the input packed Beaver triples are correct, we may freely choose the secrets after fixing the shares of any t (corrupted) parties.

To be more concrete, we focus on verifying C degree- $(t+k-1)$ packed Beaver triples $([a_i]_{t+k-1}, [b_i]_{t+k-1}, [c_i]_{t+k-1})$. Let P_0 be a single party and each of P_1, P_2, P_3 be a collection of t parties. In the following, we abuse the notation by using P_0 's view to denote his input and all messages he receives and sends. I.e., we exclude the internal randomness in P_0 's view. Suppose m is the size of P_0 's view. Then there are at most $|\mathbb{F}|^m$ possible assignments for P_0 's view. Our goal is to find $\text{exec}_0, \text{exec}_1, \text{exec}_2$ such that P_0 's view is identical in all three executions.

We start with an arbitrary honest execution exec_0 with correct packed Beaver triples. Suppose the inputs are denoted by (I_0, I_1, I_2, I_3) . Now for any other inputs (I'_0, I'_1, I_2, I_3) such that P_2 's input remains unchanged and all parties' inputs form correct packed Beaver triples, by the perfect privacy against P_2 , there exists an execution such that P_2 's view remains the same. Then there are at least $|\mathbb{F}|^{2Ck}$ executions that keep P_2 's view unchanged since for each packed Beaver triple, after fixing P_2 's input, we may freely choose the secrets of the first two sharings, and then compute the secrets of the last sharing. All these executions are classified based on P_0 's view. Then we may pick any exec_1 that shares the same P_0 's view as exec_0 if exists. While the existence of exec_1 is not guaranteed for any exec_0 , we note that if we pick a random exec_0 , then with probability $1 - |\mathbb{F}|^{m-2Ck}$, there exists such an execution exec_1 .

Similarly, when we pick a random exec_0 , with probability $1 - |\mathbb{F}|^{m-2Ck}$, there exists an execution exec_2 such that P_3 's view remains unchanged and exec_2 shares the same P_0 's view as exec_0 . By union bound, with probability $1 - 2|\mathbb{F}|^{m-2Ck}$, we may find $\text{exec}_0, \text{exec}_1, \text{exec}_2$. Now we construct exec_3 as follows.

- P_2 uses the same input and randomness as that in exec_2 .
- P_3 uses the same input and randomness as that in exec_1 .
- P_0 uses the same input and randomness as that in exec_0 .
- The corrupted P_1 first emulates $\text{exec}_0, \text{exec}_1, \text{exec}_2$ and obtains the messages he would send to P_0, P_2, P_3 . Then when interacting with P_0 , P_1 uses messages in exec_0 ; when interacting with P_2 , P_1 uses messages in exec_2 ; when interacting with P_3 , P_1 uses messages in exec_1 .

We can show that (1) P_0 's view remains the same as that in $\text{exec}_0, \text{exec}_1, \text{exec}_2$, (2) P_2 's view is identical to that in exec_2 , and (3) P_3 's view is identical to that in exec_1 . Thus, all honest parties would output accept. To derive a contradiction, we want to show that the input packed Beaver triples are *incorrect* in exec_3 . First note that for each party P_i , his input in exec_3 can be computed by the summation of his inputs in exec_1 and exec_2 , and then subtracting his input in exec_0 . Thus, if we use $\mathbf{a}^{(i)}, \mathbf{b}^{(i)}, \mathbf{c}^{(i)}$ to denote the secrets of the input packed Beaver triples in exec_i , we have

$$(\mathbf{a}^{(0)}, \mathbf{b}^{(0)}, \mathbf{c}^{(0)}) + (\mathbf{a}^{(3)}, \mathbf{b}^{(3)}, \mathbf{c}^{(3)}) = (\mathbf{a}^{(1)}, \mathbf{b}^{(1)}, \mathbf{c}^{(1)}) + (\mathbf{a}^{(2)}, \mathbf{b}^{(2)}, \mathbf{c}^{(2)}).$$

Now for the sake of contradiction, suppose $\mathbf{c}^{(i)} = \mathbf{a}^{(i)} * \mathbf{b}^{(i)}$ for all $i \in \{0, 1, 2, 3\}$. Then it implies that

$$(\mathbf{a}^{(0)} - \mathbf{a}^{(1)}) * (\mathbf{b}^{(0)} - \mathbf{b}^{(2)}) + (\mathbf{a}^{(0)} - \mathbf{a}^{(2)}) * (\mathbf{b}^{(0)} - \mathbf{b}^{(1)}) = \mathbf{0}.$$

Unfortunately we may not obtain any contradiction from this requirement. In our proof we make the following two changes:

- First when finding exec_1 , we further restrict that $\mathbf{b}^{(1)} = \mathbf{b}^{(0)}$. I.e., we only change the secrets of the first sharing in exec_1 . Similarly, when finding exec_2 , we restrict that $\mathbf{a}^{(2)} = \mathbf{a}^{(0)}$. I.e., we only change the secrets of the second sharing in exec_2 . In this way, the above requirement becomes $(\mathbf{a}^{(0)} - \mathbf{a}^{(1)}) * (\mathbf{b}^{(0)} - \mathbf{b}^{(2)}) = \mathbf{0}$ since the second term is always $\mathbf{0}$. However this is still not sufficient for us to derive a contradiction.
- For the second change, we want to make sure there is an index i such that $a_i^{(0)} \neq a_i^{(1)}$ and $b_i^{(0)} \neq b_i^{(2)}$. In this way, $(a_i^{(0)} - a_i^{(1)}) \cdot (b_i^{(0)} - b_i^{(2)}) \neq 0$, which contradicts with the above requirement. To this end, when finding exec_1 , we require to sample a random exec_0 such that there are more than $|\mathbb{F}|^{Ck/2}$ executions such that the views of P_2 and P_0 remain unchanged. This allows us to pick an exec_1 such that $\mathbf{a}^{(1)}$ differs from $\mathbf{a}^{(0)}$ in more than $Ck/2$ entries. The same change is applied when finding exec_2 .

With the above changes, we are able to show contradiction as long as $m < Ck/2$, which implies that the size of messages P_0 sends or receives should be at least $\Theta(Ck)$.

Recall that P_1 is a collection of t parties and each party in P_1 is equivalent to P_0 . Thus we can show that any party in P_1 also needs to communicate at least $\Theta(Ck)$ field elements, leading to $\Theta(Ckn) = \Theta(Cn^2)$ communication lower bound for verifying packed Beaver triples.

Negative Result of Preparing Packed Beaver Triples. Finally, to show the communication lower bound for preparing packed Beaver triples, we construct a verification protocol for C packed Beaver triples with $O(Cn)$ communication in the preprocessing model assuming $O(C/n)$ correct packed Beaver triples over bivariate polynomials. At a high level, we first depack the prepared $O(C/n)$ packed Beaver triples over bivariate polynomials to $O(C)$ packed Beaver triples over standard packed Shamir sharings, and then apply the sacrifice technique to check each input packed Beaver triple. We refer the readers to Section 5.3 for more details. Since we have shown that $\Theta(Cn^2)$ elements of communication is necessary for the verification of packed Beaver triples, it implies that preparing each packed Beaver triple over bivariate polynomial requires at least $\Theta(n^3)$ elements of communication.

Moreover, we also construct a perfectly secure MPC protocol for packed Beaver triples over bivariate polynomials that matches the communication lower bound, showing the tightness of our result. Finally, following a similar technique in [SY25a], we prepare packed Beaver triples over bivariate polynomials assuming three-party Beaver triples, with amortized cost of $O(1)$ elements and $O(1)$ three-party Beaver triples per secret. This efficiently reduces the generation of packed Beaver triples to simpler correlations.

3 Preliminary

3.1 Security Definition

We consider a set of n parties $\{P_1, \dots, P_n\}$ where each party can provide inputs, receive outputs, and participate in the computation. For every pair of parties, there exists a secure (private and authenticated) synchronous channel so that they can directly send messages to each other. The communication complexity is measured by the number of bits via private channels.

In this work, we focus on functions that can be represented as arithmetic circuits over a finite field $\mathbb{F}$ with $|\mathbb{F}| \geq 2n$, consisting of input, addition, multiplication, and output gates. We adopt the standard simulation-based definition of MPC [Can00]. An adversary is able to corrupt at most $t = (n - 1)/3$ parties, provide inputs to corrupted parties, and receive all messages sent to the corrupted parties. Corrupted parties can deviate from the protocol arbitrarily. Our goal is to achieve perfect security.

3.2 Packed Secret Sharing via Bivariate Polynomials

A degree- (d, d) bivariate polynomial S over a finite field $\mathbb{F}$ is of the form $S(x, y) = \sum_{i=0}^d \sum_{j=0}^d a_{i,j} \cdot x^i y^j$, where $a_{i,j} \in \mathbb{F}$ for every $i, j \in \{0, 1, \dots, d\}$. Let k denote the packing parameter. Let $\alpha_1, \dots, \alpha_n$ and $\beta_1, \dots, \beta_k$ be $n + k$ distinct field elements.

The sharing and reconstruction algorithms of a degree- (d, d) packed secret sharing via bivariate polynomials with a $k \times k$ matrix of k^2 secrets $\mathbf{X} = (x_{ij})_{ij} \in \mathbb{F}^{k \times k}$ are defined as follows:

Share($\mathbf{X}$): Sample a random degree- (d, d) bivariate polynomial $S(x, y)$ such that $S(\beta_i, \beta_j) = x_{ij}$ for every $i, j \in [k]$. Then i -th share is defined as two degree- d polynomials $(f_i(x), g_i(y))$, where $f_i(x) := S(x, \alpha_i)$ and $g_i(y) := S(\alpha_i, y)$. Output $\{f_i(x), g_i(y)\}_{i \in [n]}$.

Recon($\mathcal{I}, \{f_i(x), g_i(y)\}_{i \in \mathcal{I}}$):

- If $|\mathcal{I}| < d + 1$, output $\perp$.
- If $|\mathcal{I}| \geq d + 1$, let $\mathcal{I}' \subset \mathcal{I}$ such that $|\mathcal{I}'| = d + 1$. For $j \in [k]$, compute $g_i(\beta_j)$ for every $i \in \mathcal{I}'$, interpolate $S(x, \beta_j)$, and compute $S(\beta_\ell, \beta_j)$ for every $\ell \in [k]$. Output $\mathbf{X} \leftarrow (S(\beta_\ell, \beta_j))_{\ell, j \in [k]}$.

We use $\llbracket \mathbf{X} \rrbracket_d$ to denote such a degree- (d, d) packed Shamir secret sharing over bivariate polynomials with secret $\mathbf{X} \in \mathbb{F}^{k \times k}$. Similar to standard Shamir secret sharings, packed Shamir sharings over bivariate polynomials also satisfy two properties:

- Linearly homomorphic. For all $\mathbf{X}, \mathbf{Y} \in \mathbb{F}^{k \times k}$, $\llbracket \mathbf{X} + \mathbf{Y} \rrbracket_d = \llbracket \mathbf{X} \rrbracket_d + \llbracket \mathbf{Y} \rrbracket_d$.
- Multiplicative. Let $*$ denote coordinate-wise multiplication. For all $\mathbf{X}, \mathbf{Y} \in \mathbb{F}^{k \times k}$ and $k-1 \leq d_1, d_2 \leq n-1$ such that $d_1 + d_2 \leq n-1$, $\llbracket \mathbf{X} * \mathbf{Y} \rrbracket_{d_1+d_2} = \llbracket \mathbf{X} \rrbracket_{d_1} \cdot \llbracket \mathbf{Y} \rrbracket_{d_2}$.

These two properties directly follow from the computation of the underlying bivariate polynomials.

The following proposition regarding degree- (d, d) bivariate polynomials implies two useful properties of degree- (d, d) packed Shamir sharings: 1) the whole sharing is fully determined by $d + 1$ parties' shares when pairwise consistency is assumed and 2) any set of shares of $d + 1 - k$ parties gives no information about the secret, which are stated formally as two claims below.

Proposition 1. *For $d+1$ distinct field elements $\alpha_1, \dots, \alpha_d, \alpha_{d+1}$, let $f_1(x), \dots, f_{d+1}(x)$ be arbitrary degree- d polynomials. Then there exists a unique degree- (d, d) bivariate polynomial $S(x, y)$ such that for each $i \in [d+1]$, $S(x, \alpha_i) = f_i(x)$.*

Proof. Considering the existence, define a degree- (d, d) bivariate polynomial $S(x, y)$ as $S(x, y) = \sum_{i=1}^{d+1} f_i(x) \cdot \prod_{j \in [d], j \neq i} \frac{y - \alpha_j}{\alpha_i - \alpha_j}$. We claim that $S(x, \alpha_r) \equiv f_r(x)$ for every $r \in [d+1]$.

In fact, for every $r \in [d+1]$,

$$\begin{aligned} S(x, \alpha_r) &= \sum_{i \in [d+1]} f_i(x) \prod_{j \in [d+1], j \neq i} \frac{\alpha_r - \alpha_j}{\alpha_i - \alpha_j} \\ &= f_r(x) + \sum_{i \in [d+1], i \neq r} f_i(x) \prod_{j \in [d+1], j \neq i} \frac{\alpha_r - \alpha_j}{\alpha_i - \alpha_j} \\ &= f_r(x). \end{aligned}$$

Now we focus on the uniqueness of S . For the sake of contradiction, suppose there exists a degree- (d, d) bivariate polynomial $S' \neq S$ such that $S'(x, \alpha_i) = f_i(x)$ for every $i \in [d+1]$. Let $R(x, y) = S(x, y) - S'(x, y)$. Since $R(x, y)$ is a degree- (d, d) bivariate polynomial, there exists degree- d polynomials $r_0(y), \dots, r_d(y)$ such that $R(x, y) = \sum_{j=0}^d r_j(y) \cdot x^j$. Then for every $i \in [d+1]$, $R(x, \alpha_i) = S(x, \alpha_i) - S'(x, \alpha_i) = f_i(x) - f_i(x) = 0$, which implies $\sum_{j=0}^d r_j(\alpha_i) \cdot \alpha_i^j = 0$. Hence, for every $i \in [d+1]$, $r_j(\alpha_i) = 0$ holds for every $j \in \{0, 1, \dots, d\}$. Since for every $j \in \{0, 1, \dots, d\}$, $r_j(\cdot)$ is a degree- d polynomial, we have $r_j(y) \equiv 0$. Therefore, $R(x, y) \equiv 0$, which leads to a contradiction. $\square$

Claim (Pairwise consistency check). Let $\mathcal{I} \subset [n]$ such that $|\mathcal{I}| \geq d + 1$. Let $\{f_i(x), g_i(y)\}_{i \in \mathcal{I}}$ be a set of pairs of degree- d polynomials. If for every $i, j \in \mathcal{I}$, it holds that $f_i(\alpha_j) = g_j(\alpha_i)$, then there exists a unique degree- (d, d) bivariate polynomial S such that $f_i(x) = S(x, \alpha_i)$ and $g_i(y) = S(\alpha_i, y)$ for every $i \in \mathcal{I}$.

Proof. Let $\mathcal{I}' \subset [n]$ such that $\mathcal{I} \subseteq \mathcal{I}'$ and $|\mathcal{I}'| = d + 1$. Considering the existence, by Proposition 1, there exists a degree- (d, d) polynomial $S(x, y)$ such that $S(x, \alpha_i) = f_i(x)$ for every $i \in \mathcal{I}'$. Then we prove $S(x, y)$ satisfies the requirement. In fact, for every $j \in \mathcal{I}$,

$$\forall i \in \mathcal{I}', g_j(\alpha_i) = f_i(\alpha_j) = S(\alpha_j, \alpha_i),$$

which implies $g_j(y) \equiv S(\alpha_j, y)$. For every $j \in \mathcal{I}$,

$$\forall i \in \mathcal{I}', f_j(\alpha_i) = g_i(\alpha_j) = S(\alpha_i, \alpha_j),$$

which implies $f_j(x) \equiv S(x, \alpha_j)$. The uniqueness of S follows from the uniqueness in Proposition 1. $\square$

Claim (Secrecy lemma). Let $\mathcal{I} \subset [n]$ such that $|\mathcal{I}| \leq d+1-k$. For any two $k \times k$ matrices of k^2 secrets $\mathbf{X}_1, \mathbf{X}_2 \in \mathbb{F}^{k \times k}$, then

$$\{f_i(x), g_i(y)\}_{i \in \mathcal{I}}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}(\mathbf{X}_1) \equiv \{f_i(x), g_i(y)\}_{i \in \mathcal{I}}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}(\mathbf{X}_2).$$

Proof. Let $\mathcal{I}' \subset [n]$ be subset of size $d+1-k$ such that $\mathcal{I} \subseteq \mathcal{I}'$. The last claim implies there is a one-to-one correspondence between a degree- (d, d) bivariate polynomial and a set of $(d+1)$ pairs of consistent degree- d polynomials $\{f_i(x), g_i(y)\}_{i \in [d+1]}$, which further implies a one-to-one correspondence between a degree- (d, d) bivariate polynomial and a set of $(d+1-k)$ pairs of consistent degree- d polynomials plus a $k \times k$ matrix defining the secret $\{f_i(x), g_i(y)\}_{i \in \mathcal{I}' \cup \mathbf{X}}$.

Based on this correspondence, we consider the following process of sharing a secret $\mathbf{X} = (X_{ij})_{ij}$:

Share'($\mathbf{X}$): sample random $d+1-k$ pairs of degree- d polynomials as $\{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}$ such that $f_i(\alpha_j) = g_j(\alpha_i)$ for every $i, j \in \mathcal{I}'$, compute a degree- (d, d) bivariate polynomial such that

- for $i \in \mathcal{I}'$, $S(x, \alpha_i) = f_i(x)$ and $S(\alpha_i, y) = g_i(y)$;
- for $i, j \in [k]$, $S(\beta_i, \beta_j) = X_{ij}$,

and output $\{f_i(x), g_i(y)\}_{i \in [n]}$, where $f_i(x) := S(x, \alpha_i)$ and $g_i(y) := S(\alpha_i, y)$.

Then given $\mathbf{X} = (X_{ij})_{ij} \in \mathbb{F}^{k \times k}$, there is a one-to-one correspondence between a degree- (d, d) bivariate polynomial S satisfying $S(\beta_i, \beta_j) = X_{ij}$ for $i, j \in [k]$ and a set of $(d+1-k)$ pairs of consistent degree- d polynomials $\{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}$, which implies

$$\{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}(\mathbf{X}) \equiv \{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}'(\mathbf{X}).$$

Notice that the view of parties in $\mathcal{I}'$ generated by **Share'($\mathbf{X}$)** is independent of the secret $\mathbf{X}$. More precisely, we have that

$$\begin{aligned} & \{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}(\mathbf{X}_1) \\ & \equiv \{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}'(\mathbf{X}_1) \\ & \equiv \{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}'(\mathbf{X}_2) \\ & \equiv \{f_i(x), g_i(y)\}_{i \in \mathcal{I}'}, \{f_i(x), g_i(y)\}_{i \in [n]} \leftarrow \text{Share}(\mathbf{X}_2), \end{aligned}$$

which implies the statement. $\square$

3.3 Byzantine Agreement

Our MPC protocol uses Byzantine agreement for both broadcast and consensus. Broadcast channels are implemented over authenticated and synchronous which allow a sender to distribute a message with a guarantee that all parties will receive the same value. Consensus allows parties, each holding an input x_i , to agree on a value x' such that $x' = x$ if all honest parties hold the same input $x_i = x$. With $t < n/3$, both consensus and broadcast can be achieved by a perfect byzantine agreement protocol communicating $O(n^2)$ bits [BGP92, CW92]. Alternatively, we can use an expected constant-round broadcast protocol in [AC24] which achieves communication complexity of $O(n \cdot L)$ bits plus expected $O(n^3 \cdot \log^2 n)$ bits for broadcasting a message of L bits.

We denote the communication complexity of a protocol by $\text{P2P}(M) + N_1 \times \text{BA}(L_1) + N_2 \times \text{BC}(L_2)$, which means the protocol costs M elements in total over the point-to-point private channels, calls the Byzantine agreement N_1 times to reach an agreement on a message of L_1 bits, and calls the broadcast channel N_2 times with a message of L_2 bits.

3.4 Party Elimination Framework

To achieve perfect security, we follow the party elimination framework [HMP00]. In this framework, the computation task is uniformly divided into n segments which are evaluated sequentially. In each segment, parties first evaluate the segment and then verify the computation. After the verification, all active parties reach an agreement on whether the segment is successfully evaluated. In the case of success, all parties proceed to evaluate the next segment. In the case of failure, a new disputed pair containing at least one corrupted party is identified, and will be eliminated. The rest of the parties will re-evaluate the current segment. Since each failure would lead to a reduction on the number of active corrupted parties, the failure can occur at most $O(n)$ times.

We use $\mathcal{P}_{\text{active}}$ to denote the set of all active parties in the current segment. Let n' denote the number of active parties, i.e., $n' = |\mathcal{P}_{\text{active}}|$. Let $\mathcal{C}$ denote the set of corrupted parties and $\mathcal{H}$ denote the set of honest parties. Let t' denote the number of active corrupted parties, i.e., $t' = |\mathcal{C} \cap \mathcal{P}_{\text{active}}|$. In the beginning, all parties initialize $\mathcal{P}_{\text{active}} \leftarrow \{P_1, \dots, P_n\}$. Each time a fault occurs, a disputes pair $E \subset \mathcal{P}_{\text{active}}$ ⁴ with $E \cap \mathcal{C} \neq \emptyset$ is identified. All parties update $\mathcal{P}_{\text{active}} \leftarrow \mathcal{P}_{\text{active}} \setminus E$, $n' \leftarrow n' - |E|$, and $t' \leftarrow t' - |E \cap \mathcal{C}|$. Then we have the invariant that $n' - 2t' \geq n - 2t \geq t + 1$.

Importantly, as mentioned in Section 2, we require at least $t + 2k - 1$ honest parties are active so that a degree- $(t + 2k - 2, t + 2k - 2)$ packed Shamir sharing is fully determined by the shares of honest parties. Setting $k = n/12$, if the number of eliminated parties is fewer than $n/3$, all parties continue the party elimination framework. However, if a number of $n/3$ parties have been eliminated, the corruption ratio for the remaining parties is reduced to at most $1/4$. The parties will restart the computation by invoking an MPC protocol achieving perfect security against a fully malicious adversary controlling at most $1/4$ fraction of parties.

3.5 Building Blocks

We list some building blocks that are useful for our constructions. Most of the functionalities are presented in the party elimination framework.

$\mathcal{F}_{\text{rand-sharing}}(\Sigma, N)$ aims to verifiably prepare random Σ -sharings. We follow [BH08] to instantiate this functionality. In particular, to prepare every group of $n' - 2t'$ Σ -sharings, each party first distributes a random Σ -sharing to all parties. Then all parties compute n' random Σ -sharings by applying a hyper-invertible matrix on original n' sharings and send their shares of $2t'$ of them to the first $2t'$ parties. These $2t'$ parties check the consistency of the sharings, where at least t' Σ -sharings are checked by honest parties. By property of the hyper-invertible matrix, the rest of $n' - 2t'$ Σ -sharings are linear combinations of $n' - t'$ Σ -sharings distributed by honest parties and t' Σ -sharings checked by honest parties. Then the $n' - 2t'$ Σ -sharings are guaranteed to be consistent.

The communication cost to prepare N Σ -sharings is $\text{P2P}(O(N \cdot n' + (n')^2) \cdot |\Sigma|) + O(1) \times \text{BA}(1) + O(1) \times \text{BC}(O(\log N + \log n' + \log |\Sigma| + \log |\mathbb{F}|))$, amounting to $O(n' \cdot |\Sigma|)$ elements per Σ -sharing, where $|\Sigma|$ denotes the share size of a Σ -sharing.

Functionality 1: $\mathcal{F}_{\text{rand-sharing}}(\Sigma, N)$

$\mathcal{F}_{\text{rand-sharing}}$ prepares a number of $(n' - 2t')$ the following kinds of random linear secret sharing for all active parties.

- Σ_1 -sharing: a degree- $(t + 2k - 2)$ packed Shamir sharing $[\mathbf{o}]_{t+2k-2}$ with secret $\mathbf{o} = \mathbf{0} \in \mathbb{F}^k$.
- Σ_2 -sharing: a degree- $(t + 2k - 2, t + 2k - 2)$ packed Shamir sharing $[\mathbf{O}]_{t+2k-2}$ with secret $\mathbf{O} = \mathbf{0} \in \mathbb{F}^{k \times k}$.
- Σ_3 -sharing: a degree- $(t + k - 1, t + k - 1)$ packed Shamir sharing $[\mathbf{R}]_{t+k-1}$ together with degree- t Shamir sharing for each entry $[r_{ij}|j]_t$, where $\mathbf{R} = (r_{ij})_{ij}$, and $[r|j]_t$ denotes a random degree- t Shamir sharing with the secret r stored at the j -th position β_j .
- Σ_4 -sharing: a random degree- t Shamir sharing $[r]_t$.

⁴ In our construction, sometimes a disputed pair only contains a corrupted party.

- Σ_5 -sharing: a random degree- $(t + k - 1, t + k - 1)$ packed Shamir sharing $[\mathbf{R}]_{t+k-1}$.
 - Σ_6 -sharing: a tuple of three degree- $(t + k - 1, t + k - 1)$ packed Shamir sharing in the form of $[\mathbf{O}]_{t+k-1}$ with secret $\mathbf{O} = \mathbf{0} \in \mathbb{F}^{k \times k}$.
1. $\mathcal{F}_{\text{rand-sharing}}$ receives the set $\mathcal{C}$ of corrupted parties' identities, the set $\mathcal{P}_{\text{active}}$ of all active parties' identities, and a public parameter N .
 2. For each $\ell \in [N]$, $\mathcal{F}_{\text{rand-sharing}}$ receives active corrupted parties' shares $\{u_j\}_{j \in \mathcal{C} \cap \mathcal{P}_{\text{active}}}$ from the adversary, samples a random Σ -sharing such that for all $j \in \mathcal{C} \cap \mathcal{P}_{\text{active}}$, the j -th share of the sharing is u_j .
 3. $\mathcal{F}_{\text{rand-sharing}}$ waits for an instruction from the adversary.
 - If $\mathcal{F}_{\text{rand-sharing}}$ receives **accept** from the adversary, $\mathcal{F}_{\text{rand-sharing}}$ distributes the shares to all active parties.
 - If $\mathcal{F}_{\text{rand-sharing}}$ receives **reject** from the adversary, $\mathcal{F}_{\text{rand-sharing}}$ receives a new disputed pair $E \subset \mathcal{P}_{\text{active}}$ containing at least one corrupted party from the adversary, and outputs **abort** together with E to all active parties.

$\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma, N)$ targets to check the consistency of a number of N Σ -sharings, where the whole Σ -sharing is fully determined by shares of honest parties. This functionality is not invoked in the framework of party elimination. We also follow [BH08] to instantiate this functionality. Concretely, for every group of $n - t$ Σ -sharings, all parties hold their shares of $n - t$ Σ -sharings, apply a Vandermatrix to compute n Σ -sharings which are linear combinations of original $n - t$ Σ -sharings, and reconstruct each of n sharings to a different party, who checks the consistency of the sharing. If all checks pass, this implies $n - t$ Σ -sharings checked by $n - t$ honest parties are consistent. Since there is a one-to-one correspondence between the original $n - t$ Σ -sharings and $n - t$ Σ -sharings checked by honest parties by property of Vandermonde matrix, the original $n - t$ Σ -sharings are guaranteed to be consistent.

The communication cost to check N Σ -sharings is $\text{P2P}(O(N \cdot n + n^2) \cdot |\Sigma|) + O(1) \times \text{BA}(1)$, amounting to $O(n \cdot |\Sigma|)$ elements per Σ -sharing, where $|\Sigma|$ denotes the share size of a Σ -sharing.

Functionality 2: $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma, N)$

$\mathcal{F}_{\text{vrfy-linear-with-abort}}$ verifies whether the shares of all (honest) parties jointly form the following kinds of consistent linear secret sharings.

- Σ_1 -sharing: a degree- $(t + 2k - 2)$ packed Shamir sharing $[\mathbf{o}]_{t+2k-2}$ with secret $\mathbf{o} = \mathbf{0} \in \mathbb{F}^k$.
 - Σ_7 -sharing: a pair of packed Shamir sharings in the form of $([\mathbf{u}]_{t+k-1}, [\mathbf{u}]_{k-1})$, where the first degree- $(t + k - 1)$ packed Shamir sharing and the second degree- $(k - 1)$ packed Shamir sharing have the same secrets.
1. $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ receives the set $\mathcal{C}$ of corrupted parties' identities.
 2. $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ receives honest parties' shares of $U_1, \dots, U_N$ from the honest parties and sends them to the adversary.
 3. $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ checks if each group of shares form a valid Σ -sharing.
 - If all checks pass, $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ receives an instruction from the adversary:
 - If $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ receives **reject** from the adversary, it outputs 0 to all parties.
 - If $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ receives **accept** from the adversary, it outputs 1 to all parties.
 - If one of the checks fails, $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ outputs 0 to all parties.

$\mathcal{F}_{\text{batched-vrfy}}(P_r, \Sigma, N)$ targets to check the consistency of a number of N Σ -sharings which are distributed by an active party P_r . Importantly, in case of inconsistency, $\mathcal{F}_{\text{batched-vrfy}}$ will receive a disputed pair from the adversary and output it to all parties, which makes it different from $\mathcal{F}_{\text{vrfy-linear-with-abort}}$. The realization

of $\mathcal{F}_{\text{batched-vrfy}}$ closely follows that of $\mathcal{F}_{\text{vrfy-linear-with-abort}}$, except that in the case of an inconsistency, P_r is responsible for identifying a disputed pair.

The communication to check N Σ -sharings is $\text{P2P}(O(N \cdot n' + (n')^2) \cdot |\Sigma|) + O(1) \times \text{BA}(1) + O(1) \times \text{BC}(O(\log N + \log n' + \log |\Sigma| + \log |\mathbb{F}|))$, amounting $O(n' \cdot |\Sigma|)$ elements per Σ -sharing, where $|\Sigma|$ denotes the share size of a Σ -sharing.

Functionality 3: $\mathcal{F}_{\text{batched-vrfy}}(P_r, \Sigma, N)$

$\mathcal{F}_{\text{batched-vrfy}}$ checks whether all active (honest) parties hold N valid Σ -sharings distributed by an active party P_r .

- Σ -sharing: a pair of packed Shamir sharings in the form of $(\llbracket U \rrbracket_{k-1}, \llbracket U \rrbracket_{t+2k-2})$, where the first degree- $(k-1, k-1)$ packed Shamir sharing and the second degree- $(t+2k-2, t+2k-2)$ packed Shamir sharing have the same secrets $U \in \mathbb{F}^{k \times k}$.
- 1. $\mathcal{F}_{\text{batched-vrfy}}$ receives the set $\mathcal{C}$ of corrupted parties' identities, the set $\mathcal{P}_{\text{active}}$ of active parties, and a public parameter N .
- 2. $\mathcal{F}_{\text{batched-vrfy}}$ receives honest parties' shares of $U_1, \dots, U_N$ from the honest parties and sends them to the adversary.
- 3. $\mathcal{F}_{\text{batched-vrfy}}$ checks if each group of shares forms a valid Σ -sharing.
 - If all checks pass, $\mathcal{F}_{\text{batched-vrfy}}$ receives an instruction from the adversary:
 - If $\mathcal{F}_{\text{batched-vrfy}}$ receives **reject** from the adversary, it receives a disputed pair $E \subset \mathcal{P}_{\text{active}}$ containing at least one corrupted party, and outputs **abort** together with E to all parties.
 - If $\mathcal{F}_{\text{batched-vrfy}}$ receives **accept** from the adversary, it outputs **accept** to all parties.
 - If one of the checks fails, $\mathcal{F}_{\text{batched-vrfy}}$ receives a disputed pair $E \subset \mathcal{P}_{\text{active}}$ containing at least one corrupted party, and outputs **abort** together with E to all active parties.

$\mathcal{F}_{\text{triple}}(N)$ targets to verifiably prepare a number of N Beaver triples of degree- t with secrets for all active parties. We follow [BH08] to realize this functionality. In particular, all parties first invoke $\mathcal{F}_{\text{rand-sharing}}$ to verifiably prepare random degree- t Shamir sharings with secrets as $[a]_t$ and $[b]_t$ and a pair of double sharings $([r]_t, [r]_{2t})$. They locally compute $[e]_{2t} \leftarrow [a]_t \cdot [b]_t + [r]_{2t}$ and publicly and detectably reconstruct the secret e . All parties locally compute $[c]_t \leftarrow [e]_0 - [r]_t$, where $c = a \cdot b$.

The communication cost to prepare N Beaver triples of degree- t is $\text{P2P}(O(N \cdot n' + (n')^2)) + O(1) \times \text{BA}(1) + O(1) \times \text{BC}(O(\log N + \log n' + \log |\mathbb{F}|))$, amounting to $O(n')$ elements per triple.

Functionality 4: $\mathcal{F}_{\text{triple}}(N)$

$\mathcal{F}_{\text{triple}}(N)$ prepares N Beaver triples for all active parties.

- 1. $\mathcal{F}_{\text{triple}}$ receives the set $\mathcal{C}$ of corrupted parties' identities, the set $\mathcal{P}_{\text{active}}$ of active parties' identities, and a public parameter N .
- 2. For each $\ell \in [N]$, $\mathcal{F}_{\text{triple}}$ receives corrupted parties' shares $\{u_j^{(1)}, u_j^{(2)}, u_j^{(3)}\}_{j \in \mathcal{C} \cap \mathcal{P}_{\text{active}}}$. $\mathcal{F}_{\text{triple}}$ randomly samples two elements a, b , computes $c \leftarrow a \cdot b$, and samples random degree- t Shamir sharings $([a]_t, [b]_t, [c]_t)$ such that for all $j \in \mathcal{C} \cap \mathcal{P}_{\text{active}}$, the j -th share of $([a]_t, [b]_t, [c]_t)$ is $(u_j^{(1)}, u_j^{(2)}, u_j^{(3)})$.
- 3. $\mathcal{F}_{\text{triple}}$ waits for an instruction from the adversary:
 - If $\mathcal{F}_{\text{triple}}$ receives **accept** from the adversary, then for each $\ell \in [N]$, $j \in \mathcal{P}_{\text{active}}$, $\mathcal{F}_{\text{triple}}$ sends to P_j its shares of $([a]_t, [b]_t, [c]_t)$.
 - If $\mathcal{F}_{\text{triple}}$ receives **reject** from the adversary, $\mathcal{F}_{\text{triple}}$ receives a new disputed pair $E \subset \mathcal{P}_{\text{active}}$ containing at least one corrupted party from the adversary, and outputs **abort** together with E to all active parties.

4 Perfectly Secure MPC in the Preprocessing Model

In this section, we will construct a perfectly secure MPC in the preprocessing model. For simplicity, we consider SIMD circuits, where the computation involves applying the same circuit to a batch of $O(n^2)$ inputs. We first introduce our construction of preprocessing data of size $O(|C|)$ and then design an online protocol consuming the preprocessing data to compute the circuit.

4.1 Preprocessing Data

Following [GPS22], our construction requires preprocessing data of packed Beaver triples. With packed Beaver triples at hand, the computation of multiplication only involves degree reduction. As mentioned in Section 2, to identify a disputed pair when faults occur, we make use of secret sharing via bivariate polynomials, which allows a party to verify its shares by checking common points jointly held by another party. We show the preprocessing functionality in $\mathcal{F}_{\text{packed-triple}}(N, k)$, which prepares N packed Beaver triples over bivariate polynomials with each packing k^2 secrets.

We emphasize that $\mathcal{F}_{\text{packed-triple}}$ achieves security with identifiable abort, in which a disputed pair is guaranteed to be identified when the functionality decides to abort.

Functionality 5: $\mathcal{F}_{\text{packed-triple}}(N, k)$

1. $\mathcal{F}_{\text{packed-triple}}$ receives the set $\mathcal{C}$ of corrupted parties' identities, the set $\mathcal{P}_{\text{active}}$ of active parties' identities, and a public parameter N .
2. $\mathcal{F}_{\text{packed-triple}}$ receives from the adversary shares of corrupted parties $\{f_{\ell,i}^a(x), g_{\ell,i}^a(y), f_{\ell,i}^b(x), g_{\ell,i}^b(y), f_{\ell,i}^c(x), g_{\ell,i}^c(y)\}_{\ell \in [N], i \in \mathcal{C} \cap \mathcal{P}_{\text{active}}}$ with a promise that
 - $f_{\ell,i}^z(x), g_{\ell,i}^z(y)$ are degree- $(t+k-1)$ polynomials for $\ell \in [N], i \in \mathcal{C} \cap \mathcal{P}_{\text{active}}, z \in \{a, b, c\}$,
 - $f_{\ell,i}^z(\alpha_j) = g_{\ell,j}^z(\alpha_i)$ for every $i, j \in \mathcal{C} \cap \mathcal{P}_{\text{active}}, z \in \{a, b, c\}$.
3. For $\ell \in [N]$, $\mathcal{F}_{\text{packed-triple}}$ randomly samples a packed triple over bivariate polynomials $([\mathbf{A}_\ell]_{t+k-1}, [\mathbf{B}_\ell]_{t+k-1}, [\mathbf{C}_\ell]_{t+k-1})$, where $\mathbf{C}_\ell = \mathbf{A}_\ell * \mathbf{B}_\ell$, $\mathbf{A}_\ell = (A_{\ell,ij})_{ij}$, $\mathbf{B}_\ell = (B_{\ell,ij})_{ij}$, and $\mathbf{C}_\ell = (C_{\ell,ij})_{ij}$, according to shares of corrupted parties. Concretely, $\mathcal{F}_{\text{packed-triple}}$ samples three degree- $(t+k-1, t+k-1)$ bi-variate polynomials $S_\ell^a(x, y), S_\ell^b(x, y), S_\ell^c(x, y)$ such that
 - $S_\ell^a(\beta_i, \beta_j) = A_{\ell,ij}$, $S_\ell^b(\beta_i, \beta_j) = B_{\ell,ij}$, and $S_\ell^c(\beta_i, \beta_j) = C_{\ell,ij}$ for $i, j \in [k]$,
 - $S_\ell^z(x, \alpha_i) = f_{\ell,i}^z(x)$, $S_\ell^z(\alpha_i, y) = g_{\ell,i}^z(y)$ for $i \in \mathcal{C} \cap \mathcal{P}_{\text{active}}, z \in \{a, b, c\}$.
4. $\mathcal{F}_{\text{packed-triple}}$ waits for an instruction from the adversary.
 - If $\mathcal{F}_{\text{packed-triple}}$ receives **accept** from the adversary, then for each $\ell \in [N], i \in \mathcal{P}_{\text{active}}, z \in \{a, b, c\}$, $\mathcal{F}_{\text{packed-triple}}$ computes $f_{\ell,i}^z(x) := S_\ell^z(x, \alpha_i)$, $g_{\ell,i}^z(y) := S_\ell^z(\alpha_i, y)$ and sends to P_i $\{f_{\ell,i}^z(x), g_{\ell,i}^z(y)\}_{\ell \in [N], z \in \{a, b, c\}}$ as its shares of $\{[\mathbf{A}_\ell]_{t+k-1}, [\mathbf{B}_\ell]_{t+k-1}, [\mathbf{C}_\ell]_{t+k-1}\}_{\ell \in [N]}$.
 - If $\mathcal{F}_{\text{packed-triple}}$ receives **reject** from the adversary, $\mathcal{F}_{\text{packed-triple}}$ receives a new disputed pair $E \subset \mathcal{P}_{\text{active}}$ containing at least one corrupted party from the adversary, and outputs **abort** together with E to all active parties.

4.2 Computation Phase

In the computation phase, all parties hold shares of input wires $[\mathbf{X}]_{t+k-1}$ and $[\mathbf{Y}]_{t+k-1}$, and aim to compute their shares of output wires $[\mathbf{Z}]_{t+k-1}$. For a group of k^2 addition gates, all parties locally compute $[\mathbf{Z}]_{t+k-1} \leftarrow [\mathbf{X}]_{t+k-1} + [\mathbf{Y}]_{t+k-1}$. For a group of k^2 multiplication gates, all parties follow Beaver multiplication protocol by consuming a packed Beaver triple $([\mathbf{A}]_{t+k-1}, [\mathbf{B}]_{t+k-1}, [\mathbf{C}]_{t+k-1})$. To perform degree reduction, a double sharing $([\mathbf{R}]_{t+k-1}, [\mathbf{R}]_{t+2k-2})$ is required, which is prepared by picking one sharing of an unused triple as $[\mathbf{R}]_{t+k-1}$ and one random degree- $(t+2k-2, t+2k-2)$ packed Shamir sharing $[\mathbf{O}]_{t+2k-2}$ generated in the preprocessing phase, and by computing $[\mathbf{R}]_{t+2k-2} \leftarrow [\mathbf{R}]_{t+k-1} + [\mathbf{O}]_{t+2k-2}$. Moreover, to prevent so-called double dipping attack [GBO⁺23, DEN24], first pointed out by [GLS19], due to cross-layer computation without verification, we follow a solution proposed in [SY25b]. In this approach, before the

computation, all parties first prepare degree- $(n' - 1, n' - 1)$ packed Shamir sharings in the form of $\llbracket \mathbf{O} \rrbracket_{n'-1}$ with secret $\mathbf{O} = \mathbf{0} \in \mathbb{F}^{k \times k}$, then they add these high-degree packed Shamir sharings as masks to protect privacy of honest parties against a corrupted P_{king} . Concretely, the protocol works as follows.

1. All parties locally compute $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{n'-1} \leftarrow \llbracket \mathbf{X} \rrbracket_{t+k-1} + \llbracket \mathbf{A} \rrbracket_{t+k-1} + \llbracket \mathbf{O}_1 \rrbracket_{n'-1}$, $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{n'-1} \leftarrow \llbracket \mathbf{Y} \rrbracket_{t+k-1} + \llbracket \mathbf{B} \rrbracket_{t+k-1} + \llbracket \mathbf{O}_2 \rrbracket_{n'-1}$, and send their shares to P_{king} .
2. P_{king} reconstructs the secrets $\mathbf{X} + \mathbf{A}$, $\mathbf{Y} + \mathbf{B}$, and distributes $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1}$, $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1}$ to all parties.
3. All parties locally compute

$$\begin{aligned} \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2} &\leftarrow \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} - \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{B} \rrbracket_{t+k-1} \\ &\quad - \llbracket \mathbf{A} \rrbracket_{t+k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} + \llbracket \mathbf{C} \rrbracket_{t+k-1} + \llbracket \mathbf{R} \rrbracket_{t+2k-2}. \end{aligned}$$

All parties then compute $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{n'-1} \leftarrow \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2} + \llbracket \mathbf{O}_3 \rrbracket_{n'-1}$ and send their shares to P_{king} .

4. P_{king} reconstructs the secret $\mathbf{Z} + \mathbf{R}$ and distributes $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$ to all parties who locally compute $\llbracket \mathbf{Z} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1} - \llbracket \mathbf{R} \rrbracket_{t+k-1}$.

However, malicious adversary may deviate from the protocol, leading to the following issues.

- In Step 1 or Step 3, corrupted parties can send incorrect shares to P_{king} .
- In Step 2 or Step 4, corrupted P_{king} can distribute inconsistent degree- $(k - 1, k - 1)$ packed Shamir sharings.
- In Step 2 or Step 4, corrupted P_{king} can distribute degree- $(k - 1, k - 1)$ packed Shamir sharings with incorrect secrets.

We first present the protocol for preparing random degree- $(n' - 1, n' - 1)$ packed Shamir sharings with secret $\mathbf{0} \in \mathbb{F}^{k \times k}$ in $\Pi_{\text{zero-sharing}}$ and the protocol for computing multiplication in Π_{mult} , and add verification to resolve the above issues in the next part.

The communication cost of $\Pi_{\text{zero-sharing}}$ to prepare $n' - t'$ sharings is $\text{P2P}(2(n')^3)$ elements. The communication cost of Π_{mult} to compute a group of k^2 multiplication gates is $\text{P2P}(6(n')^2 + 6k \cdot n')$ elements.

Protocol 1: $\Pi_{\text{zero-sharing}}$

All parties prepare $n' - t'$ random degree- $(n' - 1, n' - 1)$ packed Shamir sharing with secret $\mathbf{0} \in \mathbb{F}^{k \times k}$.

1. All parties agree on a Vandermonde matrix $\mathbf{M}^\top$ of size $n' \times (n' - t')$.
2. Each party P_i randomly samples a random degree- $(n' - 1, n' - 1)$ packed Shamir sharing $\llbracket \mathbf{O}^{(i)} \rrbracket_{n'-1}$ with secret $\mathbf{O}^{(i)} = \mathbf{0}$ and distributes the shares to other parties, where $\mathbf{o}$ is of dimension k .
3. All parties locally compute

$$(\llbracket \mathbf{O}_1 \rrbracket_{n'-1}, \dots, \llbracket \mathbf{O}_{n'-t'} \rrbracket_{n'-1})^\top = \mathbf{M} \cdot (\llbracket \mathbf{O}^{(1)} \rrbracket_{n'-1}, \dots, \llbracket \mathbf{O}^{(n')} \rrbracket_{n'-1})^\top.$$

Protocol 2: Π_{mult}

All parties hold their shares of $\llbracket \mathbf{X} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} \rrbracket_{t+k-1}$, and aim to compute their shares of $\llbracket \mathbf{X} * \mathbf{Y} \rrbracket_{t+k-1}$. In addition, all parties hold the following sharings prepared in the preprocessing phase.

- A packed Beaver triple $(\llbracket \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{B} \rrbracket_{t+k-1}, \llbracket \mathbf{C} \rrbracket_{t+k-1})$ such that $\mathbf{A} * \mathbf{B} = \mathbf{C}$.
- A pair of random packed Shamir sharings $(\llbracket \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{R} \rrbracket_{t+2k-2})$.

All parties also hold three degree- $(n' - 1, n' - 1)$ packed Shamir sharings with $\mathbf{0} \in \mathbb{F}^{k \times k}$ as secret $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}, \llbracket \mathbf{O}_2 \rrbracket_{n'-1}, \llbracket \mathbf{O}_3 \rrbracket_{n'-1}$.

1. All parties compute $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{n'-1} = \llbracket \mathbf{X} \rrbracket_{t+k-1} + \llbracket \mathbf{A} \rrbracket_{t+k-1} + \llbracket \mathbf{O}_1 \rrbracket_{n'-1}$ and $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{n'-1} = \llbracket \mathbf{Y} \rrbracket_{t+k-1} + \llbracket \mathbf{B} \rrbracket_{t+k-1} + \llbracket \mathbf{O}_2 \rrbracket_{n'-1}$, and send their shares to P_{king} .
2. P_{king} reconstructs $\mathbf{X} + \mathbf{A}$ and $\mathbf{Y} + \mathbf{B}$, and distributes $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1}, \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1}$ to all parties.

3. All parties locally compute

$$\begin{aligned} \llbracket \mathbf{Z} \rrbracket_{t+2k-2} \leftarrow & \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} - \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{B} \rrbracket_{t+k-1} \\ & - \llbracket \mathbf{A} \rrbracket_{t+k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} + \llbracket \mathbf{C} \rrbracket_{t+k-1}. \end{aligned}$$

All parties then locally compute $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{n'-1} \leftarrow \llbracket \mathbf{Z} \rrbracket_{t+2k-2} + \llbracket \mathbf{R} \rrbracket_{t+2k-2} + \llbracket \mathbf{O}_3 \rrbracket_{n'-1}$, and send their shares to P_{king} .

4. P_{king} reconstructs $\mathbf{Z} + \mathbf{R}$ and distributes $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$ to all parties.
5. All parties locally compute $\llbracket \mathbf{Z} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1} - \llbracket \mathbf{R} \rrbracket_{t+k-1}$.

4.3 Verification Phase

To deal with three issue listed above, we follow the approach in [SY25b]. Concretely, all parties first verify whether P_{king} distributes consistent degree- $(k-1, k-1)$ packed Shamir sharings in Step 2 and Step 4 to resolve the second issue. Relying on property of bivariate polynomials, we present a procedure in $\pi_{\text{check-consistency-all}}(P_r, \llbracket \mathbf{U} \rrbracket_d)$ that allows all parties to verify whether a party P_r distributes consistent degree- (d, d) packed Shamir sharings and locate a disputed pair in case of inconsistency.

To resolve the first issue, after making sure P_{king} indeed distributes consistent degree- $(k-1, k-1)$ packed Shamir sharings and thus does not mount a double dipping attack, it is safe to reveal parties' shares of degree- $(t+k-1, t+k-1)$ packed Shamir sharings $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1}$ to P_{king} , where the whole sharings are fully determined by shares of honest parties, providing redundancy for P_{king} to detect incorrect sharings. Making use of packed Shamir sharings over bivariate polynomials, we show a procedure in $\pi_{\text{check-consistency-one}}(P_r, \llbracket \mathbf{U} \rrbracket_d)$ that allow a party P_r to check the consistency of shares of $\llbracket \mathbf{U} \rrbracket_d$ with $d < t+2k-2$ receiving from other parties and locate a disputed pair in case of inconsistency. Similarly, P_{king} can check parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2}$.

Regarding the third issue, as P_{king} has already checked the consistency of the received packed Shamir sharings, it can be effectively viewed as that P_{king} distributes a pair of sharings $(\llbracket \mathbf{U} \rrbracket_{t+2k-2}, \llbracket \mathbf{U} \rrbracket_{k-1})$ to all parties. We make use of the batched verification technique in [BH08] to verify whether P_{king} distributes degree- $(k-1, k-1)$ packed Shamir sharings with correct secrets. The protocol for verifying computation of multiplication gates appears in $\Pi_{\text{vrfy-mult}}$. The protocol for computing a segment containing addition gates and multiplication gates appears in Π_{eval} .

The communication of $\Pi_{\text{vrfy-mult}}$ to verify the computation of a group of k^2 multiplication gates is $\text{P2P}(12(n')^2)$ elements in the case of success, and $\text{P2P}(12(n')^2 + 2(n')^3) + O(1) \times \text{BC}(O(\log n' + \log |\mathbb{F}|))$ in the case of failure. The communication cost of Π_{eval} to compute N multiplication gates is $\text{P2P}(\frac{27N \cdot n'^2}{k^2} + \frac{6N \cdot n'}{k} + 2n'^3) + O(1) \times \text{BC}(O(\log n' + \log |\mathbb{F}|))$ plus one invocation of $\mathcal{F}_{\text{batched-vrfy}}(\Sigma_8, 3N)$, amounting to $\text{P2P}(O(N + n'^3)) + O(1) \times \text{BA}(1) + O(1) \times \text{BC}(O(\log N + \log n' + \log |\mathbb{F}|))$.

Procedure 1: $\pi_{\text{check-consistency-all}}(P_r, \llbracket \mathbf{U} \rrbracket_d)$

All parties receive their shares of $\llbracket \mathbf{U} \rrbracket_d$ from an active party $P_r \in \mathcal{P}_{\text{active}}$, and check whether the shares of all (honest) parties form a valid degree- (d, d) packed Shamir sharing. In case of failure, a new disputed pair is figured out.

Suppose $\llbracket \mathbf{U} \rrbracket_d$ is shared via a degree- (d, d) bivariate polynomial $S(x, y)$ and P_r distributes shares $f_i(x) := S(x, \alpha_i)$ and $g_i(y) := S(\alpha_i, y)$ to each party P_i .

1. For every $P_i \in \mathcal{P}_{\text{active}}$, party P_i checks whether f_i, g_i are two degree- d polynomials and $f_i(\alpha_i) = g_i(\alpha_i)$.
 - If this check fails, P_i complains by broadcasting $(\text{Incorrect}, P_i)$. All parties take the disputed pair $\{P_r, P_i\}$ as output.
2. For every $(P_i, P_j) \in \mathcal{P}_{\text{active}} \times \mathcal{P}_{\text{active}}$ such that $P_i \neq P_j$, P_i sends $(f_i(\alpha_j), g_i(\alpha_j))$ to P_j . P_j checks whether $(f_i(\alpha_j), g_i(\alpha_j)) = (g_j(\alpha_i), f_j(\alpha_i))$.

- If this check fails, P_j complains by broadcasting $(\text{Inconsistent}, P_j, P_i, (f_j(\alpha_i), g_j(\alpha_i)), (f_i(\alpha_j), g_i(\alpha_j)))$.

- If P_j broadcasts values such that $f_j(\alpha_i) = g_i(\alpha_j)$ and $g_j(\alpha_i) = f_i(\alpha_j)$, all parties take the corrupted party $\{P_j\}$ as output.

Then P_r broadcasts the values it sent to P_j $(f_j(\alpha_i), g_j(\alpha_i))$, and P_i also broadcasts its values $(g_i(\alpha_j), f_i(\alpha_i))$ received from P_r . All parties figure out two parties $\{P_{i'}, P_{j'}\}$ broadcasting different values, take the disputed pair $\{P_{i'}, P_{j'}\}$ as output.

Procedure 2: $\pi_{\text{check-consistency-one}}(P_r, \llbracket U \rrbracket_d)$

P_r receives from all parties their shares of $\llbracket U \rrbracket_d$, and check whether the shares form valid degree- (d, d) packed Shamir sharing. In case of failure, a new disputed pair is figured out.

Suppose $\llbracket U \rrbracket_d$ is shared via a degree- (d, d) bivariate polynomial $S(x, y)$ and P_r receives shares $f_i(x) := S(x, \alpha_i)$ and $g_i(y) := S(\alpha_i, y)$ from each party P_i .

1. For every $i \in \mathcal{P}_{\text{active}}$, P_r checks whether $f_i(x)$ and $g_i(y)$ are degree- d polynomials such that $f_i(\alpha_i) = g_i(\alpha_i)$.
 - If this check fails, P_r complains by broadcasting $(P_i, \text{Corrupt})$. All parties take the disputed pair $\{P_r, P_i\}$ as output.
2. For every $(i, j) \in \mathcal{P}_{\text{active}} \times \mathcal{P}_{\text{active}}$ such that $i \neq j$, P_r checks whether $(f_i(\alpha_j), g_i(\alpha_j)) = (g_j(\alpha_i), f_j(\alpha_i))$.
 - If this check fails, P_r complains by broadcasting $(\text{Inconsistent}, (f_i(\alpha_j), g_i(\alpha_j)), (g_j(\alpha_i), f_j(\alpha_i)), P_i, P_j)$.
 - If P_r broadcasts values such that $f_i(\alpha_j) = f_j(\alpha_i)$ and $g_i(\alpha_j) = g_j(\alpha_i)$, all parties take the corrupted party $\{P_r\}$ as output.

Then both P_i and P_j broadcast their values they had sent to P_r . All parties figure out two parties $\{P_{i'}, P_{j'}\}$ broadcasting different values, take the disputed pair $\{P_{i'}, P_{j'}\}$ as output.

Protocol 3: $\Pi_{\text{vrfy-mult}}$

All parties aim to verify the computation of multiplying $\llbracket X \rrbracket_{t+k-1}$ and $\llbracket Y \rrbracket_{t+k-1}$. Suppose $\llbracket O_1 \rrbracket_{n'-1}, \llbracket O_2 \rrbracket_{n'-1}, \llbracket O_3 \rrbracket_{n'-1}$ are three degree- $(n' - 1)$ packed Shamir sharings with secret $\mathbf{0} \in \mathbb{F}^{k \times k}$.

1. All parties invoke $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket U \rrbracket_{k-1})$ for $U \in \{X + A, Y + B, Z + R\}$ to check whether P_{king} distributes valid degree- $(k - 1, k - 1)$ packed Shamir sharings $\llbracket X + A \rrbracket_{k-1}, \llbracket Y + B \rrbracket_{k-1}, \llbracket Z + R \rrbracket_{k-1}$.
 - In case of failure, all parties take the disputed pair E as output,.
2. All parties send their shares of $\llbracket O_1 \rrbracket_{n'-1}, \llbracket O_2 \rrbracket_{n'-1}, \llbracket O_3 \rrbracket_{n'-1}$ to P_{king} . P_{king} checks whether each group of shares form valid degree- $(n' - 1, n' - 1)$ packed Shamir sharing with secret $\mathbf{0} \in \mathbb{F}^{k \times k}$.
 - If this check fails, all parties send their transcripts produced in the invocation of $\Pi_{\text{zero-sharing}}$ to P_{king} .
 - P_{king} either broadcasts $(P_i, \text{Corrupt})$ to indicate P_i does not honestly follow the protocol,
 - or broadcasts (ℓ, v, v', P_i, P_j) to indicate P_i and P_j disagree on the ℓ -th message exchanges between them, where P_i should have sent v to P_j while P_j claims to have received $v' \neq v$.

Then P_i, P_j broadcast their messages sent to P_{king} .

In the first case, all parties take $E \leftarrow \{P_{\text{king}}, P_i\}$ as the disputed pair. In the second case, all parties figure out two parties $E \leftarrow \{P_{i'}, P_{j'}\}$ broadcasting different values and take a disputed pair E as output.

- If this check passes, P_{king} subtracts the degree- $(n' - 1, n' - 1)$ packed Shamir sharings of $\mathbf{0}$ from the sharings it received in Π_{mult} . I.e., $[\mathbf{X} + \mathbf{A}]_{t+k-1} \leftarrow [\mathbf{X} + \mathbf{A}]_{n'-1} - [\mathbf{O}_1]_{n'-1}$, $[\mathbf{Y} + \mathbf{B}]_{t+k-1} \leftarrow [\mathbf{Y} + \mathbf{B}]_{n'-1} - [\mathbf{O}_2]_{n'-1}$, and $[\mathbf{Z} + \mathbf{R}]_{t+2k-2} \leftarrow [\mathbf{Z} + \mathbf{R}]_{n'-1} - [\mathbf{O}_3]_{n'-1}$.
- 3. For each $\mathbf{U} \in \{\mathbf{X} + \mathbf{A}, \mathbf{Y} + \mathbf{B}, \mathbf{Z} + \mathbf{R}\}$, supposing all parties send their shares of $[\mathbf{U}]_d^a$ to P_{king} , P_{king} and all other parties invoke $\pi_{\text{check-consistency-one}}(P_{\text{king}}, [\mathbf{U}]_d)$ to check whether P_{king} receives valid degree- (d, d) packed Shamir sharing $[\mathbf{U}]_d$.
 - In case of failure, all parties take the disputed pair E as output.
- 4. All parties take as output their shares of $([\mathbf{X} + \mathbf{A}]_{k-1}, [\mathbf{X} + \mathbf{A}]_{t+k-1})$, $([\mathbf{Y} + \mathbf{B}]_{k-1}, [\mathbf{Y} + \mathbf{B}]_{t+k-1})$, and $([\mathbf{Z} + \mathbf{R}]_{k-1}, [\mathbf{Z} + \mathbf{R}]_{t+2k-2})$, which will be batched and verified to check whether P_{king} distributes degree- $(k - 1, k - 1)$ packed Shamir sharings with correct secrets.

^a $d = t + k - 1$ for $\mathbf{U} \in \{\mathbf{X} + \mathbf{A}, \mathbf{Y} + \mathbf{B}\}$ and $d = t + 2k - 2$ for $\mathbf{U} = \mathbf{Z} + \mathbf{R}$.

Protocol 4: Π_{eval}

All parties evaluate the segment containing at most N multiplication gates. All parties agree on an active party $P_{\text{king}} \in \mathcal{P}_{\text{active}}$.

1. All parties invoke $\lceil \frac{3N}{(n'-t') \cdot k^2} \rceil$ times of $\Pi_{\text{zero-sharing}}$ to prepare $\lceil \frac{3N}{k^2} \rceil$ random degree- $(n' - 1, n' - 1)$ packed Shamir sharings with secret $\mathbf{0} \in \mathbb{F}^{k \times k}$.
2. **Computation phase.** All parties compute the segment layer by layer.
 - For each group of k^2 addition gates with input sharings $[\mathbf{X}]_{t+k-1}$ and $[\mathbf{Y}]_{t+k-1}$, all parties locally compute $[\mathbf{Z}]_{t+k-1} \leftarrow [\mathbf{X}]_{t+k-1} + [\mathbf{Y}]_{t+k-1}$, where $\mathbf{Z} = \mathbf{X} + \mathbf{Y}$.
 - For each group of k^2 multiplication gates with input sharings $[\mathbf{X}]_{t+k-1}$ and $[\mathbf{Y}]_{t+k-1}$, all parties invoke Π_{mult} to compute $[\mathbf{Z}]_{t+k-1}$, where $\mathbf{Z} = \mathbf{X} * \mathbf{Y}$.
3. **Verification phase.** All parties verify the computation of the segment.
 - (a) For each group of k^2 multiplication gates, all parties invoke $\Pi_{\text{vrfy-mult}}$.
 - In case of failure, all parties take the disputed pair E as output.
 - (b) Suppose that after all invocations of $\Pi_{\text{vrfy-mult}}$, all parties obtain their shares of $\{([\mathbf{U}_i]_{k-1}, [\mathbf{U}_i]_{t+k-1})\}_{i \in [2N]}$ and $\{([\mathbf{V}_i]_{k-1}, [\mathbf{V}_i]_{t+2k-2})\}_{i \in [N]}$. All parties invoke $\mathcal{F}_{\text{batched-vrfy}}(P_{\text{king}}, \Sigma_8, 3N)$ with their shares of $\{([\mathbf{U}_i]_{k-1}, [\mathbf{U}_i]_{t+k-1})\}_{i \in [2N]}$ and $\{([\mathbf{V}_i]_{k-1}, [\mathbf{V}_i]_{t+2k-2})\}_{i \in [N]}$ to check whether each pair of sharings corresponds to the same secrets.
 - In case of failure, all parties take the disputed pair E as output.

4.4 Main Protocol

For input gates, when a party P_s aims to distribute its group $\mathbf{X} = (x_{ij})_{ij} \in \mathbb{F}^{k \times k}$ of k^2 inputs to all parties, it randomly samples a degree- $(t + k - 1, t + k - 1)$ packed Shamir sharing with secret $\mathbf{X}$ and distributes them to all parties. To protect against a corrupted dealer, all parties will cross check to ensure P_s distributes a valid degree- $(t + k - 1, t + k - 1)$ packed Shamir sharing. The protocol is formally presented as Π_{input} .

The communication cost of Π_{input} to handle N input gates is $\text{P2P}(\frac{4N \cdot n'^2}{k^2}) + O(1) \times \text{BC}(O(\log n' + \log |\mathbb{F}|))$.

Protocol 5: Π_{input}

All parties handle the segment containing N input gates.

For every group $\mathbf{X} = (x_{ij})_{ij}$ of k^2 input gates belonging to P_s , P_s distributes its inputs as follows:

1. P_s randomly samples a degree- $(t + k - 1, t + k - 1)$ packed Shamir sharing $[\mathbf{X}]_{t+k-1}$, and distributes it to all parties.

2. All parties invoke $\pi_{\text{check-consistency-all}}(P_s, \llbracket \mathbf{X} \rrbracket_{t+k-1})$ to check whether P_s distributes a valid degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - In case of failure, all parties take the disputed pair E as output.

After evaluating the whole circuit, when handling output gates, to ensure a success of reconstruction, all parties depack each degree- $(t+k-1, t+k-1)$ packed Shamir sharings of a group of k^2 output wires to obtain degree- t Shamir sharings of each circuit output. To this end, all parties prepare a tuple of random sharings in the form of $(\{[r_{ij}|j]_t\}_{i,j \in [k]}, \llbracket \mathbf{R} \rrbracket_{t+k-1})$, where $\mathbf{R} = (r_{ij})_{ij} \in \mathbb{F}^{k \times k}$, containing a degree- $(t+k-1, t+k-1)$ packed Shamir sharing $\llbracket \mathbf{R} \rrbracket_{t+k-1}$ and degree- t Shamir sharings of each component $[r_{ij}|j]_t$ with secret stored at the j -th position. Then all parties follow the *mask-open-unmask* template and add extra verification to ensure correctness. We present the protocol for output gates in Π_{output} .

The communication cost of Π_{output} to handle N output gates is $\text{P2P}(\frac{4N \cdot n'^2}{k^2} + \frac{2N \cdot n'}{k}) + O(1) \times \text{BC}(O(\log n' + \log |\mathbb{F}|))$, one invocation of $\mathcal{F}_{\text{rand-sharing}}(\Sigma_3, \lceil \frac{N}{k^2} \rceil)$, and one invocation of $\mathcal{F}_{\text{batched-vrfy}}(\Sigma_8, \frac{N}{k^2})$, amounting to $\text{P2P}(O(N \cdot n' + n'^4)) + O(1) \times \text{BA}(1) + O(1) \times \text{BC}(O(\log N + \log n' + \log |\mathbb{F}|))$.

Protocol 6: Π_{output}

All parties handle the segment containing N output gates.

1. All parties invoke $\mathcal{F}_{\text{rand-sharing}}(\Sigma_3, \lceil \frac{N}{k^2} \rceil)$ to prepare $\lceil \frac{N}{k^2} \rceil$ random packed Shamir sharings in the form of $(\{[r_{ij}|j]_t\}_{i,j \in [k]}, \llbracket \mathbf{R} \rrbracket_{t+k-1})$, where $\mathbf{R} = (r_{ij})_{ij}$.
2. For every group $\mathbf{Z} = (z_{ij})_{ij}$ of k^2 output gates, all parties prepare degree- t Shamir sharings of the outputs as follows:
 - (a) All parties pick one unused random sharing $(\{[r_{ij}|j]_t\}_{i,j \in [k]}, \llbracket \mathbf{R} \rrbracket_{t+k-1})$, where $\mathbf{R} = (r_{ij})_{ij}$, prepared in last step.
 - (b) All parties locally compute $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{Z} \rrbracket_{t+k-1} + \llbracket \mathbf{R} \rrbracket_{t+k-1}$ and send their shares to P_{king} .
 - (c) P_{king} and all parties invoke $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1})$ to check the shares from all parties form a valid degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - In case of failure, all parties take the disputed pair E as output.
 - (d) P_{king} reconstructs $\mathbf{Z} + \mathbf{R}$ and distributes $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$ to all parties.
 - (e) All parties invoke $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1})$ to check whether P_{king} distributes valid degree- $(k-1, k-1)$ packed Shamir sharing.
 - In case of failure, all parties take the disputed pair as output.
 - (f) Suppose $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1} = \{(f_\ell(x), g_\ell(y))\}_{\ell \in \mathcal{P}_{\text{active}}}$. For each $i, j \in [k]$, each party P_ℓ depacks its sharings by locally computing $[z_{ij}|j]_t \leftarrow g_\ell(\beta_j) - [r_{ij}|j]_t$.
3. All parties invoke $\mathcal{F}_{\text{batched-vrfy}}(P_{\text{king}}, \Sigma_8, \frac{N}{k^2})$ with their shares of $(\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1})$ to verify whether two sharings have the same secret.
 - In case of failure, all parties take the disputed pair E as output

Throughout the protocol, we follow the party elimination framework. After evaluating a segment, if a failure occurs, a disputed pair is identified. All parties then eliminate this pair from the set of active parties and re-evaluate the current segment until it succeeds. After degree- t Shamir sharings of all circuit outputs have been prepared, all parties proceed to reconstruct the secrets to a designated party, where the reconstruction always succeeds due to error correcting property of degree- t Shamir sharings. If, before reconstruction, a number of $n/3$ parties have been eliminated, all parties will re-start the computation by invoking a perfectly secure protocol in the client-server model against an adversary corrupting at most a $1/4$ fraction of servers, where all n parties would play the role of clients and all n' active parties would play the role of servers. Such a protocol can be constructed by extending [GLS19] with packed Shamir sharings to achieve overall constant communication complexity due to a smaller corruption threshold.

Putting things altogether, we present the whole protocol in Π_{main} . We conclude its security property in Lemma 1.

The communication complexity of Π_{main} is $\text{P2P}(O(C_I + C_M + C_O \cdot n + n^5)) + O(n) \times \text{BA}(1) + O(n) \times \text{BC}(O(\log |C| + \log n + \log |\mathbb{F}|))$ plus $O(n)$ invocations of $\mathcal{F}_{\text{packed-triple}}(\lceil \frac{4C_M}{n \cdot k^2} \rceil, k)$, and one invocation of $\mathcal{F}_{\text{main-client-server}}^*$ in case all parties restart the whole computation, where C_I, C_M, C_O denote the number of input gates, multiplication gates, and output gates, respectively, and $|C| = C_I + C_M + C_O$. Since $|C| \geq n^2 \cdot n$, we have $\log |C| \cdot n^2 \leq |C|$, which implies if we use a perfect Byzantine agreement with quadratic cost to instantiate both consensus and broadcast or use an expected constant-round broadcast protocol with amortized linear cost in [AC24], the total cost is $\text{P2P}(O(C_I + C_M + C_O \cdot n + n^5))$ elements plus packed Beaver triples over bivariate polynomials of size $O(|C|)$.

We conclude our construction of perfectly secure MPC in the preprocessing model in Theorem 6.

Protocol 7: Π_{main} in $\mathcal{F}_{\text{packed-triple}}, \mathcal{F}_{\text{batched-vrfy}}, \mathcal{F}_{\text{rand-sharing}}, \mathcal{F}_{\text{main-client-server}}^*$ -hybrid model

Let C_M be the number of multiplication gates. All parties initialize $\mathcal{P}_{\text{active}} \leftarrow \{1, \dots, n\}$.

- 1: **Preprocessing phase.** All parties divide the whole required preprocessing data of
 - $4C_M/k^2$ packed Beaver triples over bivariate polynomials^a and
 - $2C_M/k^2$ random degree- $(t+2k-2, t+2k-2)$ packed Shamir sharing in the form of $\llbracket O \rrbracket_{t+2k-2}$ into n segments. Initialize $i \leftarrow 1$.
 - (a) All parties invoke $\mathcal{F}_{\text{packed-triple}}(\lceil \frac{4C_M}{n \cdot k^2} \rceil, k)$ and $\mathcal{F}_{\text{rand-sharing}}(\Sigma_2, \lceil \frac{2C_M}{n \cdot k^2} \rceil)$.
 - (b) In case of failure, output a new disputed pair E , and update $\mathcal{P}_{\text{active}} \leftarrow \mathcal{P}_{\text{active}} \setminus E$.
 - If $|\mathcal{P}_{\text{active}}| \geq 2t$, go to Step 1.a to re-evaluate the current segment.
 - If $|\mathcal{P}_{\text{active}}| < 2t$, go to Step 3 to restart the computation.
 In case of success,
 - if $i < n$, set $i \leftarrow i + 1$, and go to Step 1.a to handle the next segment,
 - if $i = n$, proceed to the online phase.
- 2: **Online phase.** All parties divide the whole circuit into n segments, $\{\text{seg}_1, \dots, \text{seg}_n\}$, initialize $i \leftarrow 1$, and handle all the segments sequentially as follows.
 - (a) Depending on the type of seg_i , there are 3 cases.
 - If seg_i is an input segment, all parties invoke Π_{input} .
 - If seg_i is a computation segment, all parties invoke Π_{eval} .
 - If seg_i is an output segment, all parties invoke Π_{output} .
 - (b) In case of failure, output a new disputed pair E , and update $\mathcal{P}_{\text{active}} \leftarrow \mathcal{P}_{\text{active}} \setminus E$.
 - If $|\mathcal{P}_{\text{active}}| \geq 2t$, go to Step 2.a to re-evaluate the current segment.
 - If $|\mathcal{P}_{\text{active}}| < 2t$, go to Step 3 to restart the computation.
 In case of success,
 - if $i < n$, set $i \leftarrow i + 1$, and go to Step 2.a to handle the next segment,
 - if $i = n$, go to Step 2.c to reconstruct the outputs.
 - (c) **Reconstruction.** For each output wire o , supposing it belongs to P_d and the secret is stored at the j -th position, all parties reconstruct their shares of $[o[j]]_t$ to P_d . Then all parties learn their outputs.
- 3: **Restart.** All parties invoke $\mathcal{F}_{\text{main-client-server}}^*$, provide their inputs, and receive their outputs.

^a To compute C_M multiplication gates, $2C_M/k^2$ packed Beaver triples over bivariate polynomials are required. We prepare a number of $4C_M/k^2$ packed Beaver triples in case that a failure occurs, and all parties will re-evaluate the segment.

Functionality 6: $\mathcal{F}_{\text{main-client-server}}^*$

1. $\mathcal{F}_{\text{main-client-server}}^*$ ^a receives the input from all clients. Let x denote the input.
2. $\mathcal{F}_{\text{main-client-server}}^*$ computes $C(x)$ and distributes the output to all clients.

^a We write $\mathcal{F}_{\text{main-client-server}}^*$ to emphasize that the functionality $\mathcal{F}_{\text{main-client-server}}^*$ can be invoked only by an adversary corrupting at most a $1/4$ fraction of the parties.

Functionality 7: $\mathcal{F}_{\text{main}}$

1. $\mathcal{F}_{\text{main}}$ receives the input from all parties. Let x denote the input and C denotes the circuit and C denotes the circuit.
2. $\mathcal{F}_{\text{main}}$ computes $C(x)$ and distributes the output to all parties.

Lemma 1. *Let $k = n/12$ and $t = (n-1)/3$. Π_{main} computes $\mathcal{F}_{\text{main}}$ with perfect security in the $\mathcal{F}_{\text{packed-triple}}$, $\mathcal{F}_{\text{batched-vrfy}}$, $\mathcal{F}_{\text{rand-sharing}}$, $\mathcal{F}_{\text{main-client-server}}^*$ -hybrid model against a fully malicious adversary who controls at most t corrupted parties.*

Proof (Proof of Lemma 1). We describe the strategy of the simulator $\mathcal{S}$ phase by phase. Let $\mathcal{C}$ denote the set of corrupted parties and $\mathcal{H}$ denote the set of honest parties. Let $\mathcal{H}_{\text{active}} = \mathcal{H} \cap \mathcal{P}_{\text{active}}$ and $\mathcal{C}_{\text{active}} = \mathcal{C} \cap \mathcal{P}_{\text{active}}$. If $|\mathcal{C}| < t$, during the protocol execution, $\mathcal{S}$ will additionally generate the view of a set $\mathcal{H}'$ of $t - |\mathcal{C}|$ honest parties with smallest identities without using honest parties' inputs. $\mathcal{S}$ always records shares of parties in $\mathcal{C} \cup \mathcal{H}'$.

$\mathcal{S}$ invokes the adversary with sampled internal randomness and the functionality $\mathcal{F}_{\text{main}}$.

- **Preprocessing phase.** $\mathcal{S}$ simulates computation of each segment.
 - $\mathcal{S}$ emulates $\mathcal{F}_{\text{packed-triple}}$. $\mathcal{S}$ receives from the adversary shares of corrupted parties $\{f_{\ell,i}^z(x), g_{\ell,i}^z(y)\}_{\ell \in [N], z \in \{a,b,c\}}$ and an instruction, and records them.
 - * If $\mathcal{S}$ receives **accept** from the adversary, $\mathcal{S}$ additionally samples random polynomials as shares of honest parties in $\mathcal{H}'$, and sends back the shares of corrupted parties to the adversary on behalf of $\mathcal{F}_{\text{packed-triple}}$.
 - * If $\mathcal{S}$ receives **reject** and a new disputed pair from the adversary, $\mathcal{S}$ outputs the disputed pair to the adversary on behalf of $\mathcal{F}_{\text{packed-triple}}$, and aborts on behalf of honest parties.
 - $\mathcal{S}$ emulates each invocation of $\mathcal{F}_{\text{rand-sharing}}(\Sigma_2)$. $\mathcal{S}$ receives from the adversary shares of corrupted parties, additionally randomly samples polynomials as shares of honest parties in $\mathcal{H}'$, and sends back shares of corrupted parties to the adversary on behalf of $\mathcal{F}_{\text{rand-sharing}}$.
- **Online phase,** $\mathcal{S}$ simulates computation of each segment. In case of success, before starting simulating computation of next segment, $\mathcal{S}$ records shares of the output wires in this segment held by t parties in $\mathcal{C} \cup \mathcal{H}'$ for the future simulation.
 - Simulation of Π_{input} .
 - * If P_s is a corrupted party, $\mathcal{S}$ receives shares of honest parties, honestly follows $\pi_{\text{check-consistency-all}}(P_s, \llbracket \mathbf{X} \rrbracket_{t+k-1})$ to check the consistency of shares distributed by the adversary. If some check fails, $\mathcal{S}$ broadcasts complaints on behalf of honest parties, and honestly decides a disputed pair.
 - * If P_s is an honest party, $\mathcal{S}$ randomly samples $2t$ degree- $(t+k-1)$ polynomials $\{f_i(x), g_i(x)\}_{i \in \mathcal{C} \cup \mathcal{H}'}$ such that $f_i(\alpha_j) = g_j(\alpha_i)$ for every $(i, j) \in \mathcal{C} \cup \mathcal{H}' \times \mathcal{C} \cup \mathcal{H}'$. $\mathcal{S}$ sends shares of active corrupted parties to the adversary on behalf of honest P_s . When simulating $\pi_{\text{check-consistency-all}}(P_s, t+k-1)$, $\mathcal{S}$ computes $g_i(\alpha_j) = f_j(\alpha_i)$, $f_i(\alpha_j) = g_j(\alpha_i)$ for every $i \in \mathcal{H}$, $j \in \mathcal{C}$, and uses them as values of honest parties for rest of the procedure including cross check, broadcasting complaints, and identifying disputed pairs, on behalf of honest parties.
 - Simulation of Π_{eval} .
 - * Simulation of $\Pi_{\text{zero-sharing}}$. In Step 2, for each corrupted party P_i , $\mathcal{S}$ receives honest parties' shares of $\llbracket \mathbf{O}^{(i)} \rrbracket_{n'-1}$ from the adversary. For each honest party P_i , $\mathcal{S}$ samples random degree- $(n'-1)$ polynomials as shares of corrupted parties, and sends them to the adversary on behalf of active honest P_i .

* Simulation of Π_{mult} .

1. In Step 1, $\mathcal{S}$ samples $4 \cdot |\mathcal{H}_{\text{active}}|$ random degree- $(n'-1)$ polynomials $\{f_i^{X+A}(x), g_i^{X+A}(y)\}_{i \in \mathcal{H}_{\text{active}}}$ and $\{f_i^{Y+B}(x), g_i^{Y+B}(y)\}_{i \in \mathcal{H}_{\text{active}}}$, satisfying $f_i^{X+A}(\alpha_j) = g_j^{X+A}(\alpha_i)$ and $f_i^{Y+B}(\alpha_j) = g_j^{Y+B}(\alpha_i)$ for every $(i, j) \in \mathcal{H}_{\text{active}} \times \mathcal{H}_{\text{active}} \cup \mathcal{H}_{\text{active}} \times \mathcal{C}_{\text{active}} \cup \mathcal{C}_{\text{active}} \times \mathcal{H}_{\text{active}}$, as shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{n'-1}, \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{n'-1}$ held by active honest parties, and sends them to P_{king} on behalf of active honest parties.
2. In Step 2, if P_{king} is honest, $\mathcal{S}$ follows the protocol honestly. $\mathcal{S}$ receives active honest parties' shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1}, \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1}$ from P_{king} , where $\mathbf{X} + \mathbf{A}$ and $\mathbf{Y} + \mathbf{B}$ are used to distinguish them from $\mathbf{X} + \mathbf{A}$ and $\mathbf{Y} + \mathbf{B}$ due to potential malicious behaviors, respectively.
3. In Step 3, $\mathcal{S}$ samples $2 \cdot |\mathcal{H}_{\text{active}}|$ random degree- $(n'-1)$ polynomials $\{f_i^{Z+R}(x), g_i^{Z+R}(y)\}_{i \in \mathcal{H}_{\text{active}}}$, satisfying $f_i^{Z+R}(\alpha_j) = g_j^{Z+R}(\alpha_i)$ for every $(i, j) \in \mathcal{H}_{\text{active}} \times \mathcal{H}_{\text{active}} \cup \mathcal{H}_{\text{active}} \times \mathcal{C}_{\text{active}} \cup \mathcal{C}_{\text{active}} \times \mathcal{H}_{\text{active}}$, as shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{n'-1}$ held by active honest parties, and sends them to P_{king} on behalf of active honest parties.
4. In Step 4, if P_{king} is honest, $\mathcal{S}$ honestly follows the protocol. $\mathcal{S}$ receives active honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$ from P_{king} , where $\mathbf{Z} + \mathbf{R}$ is used to distinguish it from $\mathbf{Z} + \mathbf{R}$.

* Simulation of $\Pi_{\text{vrfy-mult}}$.

1. In Step 1, $\mathcal{S}$ simulates each invocation of $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket \mathbf{U} \rrbracket_{k-1})$. Since $\mathcal{S}$ learns shares of $\llbracket \mathbf{U} \rrbracket_{k-1}$ from P_{king} , $\mathcal{S}$ simulates honest parties' behaviors by honestly following the procedure.
2. In Step 2,
 - (a) $\mathcal{S}$ computes active honest parties' shares of $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}$ and the secrets $\mathbf{X} + \mathbf{A}$ by solving a linear system of $(n' - t')^2$ unknowns containing k^2 secrets of $\mathbf{X} + \mathbf{A}$ and $(n')^2 - k^2 - (2n' \cdot t' - (t')^2) = (n' - t')^2 - k^2$ randomness from active honest parties' shares of $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}$, defined as follows. For every $i \in \mathcal{H}_{\text{active}}$, P_i 's share of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{n'-1}$ sampled in previous simulation equals to the summation of its share of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}$, which are linear combinations of k^2 secrets $\mathbf{X} + \mathbf{A}$, and its share of $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}$, which are linear combinations of $(n' - t')^2 - k^2$ randomness mentioned above, amounting to a number $2n' \cdot (n' - t') - (n' - t')^2 - 2(n' - t') \cdot t' = (n' - t')^2$ of equations.
 - (b) Similarly, $\mathcal{S}$ computes active honest parties' shares of $\llbracket \mathbf{O}_2 \rrbracket_{n'-1}$ and the secrets $\mathbf{Y} + \mathbf{B}$.
 - (c) To compute active honest parties' shares of $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$, $\mathcal{S}$ first samples random degree- $(t + 2k - 2, t + 2k - 2)$ packed Shamir sharing $\llbracket \mathbf{O} \rrbracket_{t+2k-2}$ according to shares of corrupted parties and the secret $\mathbf{O} = \mathbf{0} \in \mathbb{F}^{k \times k}$. Then $\mathcal{S}$ computes active honest parties' shares of $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$ and the secret $\mathbf{Z} + \mathbf{R}$ by solving a linear system of $(n' - t')^2$ unknowns containing k^2 secrets of $\mathbf{Z} + \mathbf{R}$ and $(n' - t')^2 - k^2$ randomness from active honest parties' shares of $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$, defined as follows. For every $i \in \mathcal{H}_{\text{active}}$, P_i 's share of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{n'-1}$ sampled in previous simulation equals to the summation of its share of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2}$, which are linear combinations of k^2 secrets $\mathbf{Z} + \mathbf{R}$, and its share of $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$, which are linear combinations of $(n' - t')^2 - k^2$ randomness mentioned above.
 - (d) Then $\mathcal{S}$ sends active honest parties' shares of $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}, \llbracket \mathbf{O}_2 \rrbracket_{n'-1}$ and $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$ to P_{king} on behalf of honest parties. Notice that all honest parties' shares generated when invoking $\Pi_{\text{zero-sharing}}$ have been sampled, then $\mathcal{S}$ simulates honest parties' behaviors honestly.
3. In Step 3, for each invocation of $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket \mathbf{U} \rrbracket_d)$, since $\mathbf{U}$ has been computed and thus the shares of honest parties $\llbracket \mathbf{U} \rrbracket_d$ can be computed according to the secret $\mathbf{U}$ and shares of corrupted parties, $\mathcal{S}$ honestly follows the procedure to simulate honest parties' behaviors.

* In Step 3.b of Π_{eval} , $\mathcal{S}$ emulates the interaction between the adversary and $\mathcal{F}_{\text{batched-vrfy}}$ and behaviors of honest parties. $\mathcal{S}$ sends honest parties' shares of $\{(\llbracket \mathbf{U}_i \rrbracket_{k-1}, \llbracket \mathbf{U}_i \rrbracket_{t+k-1})\}_{i \in [2N]}$ and $\{(\llbracket \tilde{\mathbf{V}}_i \rrbracket_{k-1}, \llbracket \tilde{\mathbf{V}}_i \rrbracket_{t+2k-2})\}_{i \in [N]}$ to the adversary on behalf of $\mathcal{F}_{\text{batched-vrfy}}$. $\mathcal{S}$ checks whether $\tilde{\mathbf{U}}_i = \mathbf{U}_i$ for every $i \in [2N]$ and $\tilde{\mathbf{V}}_i = \mathbf{V}_i$ for every $i \in [N]$. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{batched-vrfy}}$ honestly.

• Simulation of Π_{output} .

- * Simulation of $\mathcal{F}_{\text{rand-sharing}}(\Sigma_3, N)$. $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand-sharing}}$, receives shares of corrupted parties from the adversary, and sends them back to the adversary on behalf of $\mathcal{F}_{\text{rand-sharing}}$. Additionally, according to shares of active corrupted parties, $\mathcal{S}$ also randomly samples shares for parties in $\mathcal{H}' \cup \mathcal{C} \setminus \mathcal{C}_{\text{active}}$.
- * In Step 2.b, $\mathcal{S}$ randomly samples $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1}$ according to shares of parties in $\mathcal{C} \cup \mathcal{H}'$, and sends shares of honest parties to P_{king} on behalf of honest parties.
- * In Step 2.c, since $\mathcal{S}$ has computed honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1}$, $\mathcal{S}$ follows the procedure $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1})$ honestly.
- * In Step 2.d, if P_{king} is honest, $\mathcal{S}$ follows the protocol honestly, $\mathcal{S}$ receives honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$ from P_{king} .
- * In Step 2.e, since $\mathcal{S}$ knows honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$, $\mathcal{S}$ honestly follows the procedure $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1})$.
- * In Step 2.f, $\mathcal{S}$ computes shares of $[z_{ij}|j]_t$ held by parties in $\mathcal{C} \cup \mathcal{H}'$ for each $i, j \in [k]$.
- * In Step 3, $\mathcal{S}$ emulates $\mathcal{F}_{\text{batched-vrfy}}$, sends honest parties' shares of $(\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1})$ to the adversary on behalf of $\mathcal{F}_{\text{batched-vrfy}}$. $\mathcal{S}$ checks whether $\overline{\mathbf{Z}} + \overline{\mathbf{R}} = \mathbf{Z} + \mathbf{R}$ for every group of outputs. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{batched-vrfy}}$ honestly.
- Before **reconstruction**, if a number of $n/3$ parties has been eliminated, $\mathcal{S}$ emulates $\mathcal{F}_{\text{main-client-server}}^*$. $\mathcal{S}$ receives inputs of corrupted clients from the adversary, sends them to $\mathcal{F}_{\text{main}}$, receives the outputs of corrupted clients from $\mathcal{F}_{\text{main}}$, and sends them to the adversary on behalf of $\mathcal{F}_{\text{main-client-server}}^*$.
- In **reconstruction**, for each output belonging to a corrupted party P_d , $\mathcal{S}$ receives the output o from $\mathcal{F}_{\text{main}}$ and learns t shares of $[o|j]_t$ held by parties in $\mathcal{C} \cup \mathcal{H}'$. Then $\mathcal{S}$ computes the whole sharing of $[o|j]_t$ and sends the shares of honest parties to the adversary on behalf of honest parties.
- $\mathcal{S}$ outputs what the adversary outputs and honest parties output their outputs received from $\mathcal{F}_{\text{main}}$.

We now show that the distribution of the output in the ideal world is identical to that in the real world using the following hybrid arguments.

Hyb₀: In this hybrid, we consider the execution in the real world.

Hyb₁: In this hybrid, we change the emulation of $\mathcal{F}_{\text{packed-triple}}$ and $\mathcal{F}_{\text{rand-sharing}}$. $\mathcal{S}$ emulates $\mathcal{F}_{\text{packed-triple}}$ and $\mathcal{F}_{\text{rand-sharing}}$ by receiving shares of active corrupted parties from the adversary, sending them back to the adversary. $\mathcal{S}$ will additionally and randomly samples shares of parties in $\mathcal{C} \cup \mathcal{H}'$ and delays the sampling of shares of parties in $\mathcal{H} \setminus \mathcal{H}'$. **Hyb₁** and **Hyb₀** have the same distributions.

Hyb₂: In this hybrid, when simulating Π_{input} , we change the computation of honest parties' shares when an honest party P_s shares its inputs. We first randomly samples shares (admitting consistent common points) as shares of parties in $\mathcal{C} \cup \mathcal{H}'$, then computes the whole sharing based on its inputs and shares of parties in $\mathcal{C} \cup \mathcal{H}'$. **Hyb₂** and **Hyb₁** have the same distributions.

Hyb₃: In this hybrid, in Π_{input} , when simulating $\pi_{\text{check-consistency-all}}(P_s, \llbracket \mathbf{X} \rrbracket_{t+k-1})$ for an honest P_s , for each honest party P_i , we compute $g_i(\alpha_j) = f_j(\alpha_i)$ and $f_i(\alpha_j) = g_j(\alpha_i)$ for $j \in \mathcal{C}$, and use them for simulation. Since P_s is honest parties, these values are the same as that in **Hyb₂**. **Hyb₃** and **Hyb₂** have the same distributions.

Hyb₄: In this hybrid, in Π_{mult} , we change the computation of honest parties' shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{n'-1}$ and $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{n'-1}$. We first randomly sample active honest parties' shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{n'-1}$. Notice that the common points known by both a corrupted party and an honest party have already fixed. Then this amounts to sampling $2n' \cdot (n' - t') - (n' - t')^2 - 2(n' - t') \cdot t' = (n' - t')^2$ random elements. Then we compute honest parties' shares of $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}$ and the secret $\mathbf{X} + \mathbf{A}$ as mentioned in the simulation (Step 2.a of simulation of $\Pi_{\text{vrfy-mult}}$). Similarly, we compute active honest parties' shares of $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{n'-1}$. Since there is an one-to-one correspondence between active honest parties' shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{n'-1}$ and their shares of $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}$ and k^2 secrets $\mathbf{X} + \mathbf{A}$, **Hyb₄** and **Hyb₃** have the same distributions due to the randomness of $\llbracket \mathbf{A} \rrbracket_{t+k-1}$, $\llbracket \mathbf{B} \rrbracket_{t+k-1}$, $\llbracket \mathbf{O}_1 \rrbracket_{n'-1}$, $\llbracket \mathbf{O}_2 \rrbracket_{n'-1}$.

Hyb₅: In this hybrid, in Π_{mult} , we change the computation of honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{n'-1}$. We first randomly sample active honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{n'-1}$, amounting to sampling $(n' - t')^2$ random elements. Then we sample active honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2}$, amounting to sample $(t + 2k - 1)^2 - k^2 - (2t \cdot (t + 2k - 1) - t^2) = (2k - 1)^2 - k^2$ random elements. Then we compute honest

parties' shares of $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$ and the secret $\mathbf{Z} + \mathbf{R}$ as mentioned in the simulation (Step 2.c of simulation of $\Pi_{\text{vrfy-mult}}$). Since given active honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2}$, there is an one-to-one correspondence between active honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{n'-1}$ and their shares of $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$ and k^2 secrets $\mathbf{Z} + \mathbf{R}$. **Hyb₅** and **Hyb₄** have the same distributions due to the randomness of $\llbracket \mathbf{R} \rrbracket_{t+k-1}$, $\llbracket \mathbf{O} \rrbracket_{t+2k-2}$, $\llbracket \mathbf{O}_3 \rrbracket_{n'-1}$.

Hyb₆: In this hybrid, in Π_{output} , we change the emulation of $\mathcal{F}_{\text{rand-sharing}}$. $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand-sharing}}$ by receiving shares of corrupted parties from the adversary and sending them back. Additionally, $\mathcal{S}$ randomly samples shares for parties in $\mathcal{C} \cup \mathcal{H}' \setminus \mathcal{C}_{\text{active}}$ and delays sampling of other honest parties' shares. **Hyb₆** and **Hyb₅** have the same distributions.

Hyb₇: In this hybrid, in Π_{output} , we change the computation of honest parties' shares of $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1}$. We randomly sample $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+k-1}$ according to shares of parties in $\mathcal{C} \cup \mathcal{H}'$ and compute shares of honest parties. **Hyb₇** and **Hyb₆** have the same distributions due to the randomness of $\llbracket \mathbf{R} \rrbracket_{t+k-1}$.

Hyb₈: In this hybrid, if parties invoke $\mathcal{F}_{\text{main-client-server}}^*$, we change the emulation as follows. We receive corrupted clients' inputs from the adversary, then compute the circuit output, and sends it back to the adversary. **Hyb₈** and **Hyb₇** have the same distributions.

Hyb₉: In this hybrid, in **reconstruction**, for an output wire o belonging to a corrupted party, we change the computation of honest parties' shares of $[o[j]]_t$. We first compute corrupted parties' inputs $\mathbf{X}$ from honest parties' shares of $\llbracket \mathbf{X} \rrbracket_{t+k-1}$, then compute the circuit output o . Then we compute the whole degree- t sharing $[o[j]]_t$ according to t shares held by parties in $\mathcal{C} \cup \mathcal{H}'$ and the secret o . **Hyb₉** and **Hyb₈** have the same distributions.

Hyb₁₀: In this hybrid, in **reconstruction**, we let $\mathcal{F}_{\text{main}}$ take the role of $\mathcal{S}$ to compute the circuit output belonging to corrupted parties. Since $\mathcal{F}_{\text{main}}$ does the same as $\mathcal{S}$ does in **Hyb₉**. **Hyb₁₀** and **Hyb₉** have the same distributions.

Since **Hyb₁₀** corresponds to the ideal world, where inputs of honest parties are not used, Π_{main} securely computes $\mathcal{F}_{\text{main}}$ in the $\mathcal{F}_{\text{rand-sharing}}$, $\mathcal{F}_{\text{batched-vrfy}}$, $\mathcal{F}_{\text{main-client-server}}^*$ -hybrid model. $\square$

Theorem 6. *Let n be the number of parties and $k = n/12, t = (n-1)/3$. Let $\mathbb{F}$ be a finite field of size $|\mathbb{F}| \geq 2n$. For an arithmetic circuit C over $\mathbb{F}$, there exists a perfectly secure MPC protocol in the circuit-independent preprocessing model which computes $O(n^2)$ copies of the same circuit C in the presence of a fully malicious adversary controlling at most t parties. The amortized cost per copy is $\text{P2P}(O(C_I + C_M + C_O \cdot n + n^5)) + O(n) \times \text{BA}(1) + O(n) \times \text{BC}(O(\log |C| + \log n + \log |\mathbb{F}|))$ plus $O(n)$ invocations of $\mathcal{F}_{\text{packed-triple}}(\lceil \frac{4C_M}{n \cdot k^2} \rceil, k)$, where C_I, C_M , and C_O denote the number of input gates, multiplication gates, and output gates, respectively, and $|C| = C_I + C_M + C_O$.*

5 Lower Bounds

Recall that we have already reduced the task of constructing perfectly secure MPC protocols to preparing error-free packed Beaver triples over bivariate polynomials. However, in this section, we will establish a lower bound on communication complexity of preparing such kind of error-free packed Beaver triples. To achieve this, we start with an impossibility result regarding computing AND of two bits with perfect security among 3 parties in Section 5.1. Then we generalize it to prove a communication lower bound for verifying correctness of packed Beaver triples even with perfect abort security in Section 5.2. Next, in Section 5.3, we observe that assuming error-free packed Beaver triples, all parties can efficiently verify the correctness of packed Beaver triples, which implies the functionality of preparing error-free packed Beaver triples (even for perfect security with abort) cannot be instantiated with low communication. In Section 5.4, we further extend the communication lower bound for verifying packed Beaver triple to a communication lower bound of computing inner product function, which is more natural to be expressed as a circuit.

Before diving into the study of communication lower bounds, we first introduce several useful notions in the context of protocol execution below.

Notations. Suppose there are n parties, denoted by $P_1, P_2, \dots, P_n$, and an n -party protocol Π . Note that each execution of Π is fully determined by a tuple consisting of all parties' inputs and internal randomness,

denoted by $d := (I_1, I_2, \dots, I_n; r_1, r_2, \dots, r_n)$, where I_i and r_i represent P_i 's input and internal randomness, respectively. We use $\text{exec} := \text{exec}(\Pi, d)$ to denote the execution defined by running Π on d . Before the execution, each party P_i has its input $\text{input}^{P_i}(\Pi, d) = I_i$ and samples its internal randomness $\text{rand}^{P_i}(\Pi, d) = r_i$. At each round of the protocol, for each party, its current view (including its input, internal randomness, and all messages sent or received so far) determines the messages it sends in this round. In the final round of the protocol, its view determines its protocol output.

After the execution, each party P_i obtains its view $\text{view}^{P_i}(\Pi, d)$, which contains its input $\text{input}^{P_i}(\Pi, d)$, its internal randomness $\text{rand}^{P_i}(\Pi, d)$, its transcript $\text{transcript}^{P_i}(\Pi, d)$, and its output $\text{output}^{P_i}(\Pi, d)$. We use $\text{view}^{P_i}(\Pi, I_1, \dots, I_n)$ to denote the distribution of the view of P_i generated by running Π on $(I_1, \dots, I_n)$ with parties' internal randomness sampled uniformly at random. Given the view of party P view^P , we say a tuple of inputs and internal randomness d satisfies view^P if $\text{view}^P(\Pi, d) = \text{view}^P$, which is denoted by $d \vdash \text{view}^P$. We say a view view is plausible for P_i if there exists a tuple d of inputs and internal randomness from all parties satisfying view , i.e., $\text{view}^{P_i}(\Pi, d) = \text{view}$. We assume each party has its internal randomness of finite size, which ensures the underlying probability space has a finite sample space.

5.1 Warm-up: Impossibility Result of Computing AND of Two Bits

The pioneer work [BGW88] shows a strong negative result that information-theoretic MPC cannot exist without honest majority, even when only semi-honest security is considered. In particular, it can be shown that there does not exist 2-party protocols securely realizing the functionality of computing AND of two bits $f_{\text{AND}} : (x_A, x_B) \mapsto (x_A \cdot x_B, x_A \cdot x_B)$ against one semi-honest corruption. We extend the proof for this 2-party negative result to 3-party case by additionally considering a malicious adversary. We formally present the statement in Lemma 2.

Lemma 2. *There does not exist a 3-party protocol computing $f_{\text{AND}} : (\perp, x_2, x_3) \mapsto (\perp, x_2 \cdot x_3, x_2 \cdot x_3)$ with perfect security with abort against any fully malicious adversary controlling at most one corrupted party.*

Proof (Proof of Lemma 2). Denote three parties by P_1, P_2 and P_3 . For the sake of contradiction, suppose there exists a 3-party protocol π computing f_{AND} with perfect security with abort against a fully malicious adversary controlling one corrupted party. Denote an honest execution where P_i uses x_i as input and r_i as internal randomness for $i \in \{1, 2, 3\}$ by $\text{exec}(\pi, \perp, x_2, x_3; r_1, r_2, r_3)$. We first consider an honest execution $\text{exec}_0 := \text{exec}(\pi, \perp, 0, 0; r_1, r_2, r_3)$.

By perfect privacy against P_2 , there exists an honest execution $\text{exec}_1 := \text{exec}(\pi, \perp, 0, 1; r'_1, r_2, r'_3)$ such that P_2 has the same view as that produced by exec_0 . To see why this is the case, by perfect privacy of π , there exists a simulator $\mathcal{S}_2$ which takes as input the input and output of P_2 , and outputs the view of P_2 such that for any $x_2, x_3 \in \{0, 1\}$, $\mathcal{S}_2(x_2, x_2 \cdot x_3) \equiv \text{view}^{P_2}(\pi, \perp, x_2, x_3)$. Then by property of f_{AND} , $\text{view}^{P_2}(\pi, \perp, 0, 0) \equiv \mathcal{S}_2(0, 0 \cdot 0) \equiv \mathcal{S}_2(0, 0 \cdot 1) \equiv \text{view}^{P_2}(\pi, \perp, 0, 1)$. This implies there exists a choice of internal randomness of P_1 and P_3 , r'_1, r'_3 , such that exec_0 and $\text{exec}_1 := \text{exec}(\pi, \perp, 0, 1; r'_1, r_2, r'_3)$ generate identical views for P_2 . Similarly, there exists an honest execution $\text{exec}_2 := \text{exec}(\pi, \perp, 1, 0; r''_1, r''_2, r_3)$ such that P_3 has the same view as that produced by exec_0 .

Now we claim that existence of such three executions $\text{exec}_0, \text{exec}_1$ and exec_2 contradicts with perfect correctness of π . To show this, we construct a malicious execution, denoted by exec_3 , as follows.

- P_2 uses 1 as input and r''_2 as internal randomness and honestly follows the protocol.
- P_3 uses 1 as input and r'_3 as internal randomness and honestly follows the protocol.
- Malicious P_1 first emulates exec_1 and exec_2 to compute the messages it would send to P_2 and P_3 in both executions. In the actual execution, P_1 behaves inconsistently by sending to P_2 the messages it would have sent in exec_2 , while sending to P_3 the messages it would have sent in exec_1 .

To argue the outputs of P_2 and P_3 in exec_3 , we prove the following claim.

Claim. The transcripts between P_2 and P_1 in executions exec_3 and exec_2 are identical. The transcripts between P_3 and P_1 in executions exec_3 and exec_1 are identical. The transcripts between P_2 and P_3 are identical in four executions $\text{exec}_0, \text{exec}_1, \text{exec}_2$ and exec_3 .

Proof. We prove this statement by induction. First, by construction of exec_1 and exec_2 , the transcripts between P_2 and P_3 in $\text{exec}_0, \text{exc}_1$ and exec_2 are identical.

Consider the first message in exec_3 .

- If it is from P_1 to P_2 , by construction of exec_3 , this message is identical to that of exec_2 . Similar, if it is from P_1 to P_3 , by construction of exec_3 , this messages is identical to that of exec_1 .
- If it is from P_2 (to either P_1 or P_3), since P_2 uses the same input 1 and internal randomness r_2'' in exec_3 and exec_2 , this message is identical to that of exec_2 .
- If it is from P_3 (to either P_1 or P_2), since P_3 uses the same input 1 and internal randomness r_3' in exec_3 and exec_1 , this message is identical to that of exec_1 .

Now assume the first $T - 1$ messages already satisfy the statement. We prove this holds for the T -th message.

- If it is from P_1 (to either P_2 or P_3), by construction of exec_3 , it satisfies the statement.
- If it is sent from P_2 (to either P_1 or P_3), by induction hypothesis, the messages previously sent or received by P_2 are identical in exec_3 and exec_2 . Hence, the current views for P_2 (including its input 1, internal randomness r_2'' , and messages sent or received itself) are identical in exec_3 and exec_2 , and this message sent from P_2 is identical in exec_3 and exec_2 . Moreover, if this message is from P_2 to P_3 , it is identical in all four executions.
- If it is sent from P_3 (to either P_1 or P_2), by induction hypothesis, the messages previously sent or received by P_3 are identical in exec_3 and exec_1 . Hence, the current views for P_3 (including its input 1, internal randomness r_3' , and messages sent or received itself) are identical in exec_3 and exec_1 , and this message sent from P_3 is identical in exec_3 and exec_1 . Moreover, if this message is from P_3 to P_2 , it is identical in all four executions.

By induction, the statement holds for all messages in exec_3 . $\square$

According to the claim above, when comparing exec_3 and exec_2 , P_2 has the same views in both executions, and thus P_2 would output 0 in both executions. However, by perfect correctness of π , both P_2 and P_3 should output either $1 \cdot 1 = 1$ or $\perp$, which leads to a contradiction. $\square$

Implication for n -party setting. When there are n parties and $t \geq n/3$, we show Lemma 2 implies there does not exist an n -party protocol computing the AND of two bits held by two different parties (P_A and P_B) with perfect security with abort against any fully malicious adversary controlling at most t corrupted parties. In fact, for the sake of contradiction, if there exists an n -party protocol computing the AND function with perfect security with abort, then let P_2 emulate t parties containing P_A , let P_3 emulate t parties containing P_B , and let P_1 emulate the rest of the parties. In doing this, we obtain a 3-party protocol computing f_{AND} with perfect security with abort against one corrupted party, which contradicts with Lemma 2. This negative result suggests $n > 3t$ is necessary (and sufficient) for computing the AND function with perfect security with abort.

Remark 1. We note that even assuming a broadcast channel, Lemma 2 (and its implication for n -party setting) holds. This is because the broadcast messages are in both P_2 's view and P_3 's view. Thus, they are identical in all three executions $\text{exec}_0, \text{exec}_1, \text{exec}_2$. Then in exec_3 , P_1 simply broadcasts the same messages as those in $\text{exec}_0, \text{exec}_1, \text{exec}_2$. We can show that the same argument works for exec_3 and both P_2, P_3 would output 0 in exec_3 which breaks the perfect correctness.

5.2 Communication Complexity of Verifying Packed Triples

We now generalize the above 3-party impossibility result by introducing a fourth party, P_0 , which is involved in privately computing the AND of two bits. If P_0 does not communicate with any other party, the setting is identical to the case of 3-party computation. More generally and intuitively, if P_0 communicates only a few bits, thus failing to provide sufficient information to P_2 and P_3 , then this leads to a contradiction with perfect security. This contradiction arises by considering an adversary that mounts a similar attack to that described in the proof of Lemma 2.

To derive a communication lower bound for preparing error-free packed Beaver triples, we consider the task of verifying correctness of packed triples and formally characterize this functionality as $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$, where $k(\leq (t+2)/2)$ secrets are packed into each of N packed triples. We emphasize that *perfect security with abort* is a weaker notion than perfect security in the sense that it allows an adversary to abort the functionality. In addition, we further weaken the functionality by revealing honest parties' inputs to the adversary when the functionality decides to abort. Building on the above intuition, we prove a communication lower bound that in order to verify N packed Beaver triples with perfect security with abort, each party must communicate $\Omega(N \cdot k)$ field elements in expectation, leading to a total communication of $\Omega(N \cdot n^2)$ elements if $k = \Omega(n)$. This result is formally stated as Lemma 3.

Functionality 8: $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$

1. $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ receives a set $\mathcal{C}$ of corrupted parties from the adversary.
2. $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ receives the shares $\{[a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1}\}_{\ell \in [N]}$ from all honest parties, reconstructs the secrets using honest parties' shares to learn $\{a_\ell, b_\ell, c_\ell\}_{\ell \in [N]}$, where $a_\ell, b_\ell, c_\ell \in \mathbb{F}^k$, and sends the shares of $\{[a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1}\}_{\ell \in [N]}$ held by corrupted parties to the adversary.
3. $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ checks if $a_\ell * b_\ell = c_\ell$ holds for each $\ell \in [N]$.
 - If all checks pass, $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ waits for an instruction from the adversary:
 - If $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ receives **reject** from the adversary, it outputs 0 to all parties and reveals all parties' shares $\{[a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1}\}_{\ell \in [N]}$ to the adversary.
 - If $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ receives **accept** from the adversary, it outputs 1 to all parties.
 - If one of the checks fails, $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ outputs 0 to all parties and reveals all parties' shares $\{[a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1}\}_{\ell \in [N]}$ to the adversary.

Lemma 3. *Let $n = 3t + 1$ and $k \in \mathbb{N}$ such that $1 \leq k \leq t + 1$. There exist constants $c_1, c_2 > 0$ such that for all $N > c_1$ and $k > c_2$, any protocol computing $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ with perfect security against any fully malicious adversary controlling at most t corrupted parties satisfies that each party has expected communication of at least $\frac{N \cdot k}{8}$ elements, leading to total communication of at least $\frac{N \cdot k \cdot n}{16}$ elements, when $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ outputs 1. This lower bound implies an amortized cost of $\Omega(n)$ elements per secret.*

Proof (Proof of Lemma 3). Denote $n = 3t + 1$ parties by $P_0, P_{1,1}, \dots, P_{1,t}, P_{2,1}, \dots, P_{2,t}, P_{3,1}, \dots, P_{3,t}$. Additionally, denote $\{P_{1,i}\}_{i \in [t]}$, $\{P_{2,i}\}_{i \in [t]}$ and $\{P_{3,i}\}_{i \in [t]}$ by P_1, P_2 and P_3 , respectively. Suppose π is an n -party protocol computing $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ with perfect security against any fully malicious adversary controlling at most t corrupted parties. Since all honest parties have the same output in $\mathcal{F}_{\text{vrfy-triple-with-abort}}$, we use $\text{output}(\pi, d)$ to denote the output of an honest party in execution (π, d) . We will consider the set $\mathcal{D}$ containing all tuples of inputs and internal randomness such that an honest execution defined by this tuple outputs 1, i.e., $\mathcal{D} := \{d \mid \text{output}(\pi, d) = 1\}$. We assume each party has its internal randomness of finite size, which implies the cardinality of $\mathcal{D}$ is finite. We model $\mathcal{D}$ as the sample space, and assume a uniform distribution over $\mathcal{D}$. For any event $\mathcal{E} \subseteq \mathcal{D}$, we denote its probability by $\Pr_{d \leftarrow \mathcal{D}}[\mathcal{E}]$, or simply $\Pr_d[\mathcal{E}]$.

Without loss of generality, we will focus on the communication complexity of party P_0 . We denote the length of its transcript by a random variable $C_0 : \mathcal{D} \rightarrow \mathbb{N}$. We set $c_1 = \lceil \frac{24}{\log F} \rceil$ and $c_2 = 24$. For the sake of contradiction, suppose there exist $N > c_1$ and $k > c_2$ such that the communication of P_0 in π satisfies that

$$\mathbb{E}_{d \leftarrow \mathcal{D}}[C_0(d)] < \frac{N \cdot k}{8}.$$

Extending the idea of Lemma 2, the proof is divided into three steps.

Step 1. Define three honest executions satisfying several delicate requirements.

Step 2. Construct a (dishonest) execution with an incorrect output by composing three honest executions chosen in last step, which leads to a contradiction with perfect correctness of π .

Step 3. Prove existence of such three honest executions based on perfect privacy of π and the assumption that P_0 only has limited amount of communication.

The formal proof works as follows.

Step 1. Defining three honest executions. We start with sampling an honest execution exec_0 with output 1 by sampling $d = (I_0, I_1, I_2, I_3; r_0, r_1, r_2, r_3)$ from $\mathcal{D}$ uniformly at random. Suppose $(\mathbf{a}^{(0)}, \mathbf{b}^{(0)}, \mathbf{c}^{(0)})$ with $\mathbf{a}^{(0)}, \mathbf{b}^{(0)} \in \mathbb{F}^{N \cdot k}$ and $\mathbf{c}^{(0)} = \mathbf{a}^{(0)} * \mathbf{b}^{(0)}$ are the secrets of the N packed triples defined by d . Let $\mathcal{D}_1(d) \subset \mathcal{D}$ be the set of all tuples of the form $d' = (I'_0, I'_1, I'_2, I'_3; r'_0, r'_1, r'_2, r'_3)$ satisfying the following conditions.

- $\text{view}^{P_2}(\pi, d) = \text{view}^{P_2}(\pi, d')$, which implies $I'_2 = I_2, r'_2 = r_2$;
- $\text{input}^{P_0}(\pi, d) = \text{input}^{P_0}(\pi, d')$ and $\text{transcript}^{P_0}(\pi, d) = \text{transcript}^{P_0}(\pi, d')$, implying $I'_0 = I_0$;
- Letting $(\mathbf{a}^{(1)}, \mathbf{b}^{(1)}, \mathbf{c}^{(1)})$ denote the secrets of the N packed triples defined by d' , we require that $\mathbf{c}^{(1)} = \mathbf{a}^{(1)} * \mathbf{b}^{(1)}$ and $\mathbf{b}^{(1)} = \mathbf{b}^{(0)}$. We denote this constraint compactly by writing $d' \vdash \mathbf{a}^{(1)}$.

Then we define **event 1** to be an event that the number of possible distinct secrets of the first sharing among the executions in the set $\mathcal{D}_1(d)$ is strictly greater than $F^{\frac{N \cdot k}{2}}$, i.e., $|\{\mathbf{a}^{(1)} \in \mathbb{F}^{N \cdot k} \mid \exists d' \in \mathcal{D}_1(d) \text{ s.t. } d' \vdash \mathbf{a}^{(1)}\}| > F^{\frac{N \cdot k}{2}}$. Similarly, let $\mathcal{D}_2(d) \subset \mathcal{D}$ be the set of all tuples of the form $d'' = (I''_0, I''_1, I''_2, I''_3; r''_0, r''_1, r''_2, r''_3) \in \mathcal{D}$ satisfying the following conditions.

- $\text{view}^{P_3}(\pi, d) = \text{view}^{P_3}(\pi, d'')$, which implies $I''_3 = I_3, r''_3 = r_3$;
- $\text{input}^{P_0}(\pi, d) = \text{input}^{P_0}(\pi, d'')$ and $\text{transcript}^{P_0}(\pi, d) = \text{transcript}^{P_0}(\pi, d'')$, implying $I''_0 = I_0$;
- Letting $(\mathbf{a}^{(2)}, \mathbf{b}^{(2)}, \mathbf{c}^{(2)})$ denote the secrets of the N packed triples defined by d'' , we require that $\mathbf{c}^{(2)} = \mathbf{a}^{(2)} * \mathbf{b}^{(2)}$ and $\mathbf{a}^{(2)} = \mathbf{a}^{(0)}$. We denote this constraint compactly by writing $d'' \vdash \mathbf{b}^{(0)}$.

Then we define **event 2** to be an event that the number of possible distinct secrets of the second sharing among the executions in the set $\mathcal{D}_2(d)$ is strictly greater than $F^{\frac{N \cdot k}{2}}$, i.e., $|\{\mathbf{b}^{(2)} \in \mathbb{F}^{N \cdot k} \mid \exists d'' \in \mathcal{D}_2 \text{ s.t. } d'' \vdash \mathbf{b}^{(2)}\}| > F^{\frac{N \cdot k}{2}}$.

Step 2. Constructing a dishonest execution. If **event 1** and **event 2** happen simultaneously, we claim this breaks the perfect correctness of π . In fact, we first pick $d' = (I'_0, I'_1, I'_2, I'_3; r'_0, r'_1, r'_2, r'_3)$ and $d'' = (I''_0, I''_1, I''_2, I''_3; r''_0, r''_1, r''_2, r''_3)$ as tuples of inputs and internal randomness satisfying the requirements of **event 1** and **event 2**, respectively. I.e., $d' \in \mathcal{D}_1, d'' \in \mathcal{D}_2$. Let $\text{exec}_1 := \text{exec}(\pi, d')$ and $\text{exec}_2 := \text{exec}(\pi, d'')$. Then we construct a dishonest execution denoted by $\text{exec}_3(d', d'')$ as follows and illustrate this construction in Figure 1.

- P_0 takes I_0 as its input, uses r_0 (or equivalently, r'_0, r''_0) as its internal randomness, and honestly follows the protocol.
- P_2 takes I''_2 as its input, uses r''_2 as its internal randomness, and honestly follows the protocol.
- P_3 takes I'_3 as its input, uses r'_3 as its internal randomness, and honestly follows the protocol.
- An adversary controlling parties $\{P_{1,i}\}_{i \in [t]}$ launches a similar attack as in Lemma 2. In particular, P_1 first emulates $\text{exec}_0, \text{exec}_1$, and exec_2 to compute the messages it would send to P_0, P_2, P_3 . Then malicious P_1 behaves inconsistently across parties. When interacting with P_0 , P_1 uses the messages in exec_0 (or equivalently, $\text{exec}_1, \text{exec}_2$); when interacting with P_2 , P_1 uses the messages in exec_3 ; when interacting with P_3 , P_1 uses the messages in exec_1 .

We show that honest party will obtain an incorrect output in $\text{exec}_3(d', d'')$. On the one hand, to argue the output of P_0, P_2 and P_3 in exec_3 , we prove the following claim.

Claim. The transcripts between P_2 and P_1 in executions exec_3 and exec_2 are identical. The transcripts between P_3 and P_1 in executions exec_3 and exec_1 are identical. The transcripts between P_2 and P_3 and transcripts between P_0 and each party in $\{P_1, P_2, P_3\}$ are identical in four executions $\text{exec}_0, \text{exec}_1, \text{exec}_2$ and exec_3 .

Proof. We prove this statement by induction. First, by construction of exec_1 and exec_2 , the transcripts between P_2 and P_3 , as well as the transcripts between P_0 and each party in $\{P_1, P_2, P_3\}$, are identical across $\text{exec}_0, \text{exec}_1$ and exec_2 .

Consider the first message in exec_3 .

- If it is from P_0 to any party in $\{P_1, P_2, P_3\}$, then in exec_3 , since P_0 uses the same input I_0 and internal randomness r_0 as in exec_0 , the message in exec_3 is identical to that in exec_0 , and therefore also identical to those exec_1 and exec_2 .
- If it is from P_1 to P_2 , by construction of exec_3 , since P_1 uses the messages that it would use in exec_2 when interacting with P_2 , this message is identical to that in exec_2 . Similarly, if it is from P_1 to P_3 , by construction of exec_3 , since P_1 uses the messages that it would use in exec_1 when interacting with P_3 , this message is identical to that in exec_1 .
- If it is from P_2 (to any party in $\{P_0, P_1, P_3\}$), in exec_3 , since P_2 uses the same input I_2'' and internal randomness r_2'' as in exec_2 , the message in exec_3 is identical to that in exec_2 . If it is from P_2 to P_3 , it is the same across exec_0 , exec_1 , exec_2 and exec_3 .
- If it is from P_3 (to any party in $\{P_0, P_1, P_2\}$), in exec_3 , since P_3 uses the same input I_3' and internal randomness r_3' as in exec_1 , the message in exec_3 is identical to that in exec_1 . If it is from P_3 to P_2 , it is the same across exec_0 , exec_1 , exec_2 and exec_3 .

Now assume the first $T - 1$ messages already satisfy the statement. We prove this holds for the T -th message.

- If it is from P_0 (to any party in $\{P_1, P_2, P_3\}$), by induction hypothesis, the messages previously sent or received by P_0 are identical in exec_3 and exec_0 . Hence, the current views for P_0 (including its input I_0 , internal randomness r_0 , and messages sent or received by itself) are identical in exec_3 and exec_0 . This implies this message sent from P_0 is identical in exec_3 and exec_0 , and therefore also identical to those exec_1 and exec_2 .
- If it is from P_1 (to any party in $\{P_0, P_2, P_3\}$), by construction of exec_3 , it satisfies the statement.
- If it is from P_2 (to any party in $\{P_0, P_1, P_3\}$), by induction hypothesis, the messages previously sent or received by P_2 are identical in exec_3 and exec_2 . Hence, the current views for P_2 (including its input I_2'' , internal randomness r_2'' , and messages sent or received by itself) are identical in exec_3 and exec_2 . This implies this message sent from P_2 is identical in exec_3 and exec_2 . If this message is from P_2 to P_3 , it is the same across exec_0 , exec_1 , exec_2 and exec_3 .
- If it is from P_3 (to any party in $\{P_0, P_1, P_2\}$), by induction hypothesis, the messages previously sent or received by P_3 are identical in exec_3 and exec_1 . Hence, the current views for P_3 (including its input I_3' , internal randomness r_3' , and messages sent or received by itself) are identical in exec_3 and exec_1 . This implies this message sent from P_3 is identical in exec_3 and exec_1 . If it is from P_3 to P_2 , it is the same across exec_0 , exec_1 , exec_2 and exec_3 .

By induction, the statement holds for all messages in exec_3 . $\square$

According to this claim, when comparing exec_3 and exec_2 , P_2 has the same views in both executions and thus P_2 would output 1 in both executions.

On the other hand, we observe that there exist $d' \in \mathcal{D}_1$ and $d'' \in \mathcal{D}_2$ such that the triples corresponding to the input in $\text{exec}_3(d', d'')$ do not correctly satisfy the multiplication relations. First, by linearity of packed Shamir sharings, we have the following observation.

Claim. Let $(\mathbf{a}^{(3)}, \mathbf{b}^{(3)}, \mathbf{c}^{(3)})$ be the secrets of N packed triples determined by $2t + 1$ parties' input (I_0, I_2'', I_3') in $\text{exec}_3(d', d'')$. Then it holds that

$$(\mathbf{a}^{(3)}, \mathbf{b}^{(3)}, \mathbf{c}^{(3)}) = (\mathbf{a}^{(1)}, \mathbf{b}^{(1)}, \mathbf{c}^{(1)}) + (\mathbf{a}^{(2)}, \mathbf{b}^{(2)}, \mathbf{c}^{(2)}) - (\mathbf{a}^{(0)}, \mathbf{b}^{(0)}, \mathbf{c}^{(0)}).$$

Then $(\mathbf{a}^{(3)}, \mathbf{b}^{(3)}, \mathbf{c}^{(3)}) = (\mathbf{a}^{(1)}, \mathbf{b}^{(2)}, \mathbf{a}^{(0)} * \mathbf{b}^{(2)} + \mathbf{a}^{(1)} * \mathbf{b}^{(0)} - \mathbf{a}^{(0)} * \mathbf{b}^{(0)})$, which implies that the input of $\text{exec}_3(d', d'')$ **doesn't** satisfy the multiplication relation if there exists $\ell \in [N \cdot k]$ such that $a_\ell^{(1)} \neq a_\ell^{(0)}$ and $b_\ell^{(2)} \neq b_\ell^{(0)}$.

Proof. We focus on the secrets of packed triples corresponding to input (I_2'', I_3') in $\text{exec}_3(d', d'')$. For simplicity of notations, we use $s(\bar{I}_2, \bar{I}_3)$ to denote the secrets defined by P_2 's input $\bar{I}_2$ and P_3 's input $\bar{I}_3$. By linearity of packed Shamir sharings, we have

$$s(I_2'', I_3') - s(I_2, I_3) = s(I_2'', I_3) - s(I_2, I_3),$$

which implies $s(I_2'', I_3') = s(I_2, I_3') + s(I_2'', I_3) - s(I_2, I_3)$ and thus

$$\begin{aligned} (\mathbf{a}^{(3)}, \mathbf{b}^{(3)}, \mathbf{c}^{(3)}) &= (\mathbf{a}^{(1)}, \mathbf{b}^{(1)}, \mathbf{c}^{(1)}) + (\mathbf{a}^{(2)}, \mathbf{b}^{(2)}, \mathbf{c}^{(2)}) - (\mathbf{a}^{(0)}, \mathbf{b}^{(0)}, \mathbf{c}^{(0)}) \\ &= (\mathbf{a}^{(1)}, \mathbf{b}^{(0)}, \mathbf{a}^{(1)} * \mathbf{b}^{(0)}) + (\mathbf{a}^{(0)}, \mathbf{b}^{(2)}, \mathbf{a}^{(0)} * \mathbf{b}^{(2)}) - (\mathbf{a}^{(0)}, \mathbf{b}^{(0)}, \mathbf{a}^{(0)} * \mathbf{b}^{(0)}) \\ &= (\mathbf{a}^{(1)}, \mathbf{b}^{(2)}, \mathbf{a}^{(1)} * \mathbf{b}^{(0)} + \mathbf{a}^{(0)} * \mathbf{b}^{(2)} - \mathbf{a}^{(0)} * \mathbf{b}^{(0)}). \end{aligned}$$

Notice that

$$\begin{aligned} \mathbf{a}^{(3)} * \mathbf{b}^{(3)} - \mathbf{c}^{(3)} &= \mathbf{a}^{(1)} * \mathbf{b}^{(2)} - (\mathbf{a}^{(1)} * \mathbf{b}^{(0)} + \mathbf{a}^{(0)} * \mathbf{b}^{(2)} - \mathbf{a}^{(0)} * \mathbf{b}^{(0)}) \\ &= (\mathbf{a}^{(1)} - \mathbf{a}^{(0)}) * (\mathbf{b}^{(2)} - \mathbf{b}^{(0)}), \end{aligned}$$

which means if there exists $\ell \in [N \cdot k]$ such that $a_\ell^{(1)} \neq a_\ell^{(0)}$ and $b_\ell^{(2)} \neq b_\ell^{(0)}$, then $\mathbf{a}^{(3)} * \mathbf{b}^{(3)} - \mathbf{c}^{(3)} \neq \mathbf{0}$. Consequently, in this case, $(\mathbf{a}^{(3)}, \mathbf{b}^{(3)}, \mathbf{c}^{(3)})$ doesn't satisfy the multiplication relation. $\square$

Since **event 1** happens, we have $|\{\mathbf{a}^{(1)} \in \mathbb{F}^{N \cdot k} \mid \exists d' \in \mathcal{D}_1(d) \text{ s.t. } d' \vdash \mathbf{a}^{(1)}\}| > F^{\frac{N \cdot k}{2}}$, which implies that the executions in $\mathcal{D}_1(d)$ correspond to more than $F^{\frac{N \cdot k}{2}}$ distinct tuples of secrets. Hence, there exist at least $\frac{N \cdot k}{2} + 1$ positions such that, for each of these positions, there exists an execution in $\mathcal{D}_1(d)$ whose secret of the first sharing $\mathbf{a}^{(1)}$ differs from $\mathbf{a}^{(0)}$ at that position, i.e., $|\{\ell \mid \exists d' \in \mathcal{D}_1(d), \text{ s.t. } d' \vdash \mathbf{a}^{(1)}, a_\ell^{(1)} \neq a_\ell^{(0)}\}| \geq \frac{N \cdot k}{2} + 1$. Similarly, since **event 2** happens, we have $|\{\mathbf{b}^{(2)} \in \mathbb{F}^{N \cdot k} \mid \exists d'' \in \mathcal{D}_2(d) \text{ s.t. } d'' \vdash \mathbf{b}^{(2)}\}| > F^{\frac{N \cdot k}{2}}$, and therefore $|\{\ell \mid \exists d'' \in \mathcal{D}_2(d), \text{ s.t. } d'' \vdash \mathbf{b}^{(2)}, b_\ell^{(2)} \neq b_\ell^{(0)}\}| \geq \frac{N \cdot k}{2} + 1$. Thus, by $\frac{N \cdot k}{2} + 1 + \frac{N \cdot k}{2} + 1 > N \cdot k$, there exist $d' \in \mathcal{D}_1(d)$ and $d'' \in \mathcal{D}_2(d)$ such that $\exists \ell \in [N \cdot k]$ s.t. $a_\ell^{(1)} \neq a_\ell^{(0)}$ and $b_\ell^{(2)} \neq b_\ell^{(0)}$, where $(\mathbf{a}^{(1)}, \mathbf{b}^{(1)}, \mathbf{c}^{(1)})$ and $(\mathbf{a}^{(2)}, \mathbf{b}^{(2)}, \mathbf{c}^{(2)})$ are the secrets of triples corresponding to d' and d'' , respectively. Then by our observation above, the triples in $\text{exec}_3(d', d'')$ do not satisfy the multiplication relation. Hence, the perfect correctness of π is expected to make sure all honest parties take 0 as output no matter how an adversary deviates from the protocol. Therefore, we figure out an execution $\text{exec}_3(d', d'')$ in which the adversary controlling P_1 manages to let the protocol execution to end up with an incorrect output, resulting in a contradiction with perfect correctness of π .

Step 3. Existence of three honest executions. In order to complete the proof, we are targeted to prove that $\Pr_{d \leftarrow \mathcal{D}}[\text{event 1} \wedge \text{event 2}] > 0$. By union bound, we have that

$$\Pr_{d \leftarrow \mathcal{D}}[\text{event 1} \wedge \text{event 2}] \geq 1 - \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}}] - \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 2}}].$$

Then it suffices to prove $\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}}] < \frac{1}{2}$ and $\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 2}}] < \frac{1}{2}$. We compute $\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}}]$ and the case of **event 2** is similar. Before the computation, we first show that given P_2 's view and part of the input, there exists a sufficiently large number of candidate executions, making use of perfect privacy against a corrupt P_2 . We formally state this as the following claim. Looking ahead, assuming P_0 only has a limited amount of communication in expectation, we are likely to find a large subset of these executions that generate identical views for P_0 (excluding its internal randomness), denoted by $\text{input}^{P_0} \parallel \text{transcript}^{P_0}$.

Claim. Let view^{P_2} be a plausible view of P_2 with 1 as output, i.e., there exists a tuple $d = (I_0, I_1, I_2, I_3; r_0, r_1, r_2, r_3) \in \mathcal{D}$ such that $\text{view}^{P_2}(\pi, d) = \text{view}^{P_2}$ and $\text{output}(\pi, d) = 1$. Then for every $\bar{\mathbf{a}}, \bar{\mathbf{b}} \in \mathbb{F}^{N \cdot k}$, there exists a tuple $\bar{d} = (\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3; \bar{r}_0, \bar{r}_1, r_2, \bar{r}_3) \in \mathcal{D}$ of all parties' inputs and internal randomness such that $\text{view}^{P_2}(\pi, \bar{d}) = \text{view}^{P_2}$ and $(\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3)$ forms degree- $(t + k - 1)$ packed Shamir sharings of $(\bar{\mathbf{a}}, \bar{\mathbf{b}}, \bar{\mathbf{a}} * \bar{\mathbf{b}})$.

Let $(\mathbf{a}, \mathbf{b}, \mathbf{a} * \mathbf{b})$ be the tuple of the secrets defined by d . Let $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \subset \mathcal{D}$ be the set of all honest executions satisfying 1) the view of P_2 is view^{P_2} and 2) the secret of the second sharing is $\mathbf{b}$. Then the number of possible distinct secrets of the first sharing among the executions in the set $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}$ is $F^{N \cdot k}$, i.e.,

$$|\{\bar{\mathbf{a}} \in \mathbb{F}^{N \cdot k} \mid \exists \bar{d} \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \text{ s.t. } \bar{d} \vdash \bar{\mathbf{a}}\}| = F^{N \cdot k}.$$

Proof. By perfect privacy against a (semi-honest) corrupted P_2 , there exists a simulator $\mathcal{S}_2$ which takes as input the input and output of P_2 , and outputs the view of P_2 such that for any inputs $(\bar{I}_0, \bar{I}_1, \bar{I}_3)$ of P_0, P_1, P_3 such that $(\bar{I}_0, I_2, \bar{I}_1, \bar{I}_3)$ forms valid degree- $(t+k-1)$ packed Shamir sharings, it holds that

$$\mathcal{S}_2(I_2, F_{\text{vrfy-triple}}(\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3)) \equiv \text{view}^{P_2}(\pi, \bar{I}_0, \bar{I}_1, I_2, \bar{I}_3),$$

where the function $F_{\text{vrfy-triple}}(\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3)$ is defined to output 1 if the input tuple $(\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3)$ corresponds to packed triples satisfying the multiplication relations, and 0 otherwise. Hence, for every $\bar{\mathbf{a}}, \bar{\mathbf{b}} \in \mathbb{F}^k$, letting $(\bar{I}_0, \bar{I}_1, \bar{I}_3)$ be the input tuple from P_0, P_1 and P_3 such that $(\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3)$ forms degree- $(t+k-1)$ packed Shamir sharings of $(\bar{\mathbf{a}}, \bar{\mathbf{b}}, \bar{\mathbf{a}} * \bar{\mathbf{b}})$, by property of $F_{\text{vrfy-triple}}$, we have

$$\begin{aligned} \text{view}^{P_2}(\pi, I_0, I_1, I_2, I_3) &\equiv \mathcal{S}_2(I_2, F_{\text{vrfy-triple}}(I_0, I_1, I_2, I_3)) \\ &\equiv \mathcal{S}_2(I_2, F_{\text{vrfy-triple}}(\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3)) \\ &\equiv \text{view}^{P_2}(\pi, \bar{I}_0, \bar{I}_1, I_2, \bar{I}_3). \end{aligned}$$

Then there exists a choice of a tuple of internal randomness from P_0, P_1 and P_3 , $(\bar{r}_0, \bar{r}_1, \bar{r}_3)$, such that $\text{view}^{P_2}(\pi, d) = \text{view}^{P_2}(\bar{I}_0, \bar{I}_1, I_2, \bar{I}_3; \bar{r}_0, \bar{r}_1, \bar{r}_2, \bar{r}_3)$.

By the reasoning above, fixing $\bar{\mathbf{b}} = \mathbf{b}$, for any $\bar{\mathbf{a}} \in \mathbb{F}^{N \cdot k}$, $\bar{\mathbf{a}}$ can lead to a satisfied execution in $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}$. Notice that there are $F^{N \cdot k}$ possibilities of $\bar{\mathbf{a}} \in \mathbb{F}^{N \cdot k}$. Then we conclude the number of possible distinct secrets of the first sharing among the executions in $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}$ is $F^{N \cdot k}$. $\square$

Next, we show that, given the view of P_2 view^{P_2} , and a fixed value of the second sharing's secret $\mathbf{b}$, the executions consistent with view^{P_2} and $\mathbf{b}$ are uniformly distributed over all possible values of the first sharing's secret $\bar{\mathbf{a}}$. This is formally stated in the following claim. Consequently, if there exist sufficiently many executions all consistent with the view of P_2 view^{P_2} , the second sharing's secret $\mathbf{b}$, and P_0 's input together with its transcript $\text{input}^{P_0} || \text{transcript}^{P_0}$, it implies the existence of a large number of distinct possible values for $\bar{\mathbf{a}}$, which in turn implies **event 1** occurs.

Claim. Given a view of P_2 view^{P_2} and a secret of the second sharing $\mathbf{b}$, we define the set $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}$ as in the previous claim. For any two vectors of secrets $\bar{\mathbf{a}}_1, \bar{\mathbf{a}}_2 \in \mathbb{F}^{N \cdot k}$, the number of executions in $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}$ with the first sharing's secret being $\bar{\mathbf{a}}_1$ is equal to the number of executions with the first sharing's secret being $\bar{\mathbf{a}}_2$, i.e.,

$$|\{d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \mid d \vdash \bar{\mathbf{a}}_1\}| = |\{d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \mid d \vdash \bar{\mathbf{a}}_2\}|.$$

Proof. Intuitively, by privacy against a corrupted P_2 , corrupted P_2 cannot distinguish different inputs according to its view. The formal proof goes as follows.

Given that the inputs from all parties form degree- $(t+k-1)$ packed triples satisfy the multiplication relations, a tuple $(\bar{\mathbf{a}}, \mathbf{b}, I_2)$ fully determines a tuple of inputs from all parties. For simplicity of notation, we use $(\bar{\mathbf{a}}, \mathbf{b}, I_2)$ to denote the input of all parties. Then by perfect privacy against a (semi-honest) corrupted P_2 ,

$$\text{view}^{P_2}(\pi, \bar{\mathbf{a}}_1, \mathbf{b}, I_2) \equiv \mathcal{S}_2(I_2, 1) \equiv \text{view}^{P_2}(\pi, \bar{\mathbf{a}}_2, \mathbf{b}, I_2).$$

This implies $\Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2} \mid d \vdash \bar{a}_1, \mathbf{b}, I_2] = \Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2} \mid d \vdash \bar{a}_2, \mathbf{b}, I_2]$. Then we get

$$\begin{aligned}
\frac{|\{d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \mid d \vdash \bar{a}_1\}|}{|\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} &= \Pr_d[d \vdash \bar{a}_1 \mid d \vdash \text{view}^{P_2}, \mathbf{b}, I_2] \\
&= \frac{\Pr_d[d \vdash \text{view}^{P_2}, \bar{a}_1, \mathbf{b}, I_2]}{\Pr_d[d \vdash \text{view}^{P_2}, \mathbf{b}, I_2]} \\
&= \frac{\Pr_d[d \vdash \text{view}^{P_2}, \bar{a}_1, \mathbf{b}, I_2]}{\Pr_d[d \vdash \bar{a}_1, \mathbf{b}, I_2]} \cdot \frac{\Pr_d[d \vdash \bar{a}_1, \mathbf{b}, I_2]}{\Pr_d[d \vdash \text{view}^{P_2}, \mathbf{b}, I_2]} \\
&= \Pr_d[d \vdash \text{view}^{P_2} \mid d \vdash \bar{a}_1, \mathbf{b}, I_2] \cdot \frac{\Pr_d[d \vdash \bar{a}_2, \mathbf{b}, I_2]}{\Pr_d[d \vdash \text{view}^{P_2}, \mathbf{b}, I_2]} \tag{1} \\
&= \Pr_d[d \vdash \text{view}^{P_2} \mid d \vdash \bar{a}_2, \mathbf{b}, I_2] \cdot \frac{\Pr_d[d \vdash \bar{a}_2, \mathbf{b}, I_2]}{\Pr_d[d \vdash \text{view}^{P_2}, \mathbf{b}, I_2]} \tag{2} \\
&= \frac{\Pr_d[d \vdash \text{view}^{P_2}, \bar{a}_2, \mathbf{b}, I_2]}{\Pr_d[d \vdash \bar{a}_2, \mathbf{b}, I_2]} \cdot \frac{\Pr_d[d \vdash \bar{a}_2, \mathbf{b}, I_2]}{\Pr_d[d \vdash \text{view}^{P_2}, \mathbf{b}, I_2]} \\
&= \frac{\Pr_d[d \vdash \text{view}^{P_2}, \bar{a}_2, \mathbf{b}, I_2]}{\Pr_d[d \vdash \text{view}^{P_2}, \mathbf{b}, I_2]} \\
&= \Pr_d[d \vdash \bar{a}_2 \mid d \vdash \text{view}^{P_2}, \mathbf{b}, I_2] \\
&= \frac{|\{d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \mid d \vdash \bar{a}_2\}|}{|\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|},
\end{aligned}$$

where Equality 1 follows from the fact that d is sampled uniformly at random and Equality 2 holds because of the equality derived previously. This completes the proof of the claim. $\square$

Now we upper bound $\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}}]$ as follows. Let E_1 be the event $(C_0(d) < 3 \cdot \mathbb{E}[C_0])$. For simplicity of notation, we abuse the notation E_1 to also denote the set of all executions in which $C_0(d) < 3 \cdot \mathbb{E}[C_0]$, i.e., $E_1 = \{d \in \mathcal{D} \mid C_0(d) < 3 \cdot \mathbb{E}[C_0]\}$.

$$\begin{aligned}
&\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}}] \\
&= \sum_{\text{view}^{P_2}, \mathbf{b}} \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&= \sum_{\text{view}^{P_2}, \mathbf{b}} \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \wedge E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] + \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \wedge \overline{E_1} \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&= \sum_{\text{view}^{P_2}, \mathbf{b}} \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \cdot \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&\quad + \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid \overline{E_1} \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \cdot \Pr_{d \leftarrow \mathcal{D}}[\overline{E_1} \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&= \sum_{\text{view}^{P_2}, \mathbf{b}} \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \cdot \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&\quad + \Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid \overline{E_1} \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \cdot (\Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}, \mathbf{b}] - \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}])
\end{aligned}$$

$$\begin{aligned}
&< \sum_{\text{view}^{P_2}, \mathbf{b}} \frac{1}{6} \cdot \frac{1}{\Pr_{d \leftarrow \mathcal{D}}[E_1 \mid d \vdash \text{view}^{P_2}, \mathbf{b}]} \cdot \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&\quad + 1 \cdot \left(\Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}, \mathbf{b}] - \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \right) \\
&= \sum_{\text{view}^{P_2}, \mathbf{b}} \frac{1}{6} \cdot \Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}, \mathbf{b}] + \Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}, \mathbf{b}] - \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&= 1 + \frac{1}{6} - \Pr_{d \leftarrow \mathcal{D}}[E_1] \\
&\leq 1 + \frac{1}{6} - \left(1 - \frac{1}{3}\right) \\
&= \frac{1}{2}.
\end{aligned}$$

The deduction above takes advantage of the following two inequalities:

1. $\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] < \frac{1}{6} \cdot \frac{1}{\Pr_{d \leftarrow \mathcal{D}}[E_1 \mid d \vdash \text{view}^{P_2}, \mathbf{b}]}$. We prove this as below.

To compute the probability, we first introduce some notations. For simplicity, given an honest execution defined by d , we denote a party P 's input concatenated with its transcript by $\sigma^P(\pi, d)$, i.e., $\sigma^P(\pi, d) = \text{input}^P(\pi, d) \parallel \text{transcript}^P(\pi, d)$. Define $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \subset \mathcal{D}$ to the set of all executions in which the view of P_2 is view^{P_2} and the secret of the second sharing is $\mathbf{b}$, i.e., $\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} = \{d \in \mathcal{D} \mid d \vdash \text{view}^{P_2}, \mathbf{b}\}$. Define $\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}} \subset \mathcal{D}$ to be the set of all executions in which

- the view of P_2 is view^{P_2} ,
- the secret of the second sharing is $\mathbf{b}$,
- the input and transcript of P_0 is σ^{P_0} ,
- the possibilities of distinct values of the secret of the first sharing is no more than $F^{\frac{N \cdot k}{2}}$.

Notice that

$$\begin{aligned}
&\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid E_1 \wedge d \vdash \text{view}^{P_2}, \mathbf{b}] \\
&= \frac{\sum_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \tag{3}
\end{aligned}$$

$$\leq \frac{|\{\sigma^{P_0}(\pi, d) \mid d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}\}| \cdot \max_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \tag{4}$$

$$\leq \frac{F^{3N+3 \cdot \mathbb{E}[C_0]} \cdot \max_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \tag{5}$$

$$\begin{aligned}
&= \frac{F^{3N+3 \cdot \mathbb{E}[C_0]} \cdot \max_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}|}{|\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \cdot \frac{|\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \\
&\leq \frac{F^{3N+3 \cdot \mathbb{E}[C_0]} \cdot F^{\frac{N \cdot k}{2}}}{F^{N \cdot k}} \cdot \frac{|\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \tag{6}
\end{aligned}$$

$$< \frac{1}{6} \cdot \frac{|\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \tag{7}$$

$$= \frac{1}{6} \cdot \frac{1}{\Pr_{d \leftarrow \mathcal{D}}[E_1 \mid d \vdash \text{view}^{P_2}, \mathbf{b}]},$$

where the step-by-step reasoning is given as follows. To show Equality 3, we group all executions $E_1 \cap \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}$ according to P_0 's input and transcript. Then event 1 doesn't happen if and only if the sampled execution d lies in the set $\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}$ for some σ^{P_0} .

Inequality 4 holds because $\sum_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}| \leq \sum_{\sigma^{P_0}} \max_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}| = |\{\sigma^{P_0}(\pi, d) \mid d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}\}| \cdot \max_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}|$.

As for Inequality 5, because for any execution in E_1 , the length of P_0 's input together with its transcript is at most $3N + 3 \cdot \mathbb{E}[C_0]$, resulting in at most $F^{3N+3 \cdot \mathbb{E}[C_0]}$ possibilities.

Regarding Inequality 6, by two claims above and the definition of $\text{Bad}_{\text{view}^{P_2}, \mathbf{b}}$, we have

$$\frac{\max_{\sigma^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{b}, \sigma^{P_0}}|}{|\mathcal{D}_{\text{view}^{P_2}, \mathbf{b}}|} \leq \frac{F^{\frac{N \cdot k}{2}} \cdot |\{d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \mid d \vdash \bar{\mathbf{a}}\}|}{F^{N \cdot k} \cdot |\{d \in \mathcal{D}_{\text{view}^{P_2}, \mathbf{b}} \mid d \vdash \bar{\mathbf{a}}\}|} = \frac{F^{\frac{N \cdot k}{2}}}{F^{N \cdot k}},$$

for any $\bar{\mathbf{a}} \in \mathbb{F}^{N \cdot k}$ representing the secret of the first sharing.

Finally, for Inequality 7, by assuming that $\mathbb{E}[C_0] < \frac{N \cdot k}{8}$ and setting $c_1 = \lceil \frac{24}{\log F} \rceil, c_2 = 24$, for all $N > c_1, k > c_2$, we have that

$$\frac{F^{3N+3 \cdot \mathbb{E}[C_0]} \cdot F^{\frac{N \cdot k}{2}}}{F^{N \cdot k}} < \frac{F^{3N}}{F^{\frac{N \cdot k}{8}}} \leq \frac{F^{3N}}{F^{\frac{25N}{8}}} = \frac{1}{F^{N/8}} < \frac{1}{6}.$$

2. $\Pr_{d \leftarrow \mathcal{D}}[E_1] \geq 1 - \frac{1}{3}$. We prove this as below.

By Markov's inequality, we have

$$\Pr_{d \leftarrow \mathcal{D}}[C_0(d) \geq 3 \cdot \mathbb{E}[C_0]] \leq \frac{\mathbb{E}[C_0]}{3 \cdot \mathbb{E}[C_0]} = \frac{1}{3}.$$

Then we have $\Pr_{d \leftarrow \mathcal{D}}[E_1] = 1 - \Pr_{d \leftarrow \mathcal{D}}[C_0(d) \geq 3 \cdot \mathbb{E}[C_0]] \geq 1 - \frac{1}{3}$.

According to previous analysis, we prove the statement. $\square$

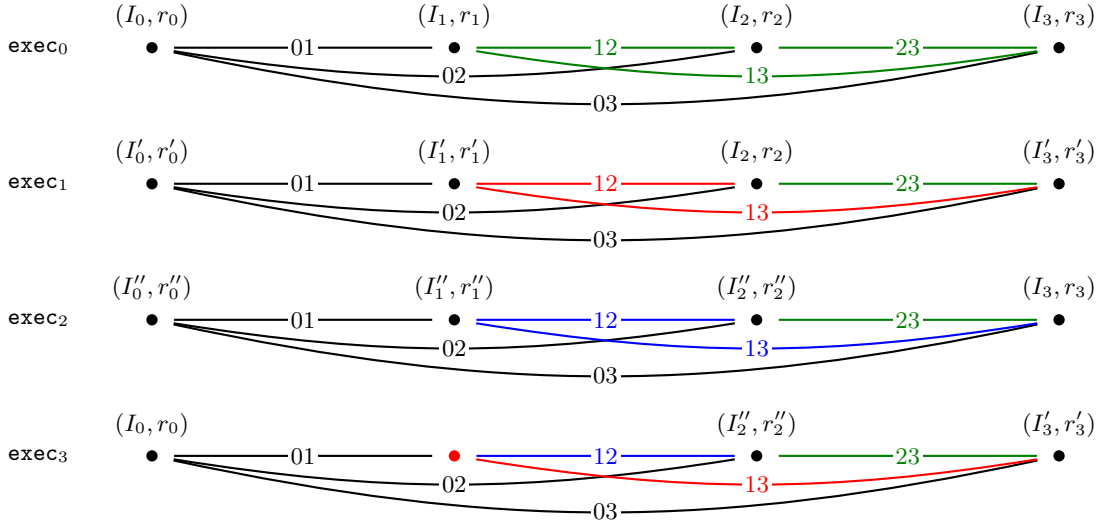


Fig. 1. Construction of four executions. In the figure, each execution is represented by four nodes and six edges. Each node corresponds to a party that takes (I, r) as input and internal randomness. Each edge labeled ij denotes the transcript between parties P_i and P_j , for $0 \leq i < j \leq 3$. In the first three executions, all parties honestly follow the protocol. In the fourth execution **exec3**, party P_1 (highlighted in **red**) is malicious and behaves inconsistently across different parties. As proved in the first claim of the proof, the transcript between P_2 and P_1 in **exec3** is identical to that in **exec2**, shown in **blue**; the transcript between P_3 and P_1 in **exec3** is identical to that in **exec1**, shown in **red**. The transcript between P_2 and P_3 is identical across all four executions and is shown in **green**. Finally, the transcripts between P_0 and each party in $\{P_1, P_2, P_3\}$ are identical across all executions and are shown in black.

5.3 Communication Complexity of Preparing Packed Triples over Bivariate Polynomials

We are now ready to derive a communication lower bound for preparing error-free packed Beaver triples over bivariate polynomials with perfect security with abort whose functionality is characterized as $\mathcal{F}_{\text{packed-triple-with-abort}}$. As a side-note, for protocols preparing preprocessing data, there is no gap between security with abort and security with identifiable abort. As if one has a protocol realizing security with abort, then in the event of an abort, after all parties open their views during the protocol execution, a disputed pair is expected to be identified, achieving security with identifiable abort. This indicates that a protocol that securely computes $\mathcal{F}_{\text{packed-triple-with-abort}}$ would imply the existence of a protocol that securely computes $\mathcal{F}_{\text{packed-triple}}$.

To establish a lower bound for $\mathcal{F}_{\text{packed-triple-with-abort}}$, we employ the technique of sacrificing to construct an efficient protocol that verifies the correctness of packed triples assuming error-free packed triples. In particular, using the batch-wise verification technique in [BH08], we can efficiently verify whether all (honest) parties jointly hold consistent linear secret sharings. Towards verifying the correctness of a packed Beaver triple $([a]_{t+k-1}, [b]_{t+k-1}, [c]_{t+k-1})$, all parties first prepare an error-free packed Beaver triple $([\tilde{a}]_{t+k-1}, [\tilde{b}]_{t+k-1}, [\tilde{c}]_{t+k-1})$ with a guarantee that $\tilde{a} * \tilde{b} = \tilde{c}$, make use of it to compute their shares of $[a * b - c]_{t+2k-2}$ following Beaver multiplication template. After verifying all parties honestly perform the computation of $[a * b - c]_{t+2k-2}$, which corresponds to a task of verifying consistency of linear secret sharings, to check the correctness of $([a]_{t+k-1}, [b]_{t+k-1}, [c]_{t+k-1})$, they only need to verify whether $[a * b - c]_{t+2k-2}$ is a valid degree- $(t + 2k - 2)$ packed Shamir sharing with secret $\mathbf{0}$, which again corresponds to a task of verifying consistency of linear secret sharings. Notice that, when this verification fails, the adversary will additionally learn the information regarding $a * b - c$. However, this suffices to achieve our goal as we are only targeted for perfect security with abort, where privacy is not guaranteed in the event of an abort.

This effectively reduces the task of verifying packed triples to that of preparing error-free packed triples. As a result, a communication lower bound of verifying packed triples would imply that of preparing error-free packed triples. The communication lower bound is stated in Theorem 7, and the protocol for verifying packed triples is presented in $\Pi_{\text{vrfy-triple-with-abort}}$, with its security property concluded in Lemma 4.

Functionality 9: $\mathcal{F}_{\text{packed-triple-with-abort}}(N, k)$

1. $\mathcal{F}_{\text{packed-triple-with-abort}}$ receives a set $\mathcal{C}$ of corrupted parties' identities.
2. $\mathcal{F}_{\text{packed-triple-with-abort}}$ receives from the adversary shares of corrupted parties $\{f_{\ell,i}^a(x), g_{\ell,i}^a(y), f_{\ell,i}^b(x), g_{\ell,i}^b(y), f_{\ell,i}^c(x), g_{\ell,i}^c(y)\}_{\ell \in [N], i \in \mathcal{C}}$ with a promise that
 - $f_{\ell,i}^z(x), g_{\ell,i}^z(y)$ are degree- $(t + k - 1)$ polynomials for $\ell \in [N], i \in \mathcal{C}, z \in \{a, b, c\}$,
 - $f_{\ell,i}^z(\alpha_j) = g_{\ell,j}^z(\alpha_i)$ for every $\ell \in [N], i, j \in \mathcal{C}, z \in \{a, b, c\}$.
3. For $\ell \in [N]$, $\mathcal{F}_{\text{packed-triple-with-abort}}$ samples a packed triple over bivariate polynomials $([\mathbf{A}_\ell]_{t+k-1}, [\mathbf{B}_\ell]_{t+k-1}, [\mathbf{C}_\ell]_{t+k-1})$, where $\mathbf{C}_\ell = \mathbf{A}_\ell * \mathbf{B}_\ell$, $\mathbf{A}_\ell = (A_{\ell,ij})_{ij}$, $\mathbf{B}_\ell = (B_{\ell,ij})_{ij}$, and $\mathbf{C}_\ell = (C_{\ell,ij})_{ij}$, according to shares of corrupted parties. Concretely, $\mathcal{F}_{\text{packed-triple-with-abort}}$ samples three degree- $(t + k - 1, t + k - 1)$ bivariate polynomials $S_\ell^a(x, y), S_\ell^b(x, y), S_\ell^c(x, y)$ such that
 - $S_\ell^a(\beta_i, \beta_j) = A_{\ell,ij}$, $S_\ell^b(\beta_i, \beta_j) = B_{\ell,ij}$, and $S_\ell^c(\beta_i, \beta_j) = C_{\ell,ij}$ for $x', y' \in [k]$,
 - $S_\ell^z(x, \alpha_i) = f_{\ell,i}^z(x)$, $S_\ell^z(\alpha_i, y) = g_{\ell,i}^z(y)$ for $i \in \mathcal{C}, z \in \{a, b, c\}$.
4. $\mathcal{F}_{\text{packed-triple-with-abort}}$ waits for an instruction from the adversary.
 - If $\mathcal{F}_{\text{packed-triple-with-abort}}$ receives **accept** from the adversary, then for each $\ell \in [N], i \in [n], z \in \{a, b, c\}$, $\mathcal{F}_{\text{packed-triple-with-abort}}$ computes $f_{\ell,i}^z(x) := S_\ell^z(x, \alpha_i)$, $g_{\ell,i}^z(y) := S_\ell^z(\alpha_i, y)$ and sends $\{f_{\ell,i}^z(x), g_{\ell,i}^z(y)\}_{\ell \in [N], z \in \{a, b, c\}}$ to P_i as its shares of $\{[\mathbf{A}_\ell]_{t+k-1}, [\mathbf{B}_\ell]_{t+k-1}, [\mathbf{C}_\ell]_{t+k-1}\}_{\ell \in [N]}$.
 - If $\mathcal{F}_{\text{packed-triple-with-abort}}$ receives **reject** from the adversary, $\mathcal{F}_{\text{packed-triple-with-abort}}$ outputs **abort** to all parties.

Theorem 7. Let $n = 3t + 1$, $k \in \mathbb{N}$ such that $1 \leq k \leq \frac{t+2}{2}$, and $N > n$. There exists a constant $c > 0$ such that for all $k > c$, any protocol computing $\mathcal{F}_{\text{packed-triple-with-abort}}(N, k)$ with perfect security against a fully malicious adversary controlling at most t corrupted parties communicates at least $\frac{N \cdot k^2 \cdot n}{32}$ field elements in

expectation, when $\mathcal{F}_{\text{packed-triple-with-abort}}$ doesn't abort. This lower bound implies an amortized cost of $\Omega(n)$ elements per secret.

Protocol 8: $\Pi_{\text{vrfy-triple-with-abort}}(N, k)$ in $\mathcal{F}_{\text{packed-triple-with-abort}}, \mathcal{F}_{\text{vrfy-linear-with-abort}}, \mathcal{F}_{\text{rand-sharing}}$ -hybrid model

All parties hold their shares of $\{[a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1}\}_{\ell \in [N]}$ and aim to check whether $c_\ell = a_\ell * b_\ell$ for each $\ell \in [N]$.

- 1: **Preparing packed triples.** All parties invoke $\mathcal{F}_{\text{packed-triple-with-abort}}(N, k)$ to obtain N packed triples over bivariate polynomials and transform them into packed triples over univariate polynomials, which are denoted by $\{[\tilde{a}_\ell]_{t+k-1}, [\tilde{b}_\ell]_{t+k-1}, [\tilde{c}_\ell]_{t+k-1}\}_{\ell \in [N]}$.^a
 - If $\mathcal{F}_{\text{packed-triple-with-abort}}$ outputs **abort**, all parties take 0 as output and abort. Otherwise, all parties proceed.
- 2: **Preparing random masks.** All parties invoke $\mathcal{F}_{\text{rand-sharing}}(\Sigma_1, N)$ to prepare N random degree- $(t + 2k - 2)$ packed Shamir sharings $\{[o_\ell]_{t+2k-2}\}_{\ell \in [N]}$ with each $o_\ell = \mathbf{0} \in \mathbb{F}^k$.
- 3: **Sacrifice.** For each $\ell \in [N]$, all parties sacrifice a packed triple $([\tilde{a}_\ell]_{t+k-1}, [\tilde{b}_\ell]_{t+k-1}, [\tilde{c}_\ell]_{t+k-1})$ to check the correctness of $([a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1})$ as follows.
 1. All parties locally compute $[a_\ell + \tilde{a}_\ell]_{t+k-1} \leftarrow [a_\ell]_{t+k-1} + [\tilde{a}_\ell]_{t+k-1}$, $[b_\ell + \tilde{b}_\ell]_{t+k-1} \leftarrow [b_\ell]_{t+k-1} + [\tilde{b}_\ell]_{t+k-1}$ and send them to P_{king} .
 2. P_{king} reconstructs the secrets $a_\ell + \tilde{a}_\ell, b_\ell + \tilde{b}_\ell$ and distributes $[a_\ell + \tilde{a}_\ell]_{k-1}, [b_\ell + \tilde{b}_\ell]_{k-1}$ to all parties.
 3. All parties locally compute

$$\begin{aligned} [\theta_\ell]_{t+2k-2} &\leftarrow [a_\ell + \tilde{a}_\ell]_{k-1} \cdot [b_\ell + \tilde{b}_\ell]_{k-1} - [\tilde{a}_\ell]_{t+k-1} \cdot [b_\ell + \tilde{b}_\ell]_{k-1} \\ &\quad - [a_\ell + \tilde{a}_\ell]_{k-1} \cdot [\tilde{b}_\ell]_{t+k-1} + [\tilde{c}_\ell]_{t+k-1} - [c_\ell]_{t+k-1} + [o_\ell]_{t+2k-2}. \end{aligned}$$

- 4: **Verification.** All parties first invoke $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7, 2N)$ with their shares of $\{([a_\ell + \tilde{a}_\ell]_{t+k-1}, [a_\ell + \tilde{a}_\ell]_{k-1})\}_{\ell \in [N]}$ and $\{([b_\ell + \tilde{b}_\ell]_{t+k-1}, [b_\ell + \tilde{b}_\ell]_{k-1})\}_{\ell \in [N]}$ to check whether P_{king} honestly follows the protocol.
 - If $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ outputs 0, all parties take 0 as output.
 - Otherwise, all parties proceed as follows.

All parties then invoke $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1, N)$ with their shares of $\{[\theta_\ell]_{t+2k-2}\}_{\ell \in [N]}$ to check whether they hold valid degree- $(t + 2k - 2)$ packed Shamir sharings with secret $\mathbf{0} \in \mathbb{F}^k$.

 - If $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ output 0, all parties take 0 as output.
 - Otherwise, all parties take 1 as output.

^a Suppose all parties obtain their shares of $[\mathbf{A}]_{t+k-1}, [\mathbf{B}]_{t+k-1}, [\mathbf{C}]_{t+k-1}$ by invoking $\mathcal{F}_{\text{packed-triple-with-abort}}$, and each party P_i obtains its shares as $(f_i^a(x), g_i^a(y), f_i^b(x), g_i^b(y), f_i^c(x), g_i^c(y))$, which are degree- $(t + k - 1)$ polynomials. Suppose $\mathbf{A} = (\mathbf{a}_j)_{j \in [k]}$, where $\mathbf{a}_j \in \mathbb{F}^k$ is the j -th row of $\mathbf{A}$. Then for each $j \in [k]$, P_i can locally compute its share of $[a_j]_{t+k-1}$ as $g_i^a(\beta_j)$. Similarly, P_i can also locally transform its shares of $[\mathbf{B}]_{t+k-1}, [\mathbf{C}]_{t+k-1}$ into its shares of packed Shamir sharings over univariate polynomials.

Lemma 4. Let $t \leq (n - 1)/3$, $k \in \mathbb{N}$ such that $1 \leq k \leq \frac{t+2}{2}$ and $N > n$. $\Pi_{\text{vrfy-triple-with-abort}}(N, k)$ computes $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ with perfect security in $\mathcal{F}_{\text{packed-triple-with-abort}}, \mathcal{F}_{\text{rand-sharing}}, \mathcal{F}_{\text{vrfy-linear-with-abort}}$ -hybrid model against any fully malicious adversary controlling t corrupted parties. The communication of $\Pi_{\text{vrfy-triple-with-abort}}(N, k)$ is $O(N \cdot n)$ field elements plus one invocation of $\mathcal{F}_{\text{packed-triple-with-abort}}(\lceil \frac{N}{k} \rceil, k)$.

Proof (Proof of Lemma 4). We will construct a simulator $\mathcal{S}$ to simulate the behaviors of honest parties. Let $\mathcal{C}$ denote the set of corrupted parties and $\mathcal{H}$ denote the set of honest parties. The core of the simulation is to ensure privacy for honest parties is protected when verification succeeds, making use of the randomness of error-free packed Beaver triples and degree- $(t + 2k - 2)$ packed Shamir sharings with secret $\mathbf{0}$.

The simulator $\mathcal{S}$ internally invokes the adversary with randomness sampled by itself and works as follows. First, $\mathcal{S}$ and all honest parties invoke $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ with honest parties providing their shares of $\{([a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1})\}_{\ell \in [N]}$. Then $\mathcal{S}$ receives corrupted parties' shares of $\{([a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1})\}_{\ell \in [N]}$ from $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ and records them for the simulation below.

1. In Step 1, $\mathcal{S}$ emulates $\mathcal{F}_{\text{packed-triple-with-abort}}$ by receiving from the adversary corrupted parties' shares of $\{([\tilde{a}_\ell]_{t+k-1}, [\tilde{b}_\ell]_{t+k-1}, [\tilde{c}_\ell]_{t+k-1})\}_{\ell \in [N]}$ and an instruction.
 - If $\mathcal{S}$ receives **reject**, $\mathcal{S}$ outputs **abort** on behalf of $\mathcal{F}_{\text{packed-triple-with-abort}}$. Then $\mathcal{S}$ sends **reject** to $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ and go to the last step of the simulation.
 - If $\mathcal{S}$ receives **accept**, $\mathcal{S}$ sends back corrupted parties' shares to the adversary on behalf of $\mathcal{F}_{\text{packed-triple-with-abort}}$, and proceeds the simulation.
2. In Step 2, $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand-sharing}}(\Sigma_1)$ by receiving corrupted parties' shares of $\{[o_\ell]_{t+2k-2}\}_{\ell \in [N]}$ from the adversary and sending them back to the adversary.

Simulation of Sacrifice.

3. In Step 3.1, for each $\ell \in [N]$, $\mathcal{S}$ computes corrupted parties' shares of $[a_\ell + \tilde{a}_\ell]_{t+k-1}$ and $[b_\ell + \tilde{b}_\ell]_{t+k-1}$ as $[a_\ell + \tilde{a}_\ell]_{t+k-1} \leftarrow [a_\ell]_{t+k-1} + [\tilde{a}_\ell]_{t+k-1}$, $[b_\ell + \tilde{b}_\ell]_{t+k-1} \leftarrow [b_\ell]_{t+k-1} + [\tilde{b}_\ell]_{t+k-1}$, samples random degree- $(t+k-1)$ packed Shamir sharings as $[\overline{a_\ell + \tilde{a}_\ell}]_{t+k-1}$ and $[\overline{b_\ell + \tilde{b}_\ell}]_{t+k-1}$ based on corrupted parties' shares, and sends the shares of honest parties to P_{king} on behalf of honest parties.
4. In Step 3.2, if P_{king} is honest, $\mathcal{S}$ follows the protocol honestly. On receiving honest parties' shares of $[\overline{a_\ell + \tilde{a}_\ell}]_{k-1}$, $[\overline{b_\ell + \tilde{b}_\ell}]_{k-1}$ from P_{king} , $\mathcal{S}$ checks as follows.
 - If one of them does not form a valid degree- $(k-1)$ packed Shamir sharing, $\mathcal{S}$ latter will output 0 when $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$ is invoked.
 - If they both form valid degree- $(k-1)$ packed Shamir sharings, $\mathcal{S}$ recovers the whole sharings and checks whether the secrets of $[\overline{a_\ell + \tilde{a}_\ell}]_{k-1}$ and $[\overline{b_\ell + \tilde{b}_\ell}]_{k-1}$ are the same as that of $[a_\ell + \tilde{a}_\ell]_{t+k-1}$ and $[b_\ell + \tilde{b}_\ell]_{t+k-1}$, respectively. If not, $\mathcal{S}$ latter will output 0 when $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$ is invoked.

Simulation of verification and output.

5. In Step 4, when emulating $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$, $\mathcal{S}$ sends honest parties' shares of $\{[a_\ell + \tilde{a}_\ell]_{t+k-1}, [\overline{a_\ell + \tilde{a}_\ell}]_{k-1}\}_{\ell \in [N]}$ and $\{[b_\ell + \tilde{b}_\ell]_{t+k-1}, [\overline{b_\ell + \tilde{b}_\ell}]_{k-1}\}_{\ell \in [N]}$ computed in Step 3 of the simulation to the adversary on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$.
 - If all checks pass in the last step, i.e., $\overline{a_\ell + \tilde{a}_\ell} = a_\ell + \tilde{a}_\ell$ and $\overline{b_\ell + \tilde{b}_\ell} = b_\ell + \tilde{b}_\ell$ for each $\ell \in [N]$, then $\mathcal{S}$ receives an instruction from the adversary.
 - If $\mathcal{S}$ receives **reject** from the adversary, $\mathcal{S}$ outputs 0 to the adversary on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$. Then $\mathcal{S}$ sends **reject** to $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ and go to the last step of the simulation.
 - If $\mathcal{S}$ receives **accept** from the adversary, $\mathcal{S}$ outputs 1 to the adversary, and proceeds to next step.
 - If one of checks in the last step fails, $\mathcal{S}$ outputs 0 to the adversary on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$. Then $\mathcal{S}$ sends **reject** to $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ and go to the last step of the simulation.
6. When emulating $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$,
 - if $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ asks for an instruction, which means all checks in $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ pass, i.e., $\theta_\ell = \mathbf{0}$ for each $\ell \in [N]$, the emulation of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$ is as follows. $\mathcal{S}$ first computes corrupted parties' shares of $[\theta_\ell]_{t+2k-2}$ as $[\theta_\ell]_{t+2k-2} \leftarrow [a_\ell + \tilde{a}_\ell]_{k-1} \cdot [b_\ell + \tilde{b}_\ell]_{k-1} - [\tilde{a}_\ell]_{t+k-1} \cdot [b_\ell + \tilde{b}_\ell]_{k-1} - [a_\ell + \tilde{a}_\ell]_{k-1} \cdot [\tilde{b}_\ell]_{t+k-1} + [\tilde{c}_\ell]_{t+k-1} - [c_\ell]_{t+k-1} + [o_\ell]_{t+2k-2}$ for each $\ell \in [N]$. Based on the secrets $\theta_\ell = \mathbf{0}$ and the shares of corrupted parties, $\mathcal{S}$ randomly samples N degree- $(t+2k-2)$ packed Shamir sharings $\{[\theta_\ell]_{t+2k-2}\}_{\ell \in [N]}$. Then $\mathcal{S}$ sends honest parties' shares of $\{[\theta_\ell]_{t+2k-2}\}_{\ell \in [N]}$ to the adversary on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$.
 - If $\mathcal{S}$ receives **reject** from the adversary, $\mathcal{S}$ outputs 0 to the adversary on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}$, sends **reject** to $\mathcal{F}_{\text{vrfy-triple-with-abort}}$, and go to the last step of the simulation.
 - If $\mathcal{S}$ receives **accept** from the adversary, $\mathcal{S}$ outputs 1 to the adversary on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}$, and sends **accept** to $\mathcal{F}_{\text{vrfy-triple-with-abort}}$.
 - If the check in $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ fails, i.e., there exists $\ell \in [N]$ such that $a_\ell * b_\ell \neq c_\ell$, $\mathcal{S}$ receives from $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ all parties' shares of $\{([a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1})\}_{\ell \in [N]}$. For each $\ell \in [N]$, in order to sample honest parties' shares of $[\theta_\ell]_{t+2k-2}$,

- (a) $\mathcal{S}$ computes $\mathbf{a}_\ell, \mathbf{b}_\ell, \mathbf{c}_\ell$ and $\boldsymbol{\theta}_\ell \leftarrow \mathbf{a}_\ell * \mathbf{b}_\ell - \mathbf{c}_\ell$.
 - (b) $\mathcal{S}$ samples a random degree- $(t + 2k - 2)$ packed Shamir sharing as $[\boldsymbol{\theta}_\ell]_{t+2k-2}$ according to the secret $\boldsymbol{\theta}_\ell$ and shares of corrupted parties.
- $\mathcal{S}$ reveals to the adversary honest parties' shares of $\{[\boldsymbol{\theta}_\ell]_{t+2k-2}\}_{\ell \in [N]}$ and outputs 0 to the adversary on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$.
7. $\mathcal{S}$ outputs what the adversary outputs and honest parties output what they receive from $\mathcal{F}_{\text{vrfy-triple-with-abort}}$.

We now show that the distribution of the output in the ideal world is identical to that in the real world using the following hybrid arguments.

Hyb₀: In this hybrid, we consider the execution in the real world.

Hyb₁: In this hybrid, instead of invoking $\mathcal{F}_{\text{packed-triple-with-abort}}$, $\mathcal{S}$ emulates the interaction between $\mathcal{F}_{\text{packed-triple-with-abort}}$ and the adversary itself by receiving the shares of corrupted parties and an instruction from the adversary. If $\mathcal{S}$ receives **abort**, $\mathcal{S}$ samples the whole sharings and sends them to the adversary. Otherwise, $\mathcal{S}$ sends corrupted parties' shares back to the adversary and delays the sampling of honest parties' shares. **Hyb₁** and **Hyb₀** have the same distributions.

Hyb₂: In this hybrid, instead of invoking $\mathcal{F}_{\text{rand-sharing}}(\Sigma_1)$, $\mathcal{S}$ emulates the interaction between $\mathcal{F}_{\text{rand-sharing}}$ and the adversary itself by receiving the shares of corrupted parties and sending them back to the adversary. $\mathcal{S}$ delays the sampling of honest parties' shares of $\{[\mathbf{o}_\ell]_{t+2k-2}\}_{\ell \in [N]}$. **Hyb₂** and **Hyb₁** have the same distributions.

Hyb₃: In this hybrid, we change the way of generating honest parties' shares of $[\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell]_{t+k-1}$ and $[\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{t+k-1}$. We first compute corrupted parties' shares of $[\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell]_{t+k-1}$ and $[\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{t+k-1}$, sample random degree- $(t + k - 1)$ packed Shamir sharings as $[\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell]_{t+k-1}$ and $[\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{t+k-1}$ based on shares of corrupted parties. Based on the secrets of $[\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell]_{t+k-1}, [\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{t+k-1}$, we compute $\tilde{\mathbf{a}}_\ell, \tilde{\mathbf{b}}_\ell$ as $\tilde{\mathbf{a}}_\ell \leftarrow (\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell) - \mathbf{a}_\ell, \tilde{\mathbf{b}}_\ell \leftarrow (\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell) - \mathbf{b}_\ell$ and $\tilde{\mathbf{c}}_\ell \leftarrow \tilde{\mathbf{a}}_\ell * \tilde{\mathbf{b}}_\ell$. Then based on corrupted parties' shares of $[\tilde{\mathbf{a}}_\ell]_{t+k-1}, [\tilde{\mathbf{b}}_\ell]_{t+k-1}, [\tilde{\mathbf{c}}_\ell]_{t+k-1}$ and the computed secrets, we can compute honest parties' shares of $[\tilde{\mathbf{a}}_\ell]_{t+k-1}, [\tilde{\mathbf{b}}_\ell]_{t+k-1}, [\tilde{\mathbf{c}}_\ell]_{t+k-1}$ and hence their shares of packed triples over bivariate polynomials. Due to the randomness of $[\tilde{\mathbf{a}}_\ell]_{t+k-1}, [\tilde{\mathbf{b}}_\ell]_{t+k-1}$, **Hyb₃** and **Hyb₂** have the same distributions.

Hyb₄: In this hybrid, instead of invoking $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$ to check the consistency of pairs of packed Shamir sharings $\{([\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell]_{t+k-1}, [\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{t+k-1})\}_{\ell \in [N]}, \{([\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{t+k-1}, [\mathbf{c}_\ell + \tilde{\mathbf{c}}_\ell]_{t+k-1})\}_{\ell \in [N]}$, $\mathcal{S}$ emulates the behavior of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$ as described in the Step 5 of the simulation. By functionality of $\mathcal{F}_{\text{vrfy-linear-with-abort}}$, **Hyb₄** and **Hyb₃** have the same distributions.

Hyb₅: In this hybrid, instead of invoking $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$ to check all parties' shares of $\{[\boldsymbol{\theta}_\ell]_{t+2k-2}\}_{\ell \in [N]}$, $\mathcal{S}$ emulates $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$ itself, which is feasible since $\mathcal{S}$ knows honest parties' shares of $\{[\tilde{\mathbf{a}}_\ell]_{t+k-1}, [\tilde{\mathbf{b}}_\ell]_{t+k-1}, [\tilde{\mathbf{c}}_\ell]_{t+k-1}, [\mathbf{c}_\ell]_{t+k-1}, [\mathbf{o}_\ell]_{t+2k-2}\}_{\ell \in [N]}$ and thus can compute honest parties' shares of $\{[\boldsymbol{\theta}_\ell]_{t+2k-2}\}_{\ell \in [N]}$. **Hyb₅** and **Hyb₄** have the same distributions.

Hyb₆: In this hybrid, we change the emulation of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$ as follows. $\mathcal{S}$ first learns the secrets $\{\mathbf{a}_\ell, \mathbf{b}_\ell, \mathbf{c}_\ell\}_{\ell \in [N]}$. For every $\ell \in [N]$, $\mathcal{S}$ checks whether $\mathbf{a}_\ell * \mathbf{b}_\ell = \mathbf{c}_\ell$.

- If this holds for every $\ell \in [N]$, $\mathcal{S}$ changes the way of computing honest parties' shares of $[\boldsymbol{\theta}_\ell]_{t+2k-2}$ by first computing corrupted parties' shares of $[\boldsymbol{\theta}_\ell]_{t+2k-2}$ and randomly sampling a degree- $(t + 2k - 2)$ packed Shamir sharing as $[\boldsymbol{\theta}_\ell]_{t+2k-2}$ based on shares of corrupted parties and the secret $\boldsymbol{\theta}_\ell = (\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell) * (\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell) - \tilde{\mathbf{a}}_\ell * (\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell) - (\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell) * \tilde{\mathbf{b}}_\ell + \tilde{\mathbf{c}}_\ell - \mathbf{c}_\ell = \mathbf{a}_\ell * \mathbf{b}_\ell - \mathbf{c}_\ell = \mathbf{0}$.
- If one of the checks fails, $\mathcal{S}$ changes the way of computing honest parties' shares of $[\boldsymbol{\theta}_\ell]_{t+2k-2}$ by first computing corrupted parties' shares of $[\boldsymbol{\theta}_\ell]_{t+2k-2}$ and randomly sampling a degree- $(t + 2k - 2)$ packed Shamir sharing as $[\boldsymbol{\theta}_\ell]_{t+2k-2}$ based on shares of corrupted parties and the secret $\boldsymbol{\theta}_\ell = \mathbf{a}_\ell * \mathbf{b}_\ell - \mathbf{c}_\ell$.

Then $\mathcal{S}$ computes honest parties' shares of $[\mathbf{a}_\ell * \mathbf{b}_\ell - \mathbf{c}_\ell]_{t+2k-2}$ as $[\mathbf{a}_\ell * \mathbf{b}_\ell - \mathbf{c}_\ell]_{t+2k-2} \leftarrow [\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell]_{k-1} \cdot [\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{k-1} - [\tilde{\mathbf{a}}_\ell]_{t+k-1} \cdot [\mathbf{b}_\ell + \tilde{\mathbf{b}}_\ell]_{k-1} - [\mathbf{a}_\ell + \tilde{\mathbf{a}}_\ell]_{k-1} \cdot [\tilde{\mathbf{b}}_\ell]_{t+k-1} + [\tilde{\mathbf{c}}_\ell]_{t+k-1} - [\mathbf{c}_\ell]_{t+k-1}$ and computes their shares of $[\mathbf{o}_\ell]_{t+2k-2}$ as $[\mathbf{o}_\ell]_{t+2k-2} \leftarrow [\boldsymbol{\theta}_\ell]_{t+2k-2} - [\mathbf{a}_\ell * \mathbf{b}_\ell - \mathbf{c}_\ell]_{t+2k-2}$. **Hyb₆** and **Hyb₅** have the same distributions due to the randomness of $[\mathbf{o}_\ell]_{t+2k-2}$ and the fact that $\tilde{\mathbf{a}}_\ell * \tilde{\mathbf{b}}_\ell = \tilde{\mathbf{c}}_\ell$ for each $\ell \in [N]$.

Hyb₇: In this hybrid, we do not compute honest parties' shares of $[\mathbf{o}_\ell]_{t+2k-2}, [\tilde{\mathbf{a}}_\ell]_{t+k-1}, [\tilde{\mathbf{b}}_\ell]_{t+k-1}$, and $[\tilde{\mathbf{c}}_\ell]_{t+k-1}$, which does not affect the output distribution as they are not used in **Hyb₆**. Furthermore, we let

$\mathcal{F}_{\text{vrfy-triple-with-abort}}$ take the role of $\mathcal{S}$ to check whether the secrets of each packed triple $([a_\ell]_{t+k-1}, [b_\ell]_{t+k-1}, [c_\ell]_{t+k-1})$ satisfy multiplication relations. $\mathcal{S}$ simulates honest parties' outputs as follows. If one of the following events happens, $\mathcal{S}$ sends **reject** to $\mathcal{F}_{\text{vrfy-triple-with-abort}}$.

- $\mathcal{S}$ outputs **abort** on behalf of $\mathcal{F}_{\text{packed-triple-with-abort}}$.
- $\mathcal{S}$ outputs 0 on behalf of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7)$.
- $\mathcal{S}$ receives **reject** from the adversary when emulating $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1)$.

Otherwise, $\mathcal{S}$ sends **accept** to $\mathcal{F}_{\text{vrfy-triple-with-abort}}$. In doing this, $\mathcal{S}$ does not use honest parties' inputs for simulation. Since $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ does the same as $\mathcal{S}$ in **Hyb**₆, **Hyb**₇ and **Hyb**₆ have the same distributions.

Since **Hyb**₇ corresponds to the ideal world, $\Pi_{\text{vrfy-triple-with-abort}}$ securely computes $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ in the $\mathcal{F}_{\text{packed-triple-with-abort}}$, $\mathcal{F}_{\text{rand-sharing}}$, $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ -hybrid model. $\square$

Proof (Proof of Theorem 7). Notice that the communication cost of $\Pi_{\text{vrfy-triple-with-abort}}$ is $4N \cdot n$ field elements, one invocation of $\mathcal{F}_{\text{packed-triple-with-abort}}(\lceil \frac{N}{k} \rceil, k)$ to prepare N degree- $(t+k-1)$ error-free packed Beaver triples, one invocation of $\mathcal{F}_{\text{rand-sharing}}(\Sigma_1, N)$ to prepare N random packed Shamir sharings in the form of $[0]_{t+2k-2}$, one invocation of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_7, 2N)$ to verify consistency of $2N$ linear secret sharings in the form of $([u]_{t+k-1}, [u]_{k-1})$, one invocation of $\mathcal{F}_{\text{vrfy-linear-with-abort}}(\Sigma_1, N)$ to verify consistency of N linear secret sharings in the form of $[0]_{t+2k-2}$. This amounts to $O(N \cdot n)$ elements plus one invocation of $\mathcal{F}_{\text{packed-triple-with-abort}}(\lceil \frac{N}{k} \rceil, k)$. By Lemma 4, plus instantiations of $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ and $\mathcal{F}_{\text{rand-sharing}}$ sketched in Section 3.5, the task of computing $\mathcal{F}_{\text{vrfy-triple-with-abort}}$ is reduced to computing $\mathcal{F}_{\text{packed-triple-with-abort}}$. Suppose $c' > 0$ is a constant such that the total concrete communication cost of $\Pi_{\text{vrfy-triple-with-abort}}(N, k)$ is no more than $c' \cdot N \cdot n$ elements.

Let π be any protocol computing $\mathcal{F}_{\text{packed-triple-with-abort}}(\lceil \frac{N}{k} \rceil, k)$ achieving perfect security against any adversary corrupting at most t parties. We set $c = \max\{32 \cdot c', 24\}$, where we need $c \geq 24$ to ensure the lower bound in Lemma 3 holds. For the sake of contradiction, suppose there exists $k \in \mathbb{N}$ with $k > c$ such that the communication of π is less than $\frac{N \cdot k^2 \cdot n}{32}$ field elements. By composing π , $\Pi_{\text{vrfy-triple-with-abort}}(N, k)$, and protocols instantiating $\mathcal{F}_{\text{vrfy-linear-with-abort}}$ and $\mathcal{F}_{\text{rand-sharing}}$, one can derive a protocol for computing $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ achieving perfect security in the plain model with communication of less than $\frac{N \cdot k^2 \cdot n}{32} + c' \cdot N \cdot n < \frac{N \cdot k \cdot n}{16}$ elements if $k > 32 \cdot c'$. This contradicts the lower bound established in Lemma 3, which requires all parties to communicate at least $\frac{N \cdot k \cdot n}{16}$ elements. Thus, we complete the proof of Theorem 7. $\square$

More generally, when considering the setting of sub-optimal corruption threshold, where at most $t = \frac{n-s}{3}$ corrupted parties are controlled by an adversary for some $s > 1$, we obtain a communication lower bound stated as Corollary 1. To show this, we follow the framework in [DLN19] and prove that to prepare N packed Beaver triples over bivariate polynomials, every group of s parties are expected to communicate at least $\Omega(N \cdot k^2)$ elements based on Lemma 3.

Corollary 1 (Sub-optimal corruption). *Let $n = 3t + s$, $k \in \mathbb{N}$ such that $1 \leq k \leq \frac{t+2}{2}$, and $N > n$. There exists a constant $c > 0$ such that for all $k > c$, any protocol computing $\mathcal{F}_{\text{packed-triple-with-abort}}(N, k)$ with perfect security against a fully malicious adversary controlling at most t corrupted parties communicates at least $\frac{N \cdot k^2 \cdot n/s}{32}$ field elements, when $\mathcal{F}_{\text{packed-triple-with-abort}}$ doesn't abort. This lower bound implies an amortized cost of $\Omega(n/s)$ elements per secret.*

Proof (Proof of Corollary 1). Before starting the proof, we notice in the proof of Lemma 3, the privacy against P_0 is not utilized to derive the lower bound. Specifically, we show that for all $N > \lceil \frac{24}{\log F} \rceil$ and $k > 24$, any protocol computing $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ with perfect security against any adversary controlling up to t parties in P_1, P_2, P_3 requires P_0 to communicate at least $\frac{N \cdot k}{8}$ elements. Our proof of this corollary will rely on this statement.

Suppose π is a $(3t + s)$ -party protocol computing $\mathcal{F}_{\text{packed-triple-with-abort}}(\lceil \frac{N}{k} \rceil, k)$ with perfect security against any adversary corrupting at most t parties. We begin by dividing n positions of π into n/s disjoint

groups of s positions, and consider an arbitrary group P of s positions. Recall $c' > 0$ is a constant such that the concrete communication cost in total of $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ is no more than $c' \cdot N \cdot n$ elements. We set $c = \max\{32 \cdot c', 24\}$. For the sake of contradiction, assume that there exists $k \in \mathbb{N}$ with $k > c$ such that the communication of π for s positions in P is less than $\frac{N \cdot k^2}{16} = \frac{N \cdot k}{16}$ elements.

Now we obtain a $(3t + 1)$ -party protocol π' computing $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ as follows.

- All $3t + 1$ parties invoke the $(3t + s)$ -party protocol π with P_0 emulating s positions in P to prepare $\lceil \frac{N}{k} \rceil$ degree- $(t + k - 1, t + k - 1)$ packed Beaver triples over bivariate polynomials. After that, P_0 obtains s parties' shares of packed triples and chooses one of them as its shares of packed triples.
- All $3t + 1$ parties then follow $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ to verify the correctness of packed triples.

On the one hand, if P_0 is honest, the resulting $(3t + 1)$ -party protocol π' computes $\mathcal{F}_{\text{vrfy-triple-with-abort}}(N, k)$ with perfect security against any adversary corrupting up to t parties in $\{P_1, P_2, P_3\}$. On the other hand, if we avoid P_0 to play the role of P_{king} in $\mathcal{F}_{\text{vrfy-triple-with-abort}}$, the communication of π' for P_0 is less than $\frac{N \cdot k}{16} + 2 \cdot c' \cdot N < \frac{N \cdot k}{8}$ elements if $k > 32 \cdot c'$. This contradicts the previously established lower bound, which requires P_0 to communicate at least $\frac{N \cdot k}{8}$ elements in any such protocol.

Therefore, every group of s positions of π should communicate at least $\frac{N \cdot k}{16}$ elements, leading to an overall communication of at least $\frac{N \cdot k \cdot n / s}{32}$ elements to instantiate $\mathcal{F}_{\text{packed-triple-with-abort}}(\lceil \frac{N}{k} \rceil, k)$. $\square$

5.4 Communication Complexity of Inner Product Functions

In this part, we extend the approach used to prove a communication lower bound for verifying packed Beaver triples to establish a lower bound for computing the inner product functions. With a notice that computing the inner product of two length- N vectors can be represented as a circuit of size $O(N)$, we prove that any protocol computing this with perfect security with abort communicates $\Omega(N \cdot n)$ elements.

Intuitively, the task of computing the inner product functions is similar to that of verifying whether triples satisfy the multiplication relations in spirit as the views of parties will contain their inputs and outputs, no matter the outputs are from computation or to be verified, in both cases. This analogue allows us to derive a contradiction by simultaneously appealing to three constraints: perfect privacy, perfect correctness, and the assumption that parties are limited to only have a small amount of communication, additionally exploring non-linearity of inner product functions. We formally conclude the communication lower bound for inner product functions in Theorem 8.

Theorem 8. *Let $n = 3t + 1$ and $P_0, P_{1,1}, \dots, P_{1,t}, P_{2,1}, \dots, P_{2,t}, P_{3,1}, \dots, P_{3,t}$ be n parties. The inner product function $f_{IP,N}$ is defined as follows. It first receives two vectors of N elements $\mathbf{x} \in \mathbb{F}^N$ and $\mathbf{y} \in \mathbb{F}^N$ from $P_{2,1}$ and $P_{3,1}$, respectively, then computes the inner product $\langle \mathbf{x}, \mathbf{y} \rangle$, and outputs z to $P_{2,1}$ and w to $P_{3,1}$, where z and w are sampled uniformly at random subject to $z + w = \langle \mathbf{x}, \mathbf{y} \rangle$.*

There exists a constant $c > 0$ such that for all $N > c$, any protocol computing $f_{IP,N}$ with perfect security with abort against a fully malicious adversary controlling at most t corrupted parties satisfies that each party (other than $P_{2,1}, P_{3,1}$) has expected communication of at least $\frac{N}{9}$ elements, leading to total communication of at least $\frac{N \cdot (n-2)}{18}$ elements, when the functionality $f_{IP,N}$ doesn't abort.

Proof (Proof of Theorem 8). Suppose π is an n -party protocol computing $f_{IP,N}$ with perfect security with abort against any fully malicious adversary controlling at most t corrupted parties. We use $\mathcal{D}$ to denote the set of all honest executions that output correctly, i.e., $\mathcal{D} := \{d = (\mathbf{x}, \mathbf{y}; r_0, r_1, r_2, r_3) \mid (z, w) \leftarrow \text{output}(\pi, d) \text{ s.t. } z + w = \langle \mathbf{x}, \mathbf{y} \rangle\}$. We assume each party has its internal randomness of finite size, which implies the set $\mathcal{D}$ has finite cardinality. We model $\mathcal{D}$ as the sample space, and assume a uniform distribution over $\mathcal{D}$. For any event $\mathcal{E} \subseteq \mathcal{D}$, we denote its probability by $\Pr_{d \leftarrow \mathcal{D}}[\mathcal{E}]$, or simply $\Pr_d[\mathcal{E}]$.

We consider communication complexity of $n - 2$ parties who don't receive outputs. Without loss of generality, we focus on the communication complexity of P_0 and denote by a random variable $C_0 : \mathcal{D} \rightarrow \mathbb{N}$

the length of its transcript. We set $c = \lceil \frac{24}{\log F} \rceil$. For the sake of contradiction, suppose there exists N with $N > c$ such that the communication of π for P_0 is such that

$$\mathbb{E}_{d \leftarrow \mathcal{D}}[C_0] < \frac{N}{9}.$$

Following the proof framework in Lemma 3, recall the proof is divided into three steps.

Step 1. Define three honest executions satisfying several delicate requirements.

Step 2. Construct a dishonest execution with an incorrect output by composing three honest executions chosen in last step, which leads to a contradiction with perfect correctness of π .

Step 3. Prove existence of such three honest executions based on perfect privacy of π and the assumption that P_0 only communicated a few elements in expectation.

Step 1. Defining three honest executions. We start with sampling an honest execution exec_0 by sampling $d = (\mathbf{x}^{(0)}, \mathbf{y}^{(0)}; r_0, r_1, r_2, r_3)$ from $\mathcal{D}$ uniformly at random. Suppose $(\mathbf{x}^{(0)}, \mathbf{y}^{(0)}, z^{(0)}, w^{(0)})$ is the tuple of inputs and outputs corresponding to d . Let $\mathcal{D}_1(d) \subset \mathcal{D}$ be the set of all tuples of the form $d' = (\mathbf{x}^{(1)}, \mathbf{y}^{(1)}; r'_0, r'_1, r'_2, r'_3)$ satisfying the conditions.

- $\text{view}^{P_2}(\pi, d) = \text{view}^{P_2}(\pi, d')$, which implies $\mathbf{x}^{(1)} = \mathbf{x}^{(0)}$, $r'_2 = r_2$, and $z^{(1)} = z^{(0)}$;
- $\text{transcript}^{P_0}(\pi, d) = \text{transcript}^{P_0}(\pi, d')$.

Then we define **event 1** to an event that the number of possible distinct inputs of $P_{3,1}$ among the executions in the set $\mathcal{D}_1(d)$ is strictly greater than $F^{\frac{N}{2}} + 1$, i.e., $|\{\bar{\mathbf{y}} \in \mathbb{F}^N \mid \exists d' \in \mathcal{D}_1(d), d' \vdash \bar{\mathbf{y}}\}| > F^{\frac{N}{2}} + 1$.

Similarly, let $\mathcal{D}_2(d) \subseteq \mathcal{D}$ be the set of all tuples of the form $d'' = (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}; r''_0, r''_1, r''_2, r''_3)$ satisfying the following conditions.

- $\text{view}^{P_3}(\pi, d) = \text{view}^{P_3}(\pi, d'')$, which implies $\mathbf{y}^{(2)} = \mathbf{y}^{(0)}$, $r''_3 = r_3$, and $w^{(2)} = w^{(0)}$;
- $\text{transcript}^{P_0}(\pi, d) = \text{transcript}^{P_0}(\pi, d'')$.

We define **event 2** to be an event that the number of possible distinct inputs of $P_{2,1}$ among the executions in the set $\mathcal{D}_2(d)$ is strictly greater than $F^{\frac{N}{2}} + 1$, i.e., $|\{\bar{\mathbf{x}} \in \mathbb{F}^N \mid \exists d'' \in \mathcal{D}_2(d), d'' \vdash \bar{\mathbf{x}}\}| > F^{\frac{N}{2}} + 1$. Latter, we write $\mathcal{D}_1$ and $\mathcal{D}_2$ short for $\mathcal{D}_1(d)$ and $\mathcal{D}_2(d)$, respectively.

Step 2. Constructing a dishonest execution. If **event 1** and **event 2** happen simultaneously, we claim this breaks down perfect correctness of π . In fact, we first pick $d' = (\mathbf{x}^{(1)}, \mathbf{y}^{(1)}; r'_0, r'_1, r'_2, r'_3)$ and $d'' = (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}; r''_0, r''_1, r''_2, r''_3)$ as tuples of inputs and internal randomness satisfying two requirements of $\mathcal{D}_1$ and $\mathcal{D}_2$, respectively. I.e., $d' \in \mathcal{D}_1, d'' \in \mathcal{D}_2$. Let $\text{exec}_1 := \text{exec}(\pi, d')$ and $\text{exec}_2 := \text{exec}(\pi, d'')$. Then we construct a dishonest execution denoted by $\text{exec}_3(d', d'')$ as follows.

- P_0 uses r_0 (or equivalently, r'_0, r''_0) as its internal randomness, and honestly follows the protocol.
- P_2 takes $\mathbf{x}^{(2)}$ as its input, uses r''_2 as its internal randomness, and honestly follows the protocol.
- P_3 takes $\mathbf{y}^{(1)}$ as its input, uses r'_3 as its internal randomness, and honestly follows the protocol.
- An adversary controlling parties $\{P_{1,i}\}_{i \in [t]}$ launches a similar attack as in Lemma 2 and Lemma 3. In particular, P_1 first emulates exec_0 , exec_1 and exec_2 to compute the messages it would send to P_0 , P_2 , P_3 . Then malicious P_1 behaves inconsistently across parties. When interacting with P_0 , P_1 uses the messages in exec_0 (or equivalently, exec_1 , exec_2); when interacting with P_2 , P_1 uses the messages in exec_3 ; when interacting with P_3 , P_1 uses the messages in exec_1 .

We show that if both **event 1** and **event 2** happen, there exist $d' \in \mathcal{D}_1, d'' \in \mathcal{D}_2$ such that $P_{2,1}, P_{3,1}$ will obtain an incorrect output in $\text{exec}_3(d', d'')$. Let $z^{(3)}$ and $w^{(3)}$ be the outputs of $P_{2,1}$ and $P_{3,1}$, respectively. On the one hand, to argue the output of $P_{2,1}$ and $P_{3,1}$ in $\text{exec}_3(d', d'')$, we rely on the following claim whose proof is analogous to its counterpart in Lemma 3, and is therefore omitted.

Claim. The transcripts between P_2 and P_1 in executions exec_3 and exec_2 are identical. The transcripts between P_3 and P_1 in executions exec_3 and exec_1 are identical. The transcripts between P_2 and P_3 and transcripts between P_0 and each party in $\{P_1, P_2, P_3\}$ are identical in four executions $\text{exec}_0, \text{exec}_1, \text{exec}_2$ and exec_3 .

According to this claim, when comparing exec_3 and exec_2 , P_2 has the same views in both executions and thus P_2 would output $z^{(2)}$ in both executions, i.e., $z^{(3)} = z^{(2)}$. Similarly, we have P_3 would output $w^{(1)}$ in both exec_3 and exec_1 , i.e., $w^{(3)} = w^{(1)}$.

On the other hand, we observe that there exist $d' \in \mathcal{D}_1$ and $d'' \in \mathcal{D}_2$ such that the inputs and outputs $(\mathbf{x}^{(3)}, \mathbf{y}^{(3)}, z^{(3)}, w^{(3)})$ in exec_3 does not satisfy $\langle \mathbf{x}^{(3)}, \mathbf{y}^{(3)} \rangle = z^{(3)} + w^{(3)}$. First, we have the following observation due to non-linearity of inner product functions.

Claim. The tuple of inputs and outputs in $\text{exec}_3(d', d'')$ is $(\mathbf{x}^{(3)}, \mathbf{y}^{(3)}, z^{(3)}, w^{(3)}) = (\mathbf{x}^{(2)}, \mathbf{y}^{(1)}, z^{(2)}, w^{(1)})$. The output in exec_3 are **incorrect** with respect to input if $\langle \mathbf{x}^{(2)} - \mathbf{x}^{(0)}, \mathbf{y}^{(1)} - \mathbf{y}^{(0)} \rangle \neq 0$.

Proof. Since $\text{exec}_0, \text{exec}_1$ and exec_2 have correct outputs with respect to their inputs, we have that $\langle \mathbf{x}^{(0)}, \mathbf{y}^{(0)} \rangle = z^{(0)} + w^{(0)}$, $\langle \mathbf{x}^{(0)}, \mathbf{y}^{(1)} \rangle = z^{(0)} + w^{(1)}$, and $\langle \mathbf{x}^{(2)}, \mathbf{y}^{(0)} \rangle = z^{(2)} + w^{(0)}$. Then we get

$$\begin{aligned} \langle \mathbf{x}^{(2)}, \mathbf{y}^{(1)} \rangle - (z^{(2)} + w^{(1)}) &= \langle \mathbf{x}^{(2)}, \mathbf{y}^{(1)} \rangle - (z^{(0)} + w^{(1)}) - (z^{(2)} + w^{(0)}) + (z^{(0)} + w^{(0)}) \\ &= \langle \mathbf{x}^{(2)}, \mathbf{y}^{(1)} \rangle - \langle \mathbf{x}^{(0)}, \mathbf{y}^{(1)} \rangle - \langle \mathbf{x}^{(2)}, \mathbf{y}^{(0)} \rangle + \langle \mathbf{x}^{(0)}, \mathbf{y}^{(0)} \rangle \\ &= \langle \mathbf{x}^{(2)} - \mathbf{x}^{(0)}, \mathbf{y}^{(1)} - \mathbf{y}^{(0)} \rangle, \end{aligned}$$

which implies if $\langle \mathbf{x}^{(2)} - \mathbf{x}^{(0)}, \mathbf{y}^{(1)} - \mathbf{y}^{(0)} \rangle \neq 0$, then the output in $\text{exec}_3(d', d'')$ is incorrect. $\square$

Let $V_1 = \{\mathbf{y}^{(1)} - \mathbf{y}^{(0)} \mid \exists d' \in \mathcal{D}_1 \text{ s.t. } d' \vdash \mathbf{y}^{(1)}\}$ and $V_2 = \{\mathbf{x}^{(2)} - \mathbf{x}^{(0)} \mid \exists d'' \in \mathcal{D}_2 \text{ s.t. } d'' \vdash \mathbf{x}^{(2)}\}$. Since both **event 1** and **event 2** happen, then we have $|V_1| > F^{\frac{N}{2}}$ and $|V_2| > F^{\frac{N}{2}}$. We claim there exist $\mathbf{v}_1 \in V_1, \mathbf{v}_2 \in V_2$ such that $\langle \mathbf{v}_1, \mathbf{v}_2 \rangle \neq 0$.

For the sake of contradiction, suppose for every $\mathbf{v}_1 \in V_1$ and $\mathbf{v}_2 \in V_2$, $\langle \mathbf{v}_1, \mathbf{v}_2 \rangle = 0$. First, we span V_1 and V_2 to linear subspaces of $\mathbb{F}^N$, i.e., $V'_1 = \text{span}(V_1)$, $V'_2 = \text{span}(V_2)$. It is clear that for any $\mathbf{v}_1 \in V'_1$ and $\mathbf{v}_2 \in V'_2$, we have $\langle \mathbf{v}_1, \mathbf{v}_2 \rangle = 0$ and thus $V'_1 \perp V'_2$, which implies that $\dim V'_1 + \dim V'_2 \leq N$. However, by observing that $|V'_1| \geq |V_1| > F^{\frac{N}{2}}, |V'_2| \geq |V_2| > F^{\frac{N}{2}}$, we have that $\dim V'_1 > \frac{N}{2}, \dim V'_2 > \frac{N}{2}$ and thus $\dim V'_1 + \dim V'_2 > N$, which leads to a contradiction.

Then according to the claim above, it is guaranteed that there exist $d' \in \mathcal{D}_1, d'' \in \mathcal{D}_2$ such that the outputs in $\text{exec}_3(d', d'')$ are incorrect with respect to its inputs, which contradicts with perfect correctness of π .

Step 3. Existence of three honest executions. Following Lemma 3, it suffices to prove $\Pr_{d \leftarrow \mathcal{D}}[\text{event 1}] < \frac{1}{2}$. The intuition for upper bounding $\Pr_{d \leftarrow \mathcal{D}}[\text{event 1}]$ is as follows.

- Given the view of P_2 view^{P_2} , we first show that for any input of $P_{3,1}$ $\bar{\mathbf{y}}$, there is an execution consistent with view^{P_2} and $\bar{\mathbf{y}}$, based on perfect privacy against corrupted P_2 . Traversing all $\bar{\mathbf{y}}$ leads a set of sufficiently many executions consistent with view^{P_2} . This is formally stated as the first claim below.
- Next, assuming P_0 only has a limited amount of communication in expectation, for some transcript of P_0 transcript^{P_0} , there are sufficiently many executions consistent with both view^{P_2} and transcript^{P_0} .
- Finally, by privacy against P_2 , we have that, given the view of P_2 view^{P_2} , the executions consistent with view^{P_2} are uniformly distributed over all possible values for the input of $P_{3,1}$ $\bar{\mathbf{y}}$. This is formally stated as the second claim below, whose proof is analogous to its counterpart in Lemma 3, and therefore is omitted.
- Consequently, there is a large number of executions consistent with both view^{P_2} and transcript^{P_0} covers a large number of distinct possible values for $\bar{\mathbf{y}}$, which in turn implies **event 1** happens.

Claim. Let view^{P_2} be the view of P_2 generated by an execution with correct outputs, i.e., there exists a tuple $d = (\mathbf{x}, \mathbf{y}; r_0, r_1, r_2, r_3) \in \mathcal{D}$ such that $\text{view}^{P_2}(\pi, d) = \text{view}^{P_2}$. Then for every $\bar{\mathbf{y}} \in \mathbb{F}^N$, there exists a tuple $\bar{d} = (\mathbf{x}, \bar{\mathbf{y}}; \bar{r}_0, \bar{r}_1, r_2, \bar{r}_3) \in \mathcal{D}$ of all parties' inputs and internal randomness such that $\text{view}^{P_2}(\pi, \bar{d}) = \text{view}^{P_2}$.

Let $\mathcal{D}_{\text{view}^{P_2}} \subset \mathcal{D}$ be the set of all honest executions satisfying the view of P_2 is view^{P_2} . Then the number of possible distinct inputs of P_2 among the executions in the set $\mathcal{D}_{\text{view}^{P_2}}$ is F^N , i.e.,

$$|\{\bar{\mathbf{y}} \in \mathbb{F}^N \mid \exists \bar{d} \in \mathcal{D}_{\text{view}^{P_2}} \text{ s.t. } \bar{d} \vdash \bar{\mathbf{y}}\}| = F^N.$$

Proof. By perfect privacy against a (semi-honest) corrupted P_2 , there exists a simulator $\mathcal{S}_2$ which takes as input the input and output of P_2 , and outputs the view of P_2 such that for any input $\bar{\mathbf{y}}$ of $P_{3,1}$, it holds that

$$\mathcal{S}_2(\mathbf{x}, f_{\text{IP},N}^2(\mathbf{x}, \bar{\mathbf{y}})) \equiv \text{view}^{P_2}(\pi, \mathbf{x}, \bar{\mathbf{y}}),$$

where $f_{\text{IP},N}^2(\mathbf{x}, \mathbf{y})$ denotes the output distribution of $P_{2,1}$, which is a uniform distribution over $\mathbb{F}$ regardless of values of $(\mathbf{x}, \mathbf{y})$. Hence, for every $\bar{\mathbf{y}}$, by property of $f_{\text{IP},N}^2$, we have

$$\begin{aligned} \text{view}^{P_2}(\pi, \mathbf{x}, \bar{\mathbf{y}}) &\equiv \mathcal{S}_2(\mathbf{x}, f_{\text{IP},N}^2(\mathbf{x}, \bar{\mathbf{y}})) \\ &\equiv \mathcal{S}_2(\mathbf{x}, f_{\text{IP},N}^2(\mathbf{x}, \mathbf{y})) \\ &\equiv \text{view}^{P_2}(\pi, \mathbf{x}, \mathbf{y}). \end{aligned}$$

Then there exists a choice of a tuple of internal randomness from P_0, P_1 and P_3 , $(\bar{r}_0, \bar{r}_1, \bar{r}_3)$, such that $\text{view}^{P_2}(\mathbf{x}, \bar{\mathbf{y}}; \bar{r}_0, \bar{r}_1, \bar{r}_3) = \text{view}^{P_2}(\pi, d)$.

By the reasoning above, for any $\bar{\mathbf{y}} \in \mathbb{F}^N$, $\bar{\mathbf{y}}$ can lead to a satisfied execution in $\mathcal{D}_{\text{view}^{P_2}}$. Since there are F^N possibilities of $\bar{\mathbf{y}}$, we conclude the number of possible distinct inputs of $P_{3,1}$ among the executions in $\mathcal{D}_{\text{view}^{P_2}}$ is F^N . $\square$

Claim. Given a view of P_2 view^{P_2} , we define the set $\mathcal{D}_{\text{view}^{P_2}}$ as in the previous claim. For any two vectors $\bar{\mathbf{y}}_1, \bar{\mathbf{y}}_2 \in \mathbb{F}^N$, the number of executions in $\mathcal{D}_{\text{view}^{P_2}}$ with $P_{3,1}$'s input being $\bar{\mathbf{y}}_1$ is equal to the number of executions with $P_{3,1}$'s input being $\bar{\mathbf{y}}_2$, i.e.,

$$|\{d \in \mathcal{D}_{\text{view}^{P_2}} \mid d \vdash \bar{\mathbf{y}}_1\}| = |\{d \in \mathcal{D}_{\text{view}^{P_2}} \mid d \vdash \bar{\mathbf{y}}_2\}|.$$

The computation of $\Pr_{d \leftarrow \mathcal{D}}[\text{event } 1]$ is also similar to that in Lemma 3. We present it here for completeness. Let E_1 be the event $(C_0(d) < 3 \cdot \mathbb{E}[C_0])$. For simplicity of notation, we abuse the notation E_1 to also denote the set of all executions in which $C_0(d) < 3 \cdot \mathbb{E}[C_0]$, i.e., $E_1 = \{d \in \mathcal{D} \mid C_0(d) < 3 \cdot \mathbb{E}[C_0]\}$.

$$\begin{aligned} &\Pr_{d \leftarrow \mathcal{D}}[\text{event } 1] \\ &= \sum_{\text{view}^{P_2}} \Pr_{d \leftarrow \mathcal{D}}[\text{event } 1 \wedge d \vdash \text{view}^{P_2}] \\ &= \sum_{\text{view}^{P_2}} \Pr_{d \leftarrow \mathcal{D}}[\text{event } 1 \wedge E_1 \wedge d \vdash \text{view}^{P_2}] + \Pr_{d \leftarrow \mathcal{D}}[\text{event } 1 \wedge \bar{E}_1 \wedge d \vdash \text{view}^{P_2}] \\ &= \sum_{\text{view}^{P_2}} \Pr_{d \leftarrow \mathcal{D}}[\text{event } 1 \mid E_1 \wedge d \vdash \text{view}^{P_2}] \cdot \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}] \\ &\quad + \Pr_{d \leftarrow \mathcal{D}}[\text{event } 1 \mid \bar{E}_1 \wedge d \vdash \text{view}^{P_2}] \cdot \Pr_{d \leftarrow \mathcal{D}}[\bar{E}_1 \wedge d \vdash \text{view}^{P_2}] \\ &= \sum_{\text{view}^{P_2}} \Pr_{d \leftarrow \mathcal{D}}[\text{event } 1 \mid E_1 \wedge d \vdash \text{view}^{P_2}] \cdot \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}] \\ &\quad + \Pr_{d \leftarrow \mathcal{D}}[\text{event } 1 \mid \bar{E}_1 \wedge d \vdash \text{view}^{P_2}] \cdot (\Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}] - \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}]) \end{aligned}$$

$$\begin{aligned}
&< \sum_{\text{view}^{P_2}} \frac{1}{6} \cdot \frac{1}{\Pr_{d \leftarrow \mathcal{D}}[E_1 \mid d \vdash \text{view}^{P_2}]} \cdot \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}] \\
&\quad + 1 \cdot (\Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}] - \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}]) \\
&= \sum_{\text{view}^{P_2}} \frac{1}{6} \cdot \Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}] + \Pr_{d \leftarrow \mathcal{D}}[d \vdash \text{view}^{P_2}] - \Pr_{d \leftarrow \mathcal{D}}[E_1 \wedge d \vdash \text{view}^{P_2}] \\
&= 1 + \frac{1}{6} - \Pr_{d \leftarrow \mathcal{D}}[E_1] \\
&\leq 1 + \frac{1}{6} - (1 - \frac{1}{3}) \\
&= \frac{1}{2}.
\end{aligned}$$

The deduction above takes advantage of the following two inequalities:

1. $\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid E_1 \wedge d \vdash \text{view}^{P_2}] < \frac{1}{6} \cdot \frac{1}{\Pr_{d \leftarrow \mathcal{D}}[E_1 \mid d \vdash \text{view}^{P_2}]}$. We prove this as below.

To compute the probability, we first introduce some notations. For simplicity, we write $\mathbf{t}^{P_0}$ short for **transcript** ^{P_0} . Define $\mathcal{D}_{\text{view}^{P_2}} \subset \mathcal{D}$ to be the set of all executions in which the view of P_2 is view^{P_2} , i.e., $\mathcal{D}_{\text{view}^{P_2}} = \{d \in \mathcal{D} \mid d \vdash \text{view}^{P_2}\}$. Define $\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}} \subset \mathcal{D}$ to be the set of all executions in which

- the view of P_2 is view^{P_2} ,
- the transcript of P_0 is $\mathbf{t}^{P_0}$,
- the possibilities of distinct values of the input of $P_{3,1}$ is no more than $F^{\frac{N}{2}}$.

Notice that

$$\begin{aligned}
&\Pr_{d \leftarrow \mathcal{D}}[\overline{\text{event 1}} \mid E_1 \wedge d \vdash \text{view}^{P_2}] \\
&= \frac{\sum_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}}|} \tag{8}
\end{aligned}$$

$$\leq \frac{|\{\mathbf{t}^{P_0}(\pi, d) \mid d \in \mathcal{D}_{\text{view}^{P_2}}\}| \cdot \max_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}}|} \tag{9}$$

$$\leq \frac{F^{3 \cdot \mathbb{E}[C_0]} \cdot \max_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}}|} \tag{10}$$

$$\begin{aligned}
&= \frac{F^{3 \cdot \mathbb{E}[C_0]} \cdot \max_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}|}{|\mathcal{D}_{\text{view}^{P_2}}|} \cdot \frac{|\mathcal{D}_{\text{view}^{P_2}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}}|} \\
&\leq \frac{F^{3 \cdot \mathbb{E}[C_0]} \cdot (F^{\frac{N}{2}} + 1)}{F^N} \cdot \frac{|\mathcal{D}_{\text{view}^{P_2}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}, b}|} \tag{11}
\end{aligned}$$

$$< \frac{1}{6} \cdot \frac{|\mathcal{D}_{\text{view}^{P_2}}|}{|E_1 \cap \mathcal{D}_{\text{view}^{P_2}}|} \tag{12}$$

$$= \frac{1}{6} \cdot \frac{1}{\Pr_{d \leftarrow \mathcal{D}}[E_1 \mid d \vdash \text{view}^{P_2}]},$$

where the step-by-step reasoning is given as follows. To show Equality 8, we group all executions $E_1 \cap \mathcal{D}_{\text{view}^{P_2}}$ according to P_0 's transcript. Then **event 1** doesn't happen if and only if the sampled execution d lies in the set $\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}$ for some $\mathbf{t}^{P_0}$.

Inequality 9 holds because $\sum_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}| \leq \sum_{\mathbf{t}^{P_0}} \max_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}| = |\{\mathbf{t}^{P_0}(\pi, d) \mid d \in \mathcal{D}_{\text{view}^{P_2}}\}| \cdot \max_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}|$.

As for Inequality 10, because for any execution in E_1 , the length of P_0 's transcript is at most $3 \cdot \mathbb{E}[C_0]$, resulting in at most $F^{3 \cdot \mathbb{E}[C_0]}$ possibilities.

Regarding Inequality 11, by two claims above and the definition of $\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}$, we have

$$\frac{\max_{\mathbf{t}^{P_0}} |\text{Bad}_{\text{view}^{P_2}, \mathbf{t}^{P_0}}|}{|\mathcal{D}_{\text{view}^{P_2}}|} \leq \frac{(F^{\frac{N}{2}} + 1) \cdot |\{d \in \mathcal{D}_{\text{view}^{P_2}} \mid d \vdash \bar{\mathbf{y}}\}|}{F^N \cdot |\{d \in \mathcal{D}_{\text{view}^{P_2}} \mid d \vdash \bar{\mathbf{y}}\}|} = \frac{F^{\frac{N}{2}} + 1}{F^N},$$

for any $\bar{\mathbf{y}} \in \mathbb{F}^N$ representing the input of $P_{3,1}$.
 Finally, regarding Inequality 12, by $N > \lceil \frac{24}{\log F} \rceil$ and the assumption that $\mathbb{E}_d[C_0] < \frac{N}{9}$, we have that

$$\frac{F^{3 \cdot \mathbb{E}[C_0]} \cdot (F^{\frac{N}{2}} + 1)}{F^N} < \frac{2 \cdot F^{\frac{N}{3}}}{F^{\frac{N}{2}}} = \frac{2}{F^{\frac{N}{6}}} < \frac{1}{6}.$$

2. $\Pr_{d \leftarrow \mathcal{D}}[E_1] \geq 1 - \frac{1}{3}$. This can be proved in the same way as in Lemma 3.

By previous analysis, we prove the statement. $\square$

Notice in the proof of Theorem 8, the privacy against P_0 is not utilized to derive the lower bound. Specifically, we show that for all $N > \lceil \frac{24}{\log F} \rceil$, any protocol computing $f_{IP,N}$ with perfect security with abort against any adversary controlling up to t parties in P_1, P_2, P_3 requires P_0 to communicate at least $\frac{N}{9}$ elements. Based on this, we generalize Theorem 8 to the sub-optimal corruption setting and to the preprocessing setting as follows.

Following a similar reasoning as Corollary 1, we get a communication lower bound for inner product functions when corruption threshold is $\frac{n-s}{3}$, for some $s > 1$. In particular, we show any protocol that computes $f_{IP,N}$ with perfect security with abort communicates at least $\Omega(N \cdot n/s)$ elements in expectation, which is formally stated as Corollary 2.

Corollary 2 (Sub-optimal corruption). *Let $n = 3t + s$. There exists a constant $c > 0$ such that for all $N > c$, any protocol computing the inner product of two length- N vectors $f_{IP,N}$ with perfect security with abort against a fully malicious adversary controlling at most t corrupted parties communicates at least $\frac{N \cdot (n-2)}{18 \cdot s}$ elements in expectation, when the functionality $f_{IP,N}$ doesn't abort.*

Proof (Proof of Corollary 2). Suppose π is a $(3t + s)$ -party protocol computing $f_{IP,N}$ with perfect security against with abort any adversary corrupting at most t parties. We begin by dividing the $n - 2$ positions in π who don't have inputs into $(n - 2)/s$ disjoint groups of s parties and consider an arbitrary group P of s parties. We set $c = \lceil \frac{24}{\log F} \rceil$. For the sake of contradiction, assume that there exists N with $N > c$ such that the communication of π for parties in P is less than $\frac{N}{9}$ elements.

Now letting P_0 emulate s positions in P , this would yield an $(3t + 1)$ -party protocol π' for $f_{IP,N}$. On the one hand, if P_0 is honest, π' computes $f_{IP,N}$ with perfect security with abort against any adversary corrupting up to t parties in $\{P_1, P_2, P_3\}$. On the other hand, the communication of π' for P_0 is less than $\frac{N}{9}$ elements. This contradicts the previously established lower bound, which requires P_0 to communicate at least $\frac{N}{9}$ elements in any such protocol. Therefore, every group of s parties should communicate at least $\frac{N}{9}$ elements, leading to an overall communication of at least $\frac{N \cdot (n-2)}{18 \cdot s}$ elements to securely compute $f_{IP,N}$. $\square$

We further extend the lower bound to the preprocessing setting. In particular, we show that any protocol that computes $f_{IP,N}$ with perfect security with abort in the preprocessing model either communicates $\Omega(N \cdot n)$ elements, or requires preprocessing data of size at least $\Omega(N)$, which is formally stated as Corollary 3.

Corollary 3 (Preprocessing data size). *Let $n = 3t + 1$. There exists a constant $c > 0$ such that for all $N > c$, any protocol computing $f_{IP,N}$ with perfect security with abort in the preprocessing model against at most t corrupted parties either communicates at least $\frac{N \cdot (n-2)}{36}$ elements in expectation, or requires preprocessing data of size at least $\frac{N}{18}$, when the functionality $f_{IP,N}$ doesn't abort.*

Proof (Proof of Corollary 3). Suppose there is a protocol π computing $f_{IP,N}$ in the preprocessing model with perfect security with abort against any fully malicious adversary controlling at most t corrupted parties. We set $c = \lceil \frac{24}{\log F} \rceil$. For the sake of contradiction, assume that there exists N with $N > c$ such that 1) a party who doesn't have input, say P_0 , communicates less than $\frac{N}{18}$ elements in expectation in π and 2) the size of preprocessing data in π is less than $\frac{N}{18}$.

Now we derive a protocol π' in the plain model by letting P_0 first distribute the preprocessing data to all parties and then letting all parties run the protocol π to compute $f_{IP,N}$. On the one hand, if P_0 is an

honest party, then π' is secure against any adversary corrupting up to t parties in P_1, P_2, P_3 . On the other hand, by assumption, the expected communication cost for P_0 in π' is less than $\frac{N}{18} + \frac{N}{18} = \frac{N}{9}$ elements. This contradicts the previously established lower bound, which requires P_0 to communicate at least $\frac{N}{9}$ elements in any such protocol. Hence, either the communication of π is at least $\frac{N \cdot (n-2)}{36}$ elements, or the preprocessing data of π is of size at least $\frac{N}{18}$, completing the proof of the corollary. $\square$

5.5 Preparing Packed Beaver Triples

Recall we build a perfectly secure MPC protocol for SIMD circuits with $O(C)$ communication per input in the preprocessing model. The preprocessing data is of size $O(C)$ and contains $O(C/k^2)$ packed Beaver triples over bivariate polynomials. However, we prove preparing $O(C/k^2)$ packed Beaver triples even with perfect security with abort requires communicating $\Omega(C/k^2 \cdot k^2 \cdot n) = \Omega(C \cdot n)$ elements, amounting to $\Omega(n)$ elements per triple. To show the tightness of this lower bound, we present a protocol to prepare N packed Beaver triples over bivariate polynomials with $O(N \cdot n^3)$ elements, amounting to $O(n)$ elements per secret if $k = \Omega(n)$ and therefore matching the lower bound.

To prepare a degree- $(t + k - 1, t + k - 1)$ packed Beaver triple $([\mathbf{A}]_{t+k-1}, [\mathbf{B}]_{t+k-1}, [\mathbf{C}]_{t+k-1})$ over bivariate polynomials, notice that the sharing process of secret sharing over bivariate polynomials is a linear circuit taking input as the secrets to be shared and the randomness. Based on this observation, all parties first prepare k^2 degree- t Beaver triples and random degree- t Shamir sharings, whose secrets will play the role of $(\mathbf{A}, \mathbf{B}, \mathbf{C})$ and the randomness of the sharing process, respectively, and fully determine the whole bivariate polynomials. Then they can compute degree- t sharings of each evaluation point of the bivariate polynomials and send to each party P_i their shares of the points that are supposed to be held by P_i , who then can reconstruct the secrets and compute its shares of $([\mathbf{A}]_{t+k-1}, [\mathbf{B}]_{t+k-1}, [\mathbf{C}]_{t+k-1})$.

We present the protocol in $\Pi_{\text{packed-triple}}(N, k)$. The security of $\Pi_{\text{packed-triple}}(N, k)$ follows directly from the error correcting property of degree- t Shamir sharings and is therefore omitted. We conclude its security property as Lemma 5.

The communication cost to prepare N degree- $(t + k - 1, t + k - 1)$ packed Beaver triple over bivariate polynomials is $3N \cdot n^2 \cdot (t + k)$ elements plus one invocation of $\mathcal{F}_{\text{triple}}(N \cdot k^2)$ and one invocations of $\mathcal{F}_{\text{rand-sharing}}(\Sigma_4, 3N \cdot ((t + k)^2 - k^2))$, amounting to $\text{P2P}(O(N \cdot n^3)) + O(1) \times \text{BA}(1) + O(1) \times \text{BC}(O(\log N + \log n + \log |\mathbb{F}|))$, and therefore $O(n)$ elements per secret.

Lemma 5. *Let $t = (n - 1)/3$ and $k \leq t + 1$. $\Pi_{\text{packed-triple}}(N, k)$ computes $\mathcal{F}_{\text{packed-triple}}(N, k)$ with perfect security in the $\mathcal{F}_{\text{rand-sharing}}, \mathcal{F}_{\text{triple}}$ -hybrid model against a fully malicious adversary who controls at most t corrupted parties.*

Protocol 9: $\Pi_{\text{packed-triple}}(N, k)$ in $\mathcal{F}_{\text{rand-sharing}}, \mathcal{F}_{\text{triple}}$ -hybrid model

For every $\ell \in [N]$, all parties prepare a packed Beaver triple over bivariate polynomials $([\mathbf{A}_\ell]_{t+k-1}, [\mathbf{B}_\ell]_{t+k-1}, [\mathbf{C}_\ell]_{t+k-1})$ as follows:

- 1: **Preparing triples.** All parties invoke $\mathcal{F}_{\text{triple}}(k^2)$ to obtain k^2 degree- t Beaver triples $\{[a_{ij}]_t, [b_{ij}]_t, [c_{ij}]_t\}_{i,j \in [k]}$. They set three matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{k \times k}$ of secrets as $\mathbf{A} \leftarrow (a_{ij})_{ij}$, $\mathbf{B} \leftarrow (b_{ij})_{ij}$, and $\mathbf{C} \leftarrow (c_{ij})_{ij}$.
 - In case of failure, output a new disputed pair E .
- 2: **Preparing random sharings.** All parties invoke $\mathcal{F}_{\text{rand-sharing}}(\Sigma_4, 3N \cdot ((t + k)^2 - k^2))$ to obtain $3N \cdot ((t + k)^2 - k^2)$ random degree- t Shamir sharings.
 - In case of failure, output a new disputed pair E .
- 3: **Computing bivariate polynomials.** For each $\mathbf{U} = (u_{ij})_{ij} \in \{\mathbf{A}_\ell, \mathbf{B}_\ell, \mathbf{C}_\ell\}_{\ell \in [N]}$, all parties compute a degree- $(t + k - 1, t + k - 1)$ packed Shamir sharing $[\mathbf{U}]_{t+k-1}$ as follows.
 1. All parties pick $(t + k)^2 - k^2$ unused random degree- t Shamir sharings $\{[r_{ij}]_t\}_{(i,j) \in [t+k] \times [t+k], (i,j) \notin [k] \times [k]}$.

2. All parties set a degree- $(t + k - 1, t + k - 1)$ bivariate $S(s, y)$ such that

$$\forall (i, j) \in [k] \times [k], [S(\beta_i, \beta_j)]_t = [u_{ij}]_t, \forall (i, j) \in [t + k] \times [t + k] \setminus [k] \times [k], [S(\alpha_i, \alpha_j)]_t = [r_{ij}]_t.$$

3. For each $i \in [n]$, all parties compute the shares of P_i as follows.
 - (a) For every $j \in [t + k]$, all parties compute their shares of $[S(\alpha_j, \alpha_i)]_t$ and $[S(\alpha_i, \alpha_j)]_t$. They then send these shares to P_i .
 - (b) On receiving messages from all parties, each P_i reconstructs the shares to learn $\{S(\alpha_j, \alpha_i)\}_{j \in [t + k]}$ and $\{S(\alpha_i, \alpha_j)\}_{j \in [t + k]}$, computes two polynomials $f_i(x), g_i(y)$ as its shares of $[[U]]_{t + k - 1}$, where $f_i(x) := S(x, \alpha_i), g_i(y) := S(\alpha_i, y)$.

6 Preparing Packed Beaver Triples in the 3-Party Beaver Triple-Hybrid Model

Recall we construct a perfectly secure MPC protocol with $O(C)$ communication assuming preprocessing data of packed Beaver triples over bivariate polynomials of size $O(C)$. However, we establish a communication lower bound that preparing such kind of packed Beaver triples requires $\Omega(n)$ communication per secret, failing to achieve overall $O(C)$ communication in the plain model. To account for this, we present a protocol preparing such kind of packed Beaver triples with communication of $O(n)$ elements per secret. In this section, we follow the idea in [SY25a] which focuses on honest majority setting and reduces preparing packed Beaver triples of size $O(C)$ to $O(C)$ pairwise random oblivious linear evaluation (OLE) correlations, to reduce packed Beaver triples over bivariate polynomials of size $O(C)$ to $O(C)$ 3-party Beaver triples over bivariate polynomials.

Concretely, following [GLS24, SY25a], we make use of packed triple extraction, an extension of triple extraction proposed in [CP17], to prepare packed Beaver triples. Specifically, each party first distributes packed Beaver triples to all parties. Then all parties use these initial $N = 3n$ packed Beaver triples with $n - 1 \sim \frac{N}{3}$ of them are known to the adversary to establish three polynomials $\mathbf{F}, \mathbf{G}$ and $\mathbf{H} := \mathbf{F} * \mathbf{G}$, where $\mathbf{F}, \mathbf{G}$ are of degree $\frac{N-1}{2}$ and $\mathbf{H}$ is of degree $N - 1$. So far all parties hold their shares of $\frac{N+1}{2}$ packed triples. Considering the correctness, among the initial N packed triples, a number of $2n + 1 \sim \frac{2N}{3}$ packed triples distributed by honest parties are guaranteed to satisfy the multiplication relations. To ensure the correctness of the extracted packed triples, we need honest parties to additionally check a number of $n - 1 \sim \frac{N}{3}$ packed triples, which would imply N packed triples distributed or checked by honest parties are correct, and therefore three polynomials $\mathbf{F}, \mathbf{G}$ and $\mathbf{H}$ are correctly established. Regarding the privacy, we count the number of extracted random packed triples. Since $n - 1 \sim \frac{N}{3}$ packed triples are checked by honest parties, $\frac{n-1}{2} \sim \frac{N}{6}$ packed triples are checked by corrupted parties and thus have been revealed to the adversary, leading to a number of $\frac{n-1}{2} + n - 1 = \frac{3n-3}{2} \sim \frac{N}{2}$ packed triples known to the adversary. Hence, only a number of $\frac{3n+1}{2} - \frac{3n-3}{2} = 2 = O(1)$ random packed triples can be extracted, leading to $O(n)$ elements per secret and failing to achieve $O(1)$ elements per secret.

To resolve this issue, we follow [SY25a] to use the idea of party virtualization, which is first proposed in [Bra87]. Based on this idea, [SY25a] manages to boost the corruption threshold outside the virtual parties to satisfy a strong honest majority which allows packed triple extraction to extract a constant fraction of random packed triples on the one hand, and emulates each virtual party who is expected to distribute packed triples to all parties using pairwise random OLE correlations on the other hand. Adapting to our setting of perfect security, we group each tuple of three parties into a virtual party and rely on three-party degree- $(1, 1)$ Beaver triples over bivariate polynomials. Such Beaver triples are shared among three parties via degree- $(1, 1)$ bivariate polynomials, enabling them to emulate a virtual party that distributes packed triples.

In doing this, for a virtual party containing at least two honest parties, the secrets of the Beaver triples are fully determined by shares of honest parties. Relying on this redundancy, checking the consistency of three sharings distributed by three parties inside this virtual party ensures packed triples distributed by this virtual party satisfy the multiplication relations with secrets unknown to the adversary. Consequently, if consistency checks pass, a $1 - (\frac{1}{3})^3 - 3 \cdot (\frac{1}{3})^2 \cdot \frac{2}{3} = \frac{20}{27}$ fraction of original packed triples correctly satisfies the

multiplication relations and is unknown to the adversary. Considering the correctness, to make sure three polynomials F, G and H are correctly computed, all parties perform the procedure of triple verification where honest parties check a $1 - \frac{20}{27} = \frac{7}{27}$ fraction of packed triples. Regarding the privacy, since a $\frac{7}{27}$ fraction of packed triples is checked by honest parties, a $\frac{7}{27} \cdot \frac{1}{2} = \frac{3.5}{27}$ fraction is also checked by corrupted parties and loses their privacy. Eventually, all parties manage to extract a $\frac{1}{2} - \frac{7}{27} - \frac{3.5}{27} = \frac{1}{9}$ fraction of random packed Beaver triples, leading to $O(1)$ elements plus $O(1)$ three-party Beaver triples per secret.

Moreover, to achieve security with identifiable abort, all parties are expected to identify a disputed pair when checks fail. When the consistency check fails and a virtual party distributing inconsistent sharings is located, three parties in this virtual party open their degree-(1, 1) sharings, and a disputed pair or a corrupted party can be identified by relying on the cross checking property of secret sharings via bivariate polynomials. When the triple verification fails and a virtual party distributing a packed triple not satisfying the multiplication relation is located, by previous analysis, this virtual party can only contain at most one honest party. Hence, any two parties of the three could be viewed as a disputed pair.

We first introduce our notations regarding three-party Beaver triples below. Then We describe the functionality preparing three-party degree-(1, 1) Beaver triples as $\mathcal{F}_{3\text{-triple}}$. We show the protocol for preparing packed Beaver triples over bivariate polynomials assuming three-party Beaver triples in $\Pi_{\text{packed-triple}}$ under the party elimination framework and conclude its security property in Lemma 6.

The communication cost for $\Pi_{\text{packed-triple}}$ to prepare a number of $\frac{(n')^3}{9}$ degree- $(t+k-1, t+k-1)$ packed Beaver triples is $\text{P2P}(59.3(n')^5 + 3(n')^4 \cdot k + 2.6(n')^3) + O(1) \times \text{BC}(O(\log n' + \log |\mathbb{F}|))$, $k^2 \cdot (n')^3$ invocations of $\mathcal{F}_{3\text{-triple}}$, invocations of $\mathcal{F}_{\text{rand-sharing}}(\Sigma_2, \frac{(n')^3-1}{2})$, $\mathcal{F}_{\text{rand-sharing}}(\Sigma_5, \frac{(n')^3-1}{2})$, $\mathcal{F}_{\text{rand-sharing}}(\Sigma_6, 3(n')^3)$, and $\mathcal{F}_{\text{batched-vrfy}}(\Sigma_8, \frac{3(n')^3-1}{2})$. This amounts to $\text{P2P}(O(n'^5)) + O(1) \times \text{BA}(1) + O(1) \times \text{BC}(O(\log n' + \log |\mathbb{F}|))$ plus $k^2 \cdot (n')^3$ invocations of $\mathcal{F}_{3\text{-triple}}$, and thus $O(1)$ elements plus $O(1)$ three-party Beaver triples per secret.

Notations. We will use $\llbracket s \rrbracket_1$ to denote a Shamir sharing of $x \in \mathbb{F}$ shared via a degree-(1, 1) bivariate polynomial $S(x, y)$ satisfying $S(0, 0) = s$ among three parties P_{ℓ_1}, P_{ℓ_2} and P_{ℓ_3} , where each party P_{ℓ_v} takes two degree-1 polynomials $f_v^S(x) := S(x, v)$, $g_v^S(y) := S(v, y)$ as its share of $\llbracket s \rrbracket_1$. Since degree-(1, 1) Shamir sharing is only used in the three-party setting (and not in the n -party setting), this notation is unambiguous.

Let $\mathbf{U} \in \mathbb{F}[X]^{k \times k}$ be a $k \times k$ matrix of degree-1 univariate polynomials. For $v \in \{1, 2, 3\}$, let $\mathbf{U}(v) \in \mathbb{F}^{k \times k}$ denote the matrix obtained by evaluating each polynomial in $\mathbf{U}$ at $x = v$.

Functionality 10: $\mathcal{F}_{3\text{-triple}}$

$\mathcal{F}_{3\text{-triple}}$ receives three parties' identities (ℓ_1, ℓ_2, ℓ_3) , where $\ell_1, \ell_2, \ell_3 \in \mathcal{P}_{\text{active}}$, and receives a set $\mathcal{C}$ of identities of corrupted parties. According to $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}|$, there are four cases.

1. If $\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\} = \emptyset$, $\mathcal{F}_{3\text{-triple}}$ samples a random degree-(1, 1) three-party triple $(\llbracket A \rrbracket_1, \llbracket B \rrbracket_1, \llbracket C \rrbracket_1)$ such that $C = A \cdot B$, and distributes it to three parties.
2. If $\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\} = \{\ell_u\}$, supposing $\{\ell_v, \ell_w\} = \{\ell_1, \ell_2, \ell_3\} \setminus \{\ell_u\}$, $\mathcal{F}_{3\text{-triple}}$ receives $\{f_u^U(x), g_u^U(y)\}_{U \in \{A, B, C\}}$ ^a from the adversary, randomly samples a degree-(1, 1) three-party triple $(\llbracket A \rrbracket_1, \llbracket B \rrbracket_1, \llbracket C \rrbracket_1)$ such that $U(x, u) = f_u^U(x)$, $U(u, y) = g_u^U(y)$ for every $U \in \{A, B, C\}$, and distributes it to three parties.
3. If $\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\} = \{\ell_u, \ell_v\}$, supposing $\{\ell_w\} = \{\ell_1, \ell_2, \ell_3\} \setminus \{\ell_u, \ell_v\}$, $\mathcal{F}_{3\text{-triple}}$ receives $\{f_u^U(x), g_u^U(y), f_v^U(x), g_v^U(y)\}_{U \in \{A, B, C\}}$ from the adversary, computes a degree-(1, 1) three-party triple $(\llbracket A \rrbracket_1, \llbracket B \rrbracket_1, \llbracket C \rrbracket_1)$ such that $U(x, u) = f_u^U(x)$, $U(u, y) = g_u^U(y)$, $U(x, v) = f_v^U(x)$, $U(v, x) = g_v^U(y)$ for every $U \in \{A, B, C\}$, and distributes it to three parties.
4. If $\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\} = \{\ell_1, \ell_2, \ell_3\}$, $\mathcal{F}_{3\text{-triple}}$ receives $\{f_v^U(x), g_v^U(y)\}_{v \in [3], U \in \{A, B, C\}}$ from the adversary, computes a degree-(1, 1) three-party triple $(\llbracket A \rrbracket_1, \llbracket B \rrbracket_1, \llbracket C \rrbracket_1)$ such that $U(x, v) = f_v^U(x)$, $U(v, y) = g_v^U(y)$ for $U \in \{A, B, C\}$, $v \in [3]$, and distributes it to three parties.

^a We assume, by default, that the adversary provides the functionality with shares of corrupted parties that are consistent to degree-(1, 1) Shamir sharings of a Beaver triple. In particular, 1) the shares are degree-1

polynomials for Cases 2, 3, and 4; 2) the polynomials agree on all common points held by two corrupted parties for Cases 3 and 4; 3) three secrets satisfy the multiplication relation for Case 4.

Protocol 10: $\Pi_{\text{packed-triple}}(k)$ in $\mathcal{F}_{\text{3-triple}}$, $\mathcal{F}_{\text{rand-sharing}}$, $\mathcal{F}_{\text{batched-vrfy}}$ -hybrid model

Let $N = (n')^3$. All parties aim to prepare $\frac{N}{9}$ random packed Beaver triples over bivariate polynomials. For simplicity of notations, let $\mathcal{P}_{\text{active}} = \{P_1, \dots, P_{n'}\}$. All parties agree on a designated party P_{king} .

0: Preparing random sharings.

1. All parties invoke $\mathcal{F}_{\text{rand-sharing}}(\Sigma_2, \frac{N-1}{2})$ and $\mathcal{F}_{\text{rand-sharing}}(\Sigma_5, \frac{N-1}{2})$ to prepare $\frac{N-1}{2}$ pairs of random double sharings in the form of $(\llbracket \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{R} \rrbracket_{t+2k-2})$, where $\llbracket \mathbf{R} \rrbracket_{t+2k-2}$ is computed as $\llbracket \mathbf{R} \rrbracket_{t+2k-2} \leftarrow \llbracket \mathbf{R} \rrbracket_{t+k-1} + \llbracket \mathbf{O} \rrbracket_{t+2k-2}$.

1: Packed triple distribution.

1. For $\ell \in [N]$, supposing $\ell = (\ell_1 - 1) \cdot (n')^2 + (\ell_2 - 1) \cdot n' + \ell_3$ for $\ell_1, \ell_2, \ell_3 \in \mathcal{P}_{\text{active}}$, three parties $P_{\ell_1}, P_{\ell_2}, P_{\ell_3}$ distribute a degree- $(t+k-1, t+k-1)$ packed Beaver triple to all parties.
 - (a) Three parties $P_{\ell_1}, P_{\ell_2}, P_{\ell_3}$ invoke k^2 times of $\mathcal{F}_{\text{3-triple}}$ to obtain k^2 three-party degree- $(1, 1)$ Beaver triples $\{\llbracket A_{\ell,ij} \rrbracket_1, \llbracket B_{\ell,ij} \rrbracket_1, \llbracket C_{\ell,ij} \rrbracket_1\}_{i,j \in [k]}$, where each P_{ℓ_v} holds two degree-1 polynomials $U_{\ell,ij}(x, v), U_{\ell,ij}(v, x)$ as its shares of $\llbracket U_{\ell,ij} \rrbracket_1$ for $U \in \{A, B, C\}, i, j \in [k]$.
 - (b) For $v \in \{1, 2, 3\}$, P_{ℓ_v} samples three random degree- $(t+k-1, t+k-1)$ packed Shamir sharings $(\llbracket \mathbf{A}_\ell(v) \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell(v) \rrbracket_{t+k-1}, \llbracket \mathbf{C}_\ell(v) \rrbracket_{t+k-1})$ such that $\mathbf{U}_\ell(v) = (U_{\ell,ij}(v, 0))_{ij \in \mathbb{F}^{k \times k}}$, and distributes them to all parties.

For $\mathbf{U} \in \{\mathbf{A}_\ell(v), \mathbf{B}_\ell(v), \mathbf{C}_\ell(v)\}$, all parties invoke $\pi_{\text{check-consistency-all}}(P_{\ell_v}, \llbracket \mathbf{U} \rrbracket_{t+k-1})$ to check whether P_{ℓ_v} distributes a valid degree- $(t+k-1, t+k-1)$ packed Shamir sharing.

– If the check fails, all parties take the disputed pair E as output.

- (c) For $\mathbf{U} \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, all parties locally compute

$$\llbracket \mathbf{U}_\ell \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(0) \cdot \llbracket \mathbf{U}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(0) \cdot \llbracket \mathbf{U}_\ell(2) \rrbracket_{t+k-1},$$

where $\lambda_{1,i}$ denotes the Lagrange coefficient for degree-1 polynomials at evaluation point $x = i$, i.e., $\lambda_{1,i}(\alpha) = \prod_{j \in \{1,2\}, j \neq i} \frac{(\alpha-j)}{(i-j)}$ for $i \in \{1, 2\}$.

2. **Checking consistency.** For $\ell \in [N]$ and $\mathbf{U} \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$, all parties locally compute

$$\llbracket \theta_\ell^{\mathbf{U}} \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(3) \cdot \llbracket \mathbf{U}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(3) \cdot \llbracket \mathbf{U}_\ell(2) \rrbracket_{t+k-1} - \llbracket \mathbf{U}_\ell(3) \rrbracket_{t+k-1}.$$

All parties invoke $\Pi_{\text{batched-vrfy}}(3N)$ with their shares of $\{\llbracket \theta_\ell^{\mathbf{U}} \rrbracket_{t+k-1}\}_{U \in \{A, B, C\}, \ell \in [N]}$ and their degree- $(1, 1)$ Shamir shares of $\{\llbracket U_{\ell,ij} \rrbracket_1\}_{U \in \{A, B, C\}, \ell \in [N], i, j \in [k]}$ to check whether each $\llbracket \theta_\ell^{\mathbf{U}} \rrbracket_{t+k-1}$ is a degree- $(t+k-1, t+k-1)$ packed Shamir sharing with secret $\mathbf{0} \in \mathbb{F}^{k \times k}$.

– If the check fails, all parties take the disputed pair E as output.

2: Packed triple extraction.

1. All parties set two $k \times k$ matrices of degree- $\frac{N-1}{2}$ polynomials $\mathbf{F}, \mathbf{G}$ such that

$$\forall \ell \in [\frac{N+1}{2}], \llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1} = \llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1} = \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}.$$

2. For every $\ell \in \{\frac{N+3}{2}, \dots, N\}$, all parties invoke Π_{mult} with their shares of $\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}$, a packed triple $(\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{C}_\ell \rrbracket_{t+k-1})$, an unused double sharing $(\llbracket \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{R} \rrbracket_{t+2k-2})$, to compute their shares of $\llbracket \mathbf{F}(\ell) * \mathbf{G}(\ell) \rrbracket_{t+k-1}$.
3. **Multiplication verification.** Suppose that in last step, all parties obtain their shares of $\{\llbracket \mathbf{V}_\ell \rrbracket_{k-1}, \llbracket \mathbf{V}_\ell \rrbracket_{t+k-1}\}_{\ell \in [N-1]}$ and $\{\llbracket \mathbf{W}_\ell \rrbracket_{k-1}, \llbracket \mathbf{W}_\ell \rrbracket_{t+2k-2}\}_{\ell \in [\frac{N-1}{2}]}$. All parties invoke $\mathcal{F}_{\text{batched-vrfy}}(P_{\text{king}}, \Sigma_8, \frac{3(N-1)}{2})$ with these shares to check whether each pair of sharings corresponds to the same secrets.

- If the check fails, all parties take the disputed pair E as output.
- 4. All parties set a $k \times k$ matrix of degree- $(N-1)$ polynomials $\mathbf{H}$ such that

$$\forall \ell \in [\frac{N+1}{2}], \llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1} = \llbracket \mathbf{C}_\ell \rrbracket_{t+k-1},$$

$$\forall \ell \in \{\frac{N+3}{2}, \dots, N\}, \llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1} = \llbracket \mathbf{F}(\ell) * \mathbf{G}(\ell) \rrbracket_{t+k-1}.$$

3: Triple verification.

1. For $\ell \in [\frac{10.5N}{27}]$, all parties compute their shares of $(\llbracket \mathbf{F}(N+\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N+\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N+\ell) \rrbracket_{t+k-1})$ and send them to $P_i := P_{(\ell \bmod n')}$, who checks whether the receiving shares form a degree- $(t+k-1, t+k-1)$ packed Beaver triple over bivariate polynomial.
 - (a) P_i invokes $\pi_{\text{check-consistency-one}}(P_i, \llbracket \mathbf{U} \rrbracket_{t+k-1})$ for every $\mathbf{U} \in \{\mathbf{F}(N+\ell), \mathbf{G}(N+\ell), \mathbf{H}(N+\ell)\}$ to check whether $\llbracket \mathbf{U} \rrbracket_{t+k-1}$ is a valid degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - If the check fails, all parties take the disputed pair E as output.
 - (b) P_i reconstructs the secrets $(\mathbf{F}(N+\ell), \mathbf{G}(N+\ell), \mathbf{H}(N+\ell))$ and checks whether $\mathbf{F}(N+\ell) * \mathbf{G}(N+\ell) = \mathbf{H}(N+\ell)$.
 - If the check fails, P_i broadcasts $(\text{Incorrect}, \ell, P_i)$.
 - Otherwise, P_i accept the sharings.
2. If every active party accepts, all parties take as output their shares of $\{(\llbracket \mathbf{F}(N+\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N+\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N+\ell) \rrbracket_{t+k-1})\}_{\ell \in \{\frac{10.5N}{27}+1, \dots, \frac{N+1}{2}\}}$.
 - Otherwise, the party broadcasting **Incorrect** with the smallest identity, say P_i , plays the role of a referee to locate a disputed pair. All parties invoke $\Pi_{\text{fault-localize-triple}}(P_i)$.

Protocol 11: $\Pi_{\text{batched-vrfy}}(N)$

All parties aim to check whether their shares of $\{\llbracket \boldsymbol{\theta}_\ell \rrbracket_{t+k-1}\}_{\ell \in [N]}$ form N degree- $(t+k-1, t+k-1)$ packed Shamir sharings with secret $\mathbf{0} \in \mathbb{F}^{k \times k}$. In case that faults occur, all parties locate a disputed pair relying on degree- $(1, 1)$ Shamir sharings.

All parties agree on an $n' \times (n' - t')$ Vandermonde matrix $\mathbf{M}$. All parties initialize their happy **Verification**.

- 1: All parties invoke $\mathcal{F}_{\text{rand-sharing}}(\Sigma_6, N)$ to prepare N random sharings in the form of $\llbracket \mathbf{O} \rrbracket_{t+k-1}$.
- 2: For each group of $n' - t'$ sharings $\{\llbracket \boldsymbol{\theta}_\ell \rrbracket_{t+k-1}\}_{\ell \in [n' - t']}$ to be checked, all parties pick $n' - t'$ unused random sharings $\{\llbracket \mathbf{O}_\ell \rrbracket_{t+k-1}\}_{\ell \in [n' - t']}$ and verify the correctness of $\{\llbracket \boldsymbol{\theta}_\ell \rrbracket_{t+k-1}\}_{\ell \in [n' - t]}$ as follows.
 1. For $\ell \in [n' - t']$, all parties locally compute $\llbracket \boldsymbol{\epsilon}_\ell \rrbracket_{t+k-1} \leftarrow \llbracket \boldsymbol{\theta}_\ell \rrbracket_{t+k-1} + \llbracket \mathbf{O}_\ell \rrbracket_{t+k-1}$.
 2. All parties locally compute

$$(\llbracket \boldsymbol{\epsilon}^{(1)} \rrbracket_{t+k-1}, \dots, \llbracket \boldsymbol{\epsilon}^{(n')} \rrbracket_{t+k-1})^\top \leftarrow \mathbf{M}^\top \cdot (\llbracket \boldsymbol{\epsilon}_1 \rrbracket_{t+k-1}, \dots, \llbracket \boldsymbol{\epsilon}_{n'-t'} \rrbracket_{t+k-1})^\top.$$

3. For $s \in \mathcal{P}_{\text{active}}$, all parties send their shares of $\llbracket \boldsymbol{\epsilon}^{(s)} \rrbracket_{t+k-1}$ to P_s . Then P_i checks the correctness of $\llbracket \boldsymbol{\epsilon}^{(s)} \rrbracket_{t+k-1}$ as follows.
 - (a) P_s invokes $\pi_{\text{check-consistency-one}}(P_s, \llbracket \boldsymbol{\epsilon}^{(s)} \rrbracket_{t+k-1})$ with all parties to check whether the received shares form degree- $(t+k-1, t+k-1)$ packed Shamir sharings.
 - If the check fails, all parties take the disputed pair E as output.
 - (b) P_i reconstructs the secrets $\boldsymbol{\epsilon}^{(s)}$ and checks whether $\boldsymbol{\epsilon}^{(s)} = \mathbf{0}$.
 - If the check fails, P_s broadcasts $(\text{Incorrect}, P_s)$.
 - Otherwise, P_s accepts the sharing.
4. If every active party accepts, all parties accept this group of sharings and continue checking the next group. Otherwise, the party broadcasting **Incorrect** with the smallest identity, say P_s , plays the role of a referee to locate a disputed pair as follows.

Fault localization.

Locating an incorrect sharing.

4. All parties send their shares of $\{\llbracket \theta_\ell \rrbracket_{t+k-1}, \llbracket O_\ell \rrbracket_{t+k-1}\}_{\ell \in [n'-t']}$ to P_s .
5. For $\ell \in [n' - t']$, P_s invokes $\pi_{\text{check-consistency-one}}(P_s, \llbracket O_\ell \rrbracket_{t+k-1})$ to check whether $\llbracket O_\ell \rrbracket_{t+k-1}$ is a degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - If the check fails, all parties take the disputed pair E as output.
6. P_s checks whether the shares sent by each party $P_{s'}$ are consistent with the shares sent in Step 2.3.
 - If not, P_s broadcasts $(P_{s'}, \text{Corrupt})$. All parties take the disputed pair $E \leftarrow \{P_s, P_{s'}\}$ as output.
7. For $\ell \in [n' - t']$, P_s checks the correctness of $\llbracket \theta_\ell \rrbracket_{t+k-1}$ as follows.
 - (a) P_s invokes $\pi_{\text{check-consistency-one}}(P_s, \llbracket \theta_\ell \rrbracket_{t+k-1})$ to check whether $\llbracket \theta_\ell \rrbracket_{t+k-1}$ is a degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - If the check fails, all parties take the disputed pair E as output.
 - (b) P_s reconstructs the secret θ_ℓ and checks whether $\theta_\ell = \mathbf{0} \in \mathbb{F}^{k \times k}$.
 - If the check fails, P_s broadcasts $(\ell, i, j, \text{Incorrect})$ to indicate $\theta_{\ell, ij} \neq 0$, where $\theta_\ell = (\theta_{\ell, ij})_{ij}$. Then a disputed pair will be identified as follows.

Identifying a disputed pair.

8. Letting $\ell = (\ell_1 - 1) \cdot (n')^2 + (\ell_2 - 1) \cdot n' + \ell_3$, each P_{ℓ_v} broadcasts its shares of $\llbracket U_{\ell, ij} \rrbracket_1$, i.e., two degree-1 polynomials $U_{\ell, ij}(x, v), U_{\ell, ij}(v, x)$, and the sharing $\llbracket U_\ell(v) \rrbracket_{t+k-1}$. All parties locate a disputed pair as follows.
 - (a) Each party $P_{s'}$ checks whether the values broadcast by P_{ℓ_v} are different from values received in Step 1.1.b of $\Pi_{\text{packed-triple}}$.
 - If the check fails, $P_{s'}$ broadcasts $(\text{Inconsistent}, P_{s'}, P_{\ell_v})$. All parties set $E \leftarrow \{P_{s'}, P_{\ell_v}\}$ and take E as output.
 - (b) All parties check whether shares of $\llbracket U_{\ell, ij} \rrbracket_1$ broadcast by P_{ℓ_v} are consistent with degree-1 Shamir sharing, i.e., $U_{\ell, ij}(x, v), U_{\ell, ij}(v, y)$ are of degree-1 and they agree on the common point.
 - If the check fails, all parties set $E \leftarrow \{P_{\ell_v}\}$, and take E as output.
 - (c) All parties check whether the sharing $\llbracket U_\ell(v) \rrbracket_{t+k-1}$ broadcast by P_{ℓ_v} is a degree- $(t+k-1, t+k-1)$ packed Shamir sharing and the secret $U_{\ell, ij}(v) = U_{\ell, ij}(v, 0)$.
 - If the check fails, all parties set $E \leftarrow \{P_{\ell_v}\}$, and take E as output.
 - (d) All parties check whether two parties $P_{\ell_v}, P_{\ell_{v'}}$ broadcast the same values for common points $U_{\ell, ij}(v, v'), U_{\ell, ij}(v', v)$ which are jointly held by them.
 - If the check fails, all parties set $E \subset \{P_{\ell_v}, P_{\ell_{v'}}\}$, and take E as output.
- If none of the cases above happens, all parties set $E \leftarrow \{P_s\}$ and take E as output.

Protocol 12: Π_{mult}

All parties hold their shares of $\llbracket \mathbf{X} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} \rrbracket_{t+k-1}$, and aim to compute their shares of $\llbracket \mathbf{X} * \mathbf{Y} \rrbracket_{t+k-1}$ by consuming

- a degree- $(t+k-1, t+k-1)$ packed Beaver triple $(\llbracket \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{B} \rrbracket_{t+k-1}, \llbracket \mathbf{C} \rrbracket_{t+k-1})$ and
 - a pair of random packed Shamir sharings $(\llbracket \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{R} \rrbracket_{t+2k-2})$.^a
1. All parties compute $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1} = \llbracket \mathbf{X} \rrbracket_{t+k-1} + \llbracket \mathbf{A} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1} = \llbracket \mathbf{Y} \rrbracket_{t+k-1} + \llbracket \mathbf{B} \rrbracket_{t+k-1}$, and send their shares to P_{king} .
 2. For $\mathbf{U} \in \{\mathbf{X} + \mathbf{A}, \mathbf{Y} + \mathbf{B}\}$, P_{king} and all parties invoke $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket \mathbf{U} \rrbracket_{t+k-1})$ to check whether all parties send correct shares to P_{king} .
 - If the check fails, all parties take the disputed pair E as output.

3. P_{king} reconstructs $\mathbf{X} + \mathbf{A}$ and $\mathbf{Y} + \mathbf{B}$, and distributes $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1}, \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1}$ to all parties.
4. For $\mathbf{U} \in \{\mathbf{X} + \mathbf{A}, \mathbf{Y} + \mathbf{B}\}$, all parties invoke $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket \mathbf{U} \rrbracket_{k-1})$ to check whether P_{king} distributes consistent degree- $(k-1, k-1)$ packed Shamir sharings.
5. All parties locally compute

$$\begin{aligned} \llbracket \mathbf{Z} \rrbracket_{t+2k-2} \leftarrow & \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} - \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1} \cdot \llbracket \mathbf{B} \rrbracket_{t+k-1} \\ & - \llbracket \mathbf{A} \rrbracket_{t+k-1} \cdot \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1} + \llbracket \mathbf{C} \rrbracket_{t+k-1}. \end{aligned}$$

All parties then locally compute $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2} \leftarrow \llbracket \mathbf{Z} \rrbracket_{t+2k-2} + \llbracket \mathbf{R} \rrbracket_{t+2k-2}$, and send their shares to P_{king} .

6. P_{king} and all parties invoke $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2})$ to check whether all parties send correct shares to P_{king} .
 - If the check fails, all parties take the disputed pair E as output.
7. P_{king} reconstructs $\mathbf{Z} + \mathbf{R}$ and distributes $\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}$ to all parties.
8. All parties invoke $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1})$ to check whether P_{king} distributes consistent degree- $(k-1, k-1)$ packed Shamir sharings.
9. All parties locally compute $\llbracket \mathbf{Z} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1} - \llbracket \mathbf{R} \rrbracket_{t+k-1}$.
10. All parties additionally take as output their shares of $(\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{k-1}, \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}), (\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{k-1}, \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1})$, and $(\llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{k-1}, \llbracket \mathbf{Z} + \mathbf{R} \rrbracket_{t+2k-2})$, which will be batched and verified to check whether P_{king} distributes degree- $(k-1, k-1)$ packed Shamir sharings with correct secrets.

^a Different from the situation in MPC in the preprocessing model, as all parties can verify whether P_{king} distributes packed Shamir sharings of degree- $(k-1, k-1)$ and doesn't mount a double dipping attack, they don't need to add degree- $(n'-1, n'-1)$ packed Shamir sharings with secret $\mathbf{0}$ to protect privacy of shares of honest parties.

Protocol 13: $\Pi_{\text{fault-localize-triple}}(P_i)$

Checking P_i .

1. P_i broadcasts the inconsistent packed triple it detected $(\llbracket \mathbf{F}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N + \ell) \rrbracket_{t+k-1})$. All parties check P_i as follows.
 - (a) Each party $P_{i'}$ checks whether P_i broadcasts the same shares as it sent to P_i .
 - If the check fails, $P_{i'}$ broadcasts $(\text{Incorrect}, P_{i'})$. All parties set $E \leftarrow \{P_i, P_{i'}\}$ and take E as output.
 - (b) All parties check each of the three sharings broadcast by P_i is a degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - (c) All parties reconstruct the secrets $\mathbf{F}(N + \ell), \mathbf{G}(N + \ell), \mathbf{H}(N + \ell)$ and check whether they violate the multiplication relation, i.e., $\mathbf{F}(N + \ell) * \mathbf{G}(N + \ell) \neq \mathbf{H}(N + \ell)$.
 - If one of two checks above fails, all parties set $E \leftarrow \{P_i\}$, and take E as output.

Locating an incorrect packed triple.

2. All parties send their shares of $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}\}_{\ell \in [N]}$ to P_i .
3. For $\ell \in [N]$, P_i invokes $\pi_{\text{check-consistency-one}}(P_i, \llbracket \mathbf{U} \rrbracket_{t+k-1})$ for every $\mathbf{U} \in \{\mathbf{A}_\ell, \mathbf{B}_\ell, \mathbf{C}_\ell\}$ to check whether the received shares form a degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - If the check fails, all parties take the disputed pair E as output.
4. For $\ell \in [N]$, P_i reconstructs the secrets $(\mathbf{A}_\ell, \mathbf{B}_\ell, \mathbf{C}_\ell)$ and checks whether $\mathbf{A}_\ell * \mathbf{B}_\ell = \mathbf{C}_\ell$.
 - P_i broadcasts $(\ell, \text{Incorrect})$ and the received shares $(\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{C}_\ell \rrbracket_{t+k-1})$ to indicate the ℓ -th packed triple does not satisfy the multiplication relation.

Identifying a disputed pair.

5. All parties locate a disputed pair as follows.

- (a) Each party $P_{i'}$ checks whether P_i broadcasts the same shares as it received from the dealer in Step 1.1.b of $\Pi_{\text{packed-triple}}$.
 - If the check fails, $P_{i'}$ broadcasts $(\text{Incorrect}, P_{i'})$. All parties set $E \leftarrow \{P_i, P_{i'}\}$ and take E as output.
- (b) All parties check whether each of the three sharings broadcast by P_i is a degree- $(t+k-1, t+k-1)$ packed Shamir sharing.
 - If the check fails, all parties set $E \leftarrow \{P_i\}$, and take E as output.
- (c) All parties reconstructs the secrets $\mathbf{A}_\ell, \mathbf{B}_\ell, \mathbf{C}_\ell$.
 - If $\mathbf{A}_\ell * \mathbf{B}_\ell \neq \mathbf{C}_\ell$, letting $\{\ell_1, \ell_2, \ell_3\}$ be the identities of three parties distributing the ℓ -th packed triple, all parties set $E \leftarrow \{P_{\ell_1}, P_{\ell_2}\}$ and take E as output. In this case, $\{\ell_1, \ell_2, \ell_3\}$ contains at least two corrupted parties.
 - If $\mathbf{A}_\ell * \mathbf{B}_\ell = \mathbf{C}_\ell$, all parties set $E \leftarrow \{P_i\}$, and take E as output.

Lemma 6. *Let $t = (n-1)/3$ and $k \leq (t+2)/2$. $\Pi_{\text{packed-triple}}(k)$ computes $\mathcal{F}_{\text{packed-triple}}(\frac{N}{9}, k)$ with perfect security in the $\mathcal{F}_{3\text{-triple}}, \mathcal{F}_{\text{rand-sharing}}, \mathcal{F}_{\text{batched-vrfy}}$ -hybrid model against a fully malicious adversary who controls at most t corrupted parties.*

Proof (Proof of Lemma 6). We will construct a simulator $\mathcal{S}$ to simulate the behaviors of honest parties. Let $\mathcal{C}$ denote the set of corrupted parties and $\mathcal{H}$ denote the set of honest parties. Let $\mathcal{C}_{\text{active}}$ denote the set of active corrupted parties and $t' = |\mathcal{C}_{\text{active}}|$. Let $\mathcal{H}_{\text{active}}$ denote the set of active honest parties. The core of the simulation is to delay the sampling of packed triples distributed by committees containing at least two honest parties to argue that the extracted $N/9$ packed triples are unknown to the adversary.

The simulator $\mathcal{S}$ internally invokes the adversary with randomness sampled by itself and works as follows.

1. In Step 0, $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand-sharing}}(\Sigma_2, \frac{N-1}{2})$ and $\mathcal{F}_{\text{rand-sharing}}(\Sigma_5, \frac{N-1}{2})$, receives corrupted parties' shares of $\llbracket \mathbf{R} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{O} \rrbracket_{t+2k-2}$ from the adversary, and sends them back to the adversary on behalf of $\mathcal{F}_{\text{rand-sharing}}$.

Simulation of packed triple distribution.

2. In Step 1.1.a, for each $\ell \in [N]$ such that $\{\ell_1, \ell_2, \ell_3\} \cap \mathcal{C} \neq \emptyset$, for $i, j \in [k]$, $\mathcal{S}$ emulates $\mathcal{F}_{3\text{-triple}}$, receives corrupted parties' shares from the adversary, records them, and sends them back to the adversary on behalf of $\mathcal{F}_{3\text{-triple}}$. According to $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}|$, there are three cases.
 - If $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| = 3$, $\mathcal{S}$ learns three values $U_{\ell,ij}(1,0), U_{\ell,ij}(2,0), U_{\ell,ij}(3,0)$.
 - If $\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\} = \{\ell_u, \ell_v\}$, supposing $\{\ell_w\} = \{\ell_1, \ell_2, \ell_3\} \setminus \{\ell_u, \ell_v\}$, $\mathcal{S}$ learns two values $U_{\ell,ij}(u,0), U_{\ell,ij}(v,0)$ and then computes the degree-1 polynomial $U_{\ell,ij}(x,0)$, and therefore $U_{\ell,ij}(w,0)$.
 - If $\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\} = \{\ell_u\}$, $\mathcal{S}$ learns $U_{\ell,ij}(u,0)$.

This implies that when $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \leq 1$, the Beaver triple is unknown to the adversary. When $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \geq 2$, the whole sharings of the Beaver triple is known to the adversary.

3. In Step 1.1.b,
 - when a corrupted P_{ℓ_v} distributes three degree- $(t+k-1, t+k-1)$ packed Shamir sharings $(\llbracket \mathbf{A}_\ell(v) \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell(v) \rrbracket_{t+k-1}, \llbracket \mathbf{C}_\ell(v) \rrbracket_{t+k-1})$, $\mathcal{S}$ receives shares of honest parties from the adversary, and honestly simulates honest parties in $\pi_{\text{check-consistency-all}}(P_{\ell_v}, \llbracket \mathbf{U} \rrbracket_{t+k-1})$ for $\mathbf{U} \in \{\mathbf{A}_\ell(v), \mathbf{B}_\ell(v), \mathbf{C}_\ell(v)\}$;
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
 - If the check passes, $\mathcal{S}$ computes the whole sharing $(\widetilde{\llbracket \mathbf{A}_\ell(v) \rrbracket}_{t+k-1}, \widetilde{\llbracket \mathbf{B}_\ell(v) \rrbracket}_{t+k-1}, \widetilde{\llbracket \mathbf{C}_\ell(v) \rrbracket}_{t+k-1})$ according to shares of honest parties, and learns the secrets $\widetilde{\mathbf{A}_\ell(v)}, \widetilde{\mathbf{B}_\ell(v)}, \widetilde{\mathbf{C}_\ell(v)}$, where we write $\widetilde{\mathbf{U}}$ to distinguish it from the value $\mathbf{U}$ computed when emulating $\mathcal{F}_{3\text{-triple}}$.
 - when an honest P_{ℓ_v} distributes three degree- $(t+k-1, t+k-1)$ packed Shamir sharings $(\llbracket \mathbf{A}_\ell(v) \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell(v) \rrbracket_{t+k-1}, \llbracket \mathbf{C}_\ell(v) \rrbracket_{t+k-1})$, $\mathcal{S}$ samples random degree- $(t+k-1)$ polynomials $\{f_j^U(x), g_j^U(y)\}_{j \in \mathcal{C}_{\text{active}}}$ such that $f_j^U(\alpha_i) = g_i^U(\alpha_j)$ for $i, j \in \mathcal{C}_{\text{active}}$, and sends them to the adversary on behalf of honest P_{ℓ_v} . When simulating $\pi_{\text{check-consistency-all}}(P_{\ell_v}, \llbracket \mathbf{U} \rrbracket_{t+k-1})$ for every $\mathbf{U} \in \{\mathbf{A}_\ell(v), \mathbf{B}_\ell(v), \mathbf{C}_\ell(v)\}$, for each active honest party P_i , $\mathcal{S}$ computes the values of P_i from sampled shares of corrupted parties, i.e., $f_i^U(\alpha_j) = g_j^U(\alpha_i)$, $g_i^U(\alpha_j) = f_j^U(\alpha_i)$ for $j \in \mathcal{C}_{\text{active}}$. Then $\mathcal{S}$ simulates behaviors of honest parties using the values computed above in $\pi_{\text{check-consistency-all}}(P_{\ell_v}, \llbracket \mathbf{U} \rrbracket_{t+k-1})$.

- If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
- In Step 1.1.c, for each $\ell \in [N]$, for $\mathbf{U} \in \{\mathbf{A}_\ell, \mathbf{B}_\ell, \mathbf{C}_\ell\}$, $\mathcal{S}$ computes the additive error $\Delta \mathbf{U}(v) \in \mathbb{F}^{k \times k}$ on $\llbracket \mathbf{U}(v) \rrbracket_{t+k-1}$ for each $v \in [3]$ as follows.
 - For $\ell_v \in \mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}$, recall $\mathcal{S}$ learns $\widetilde{\mathbf{U}(v)}$ when simulating Step 1.1.b and learns $\mathbf{U}(v)$ when emulating $\mathcal{F}_{\text{3-triple}}$. $\mathcal{S}$ computes $\Delta \mathbf{U}(v) \leftarrow \widetilde{\mathbf{U}(v)} - \mathbf{U}(v)$, where $\mathbf{U}(v) = (U_{\ell,ij}(v, 0))_{i,j \in [k]}$.
 - For $\ell_v \in \mathcal{H} \cap \{\ell_1, \ell_2, \ell_3\}$, $\mathcal{S}$ sets $\Delta \mathbf{U}(v) = \mathbf{0}$.
 - In Step 1.2, $\mathcal{S}$ simulates $\Pi_{\text{batched-vrfy}}$.
 - In Step 1, $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand-sharing}}(\Sigma_6, N)$, receives shares of corrupted parties from the adversary, records them, and sends them back to the adversary on behalf of $\mathcal{F}_{\text{rand-sharing}}$.
 - In Step 2.1, for $\ell \in [n' - t']$, $\mathcal{S}$ computes corrupted parties' shares of $\llbracket \epsilon_\ell \rrbracket_{t+k-1}$ as $\llbracket \epsilon_\ell \rrbracket_{t+k-1} \leftarrow \llbracket \theta_\ell \rrbracket_{t+k-1} + \llbracket \mathbf{O}_\ell \rrbracket_{t+k-1}$. $\mathcal{S}$ randomly samples a degree- $(t+k-1, t+k-1)$ packed Shamir sharing as $\llbracket \epsilon_\ell \rrbracket_{t+k-1}$ based on shares of corrupted parties and the secret $\epsilon_\ell = \lambda_{1,1}(3) \cdot \Delta \mathbf{U}_\ell(1) + \lambda_{1,2}(3) \cdot \Delta \mathbf{U}_\ell(2) - \Delta \mathbf{U}_\ell(3) \in \mathbb{F}^{k \times k}$.
 - In Step 2.2, $\mathcal{S}$ computes the whole sharings of $\{\llbracket \epsilon^{(\ell)} \rrbracket_{t+k-1}\}_{\ell \in [n' - t']}$ by computing

$$(\llbracket \epsilon^{(1)} \rrbracket_{t+k-1}, \dots, \llbracket \epsilon^{(n')} \rrbracket_{t+k-1})^\top \leftarrow \mathbf{M}^\top \cdot (\llbracket \epsilon_1 \rrbracket_{t+k-1}, \dots, \llbracket \epsilon_{n'-t'} \rrbracket_{t+k-1})^\top.$$

- In Step 2.3, since $\mathcal{S}$ has honest parties' shares of $\{\llbracket \epsilon^{(\ell)} \rrbracket_{t+k-1}\}_{\ell \in [N]}$, $\mathcal{S}$ faithfully follows the protocol.
 - In Step 2.3.a, if a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
- In Step 2.4, if some party P_s broadcasts **(Incorrect, P_s)**, $\mathcal{S}$ samples the honest parties' shares of $(\llbracket \theta_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{O}_\ell \rrbracket_{t+k-1})$ based on shares of corrupted parties and the secrets $\theta_\ell^U = \lambda_{1,1}(3) \cdot \Delta \mathbf{U}_\ell(1) + \lambda_{1,2}(3) \cdot \Delta \mathbf{U}_\ell(2) + \Delta \mathbf{U}_\ell(3)$ and $\mathbf{O}_\ell = \mathbf{0}$. Then $\mathcal{S}$ simulates the protocol honestly to identify a disputed pair E . Then $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.

If all parties accept, $\mathcal{S}$ further computes as follows.

- If $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \geq 2$, $\mathcal{S}$ computes the whole sharings of the packed triple as $\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(0) \cdot \llbracket \mathbf{A}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(0) \cdot \llbracket \mathbf{A}_\ell(2) \rrbracket_{t+k-1}$, $\llbracket \mathbf{B}_\ell \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(0) \cdot \llbracket \mathbf{B}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(0) \cdot \llbracket \mathbf{B}_\ell(2) \rrbracket_{t+k-1}$, $\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(0) \cdot \llbracket \mathbf{C}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(0) \cdot \llbracket \mathbf{C}_\ell(2) \rrbracket_{t+k-1}$, which implies the whole packed triple is known to the adversary.
 Define the set $\mathcal{E} \subset [N]$ to contain all such ℓ s. Then $|\mathcal{E}| = \frac{7N}{27}$. Let $\mathcal{E}_1 = \mathcal{E} \cap [\frac{N+1}{2}]$ and $\mathcal{E}_2 = \mathcal{E} \setminus \mathcal{E}_1$, i.e., $\mathcal{E}_2 = \mathcal{E} \cap \{\frac{N+3}{2}, \dots, N\}$.
- If $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \leq 1$, $\mathcal{S}$ computes corrupted parties' shares as $\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(0) \cdot \llbracket \mathbf{A}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(0) \cdot \llbracket \mathbf{A}_\ell(2) \rrbracket_{t+k-1}$, $\llbracket \mathbf{B}_\ell \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(0) \cdot \llbracket \mathbf{B}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(0) \cdot \llbracket \mathbf{B}_\ell(2) \rrbracket_{t+k-1}$, $\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1} \leftarrow \lambda_{1,1}(0) \cdot \llbracket \mathbf{C}_\ell(1) \rrbracket_{t+k-1} + \lambda_{1,2}(0) \cdot \llbracket \mathbf{C}_\ell(2) \rrbracket_{t+k-1}$, which implies the secrets of the packed triple is unknown to the adversary.
 Define the set $\mathcal{D} \subset [N]$ to contain all such ℓ s. Then $|\mathcal{D}| = \frac{20N}{27}$. Let $\mathcal{D}_1 = \mathcal{D} \cap [\frac{N+1}{2}]$ and $\mathcal{D}_2 = \mathcal{D} \setminus \mathcal{D}_1$, i.e., $\mathcal{D}_2 = \mathcal{D} \cap \{\frac{N+3}{2}, \dots, N\}$. Let $\mathcal{D}_{1,1} \subset \mathcal{D}_1$ be the subset in $\mathcal{D}_1$ with $|\mathcal{E}_2|$ smallest identities. Let $\mathcal{D}_{1,2} \subset \mathcal{D} \setminus \mathcal{D}_{1,1}$ be the subset of $\mathcal{D}_1 \setminus \mathcal{D}_{1,1}$ with $|\frac{3.5N}{27}|$ smallest identities. Finally, let $\mathcal{D}_{1,0} = \mathcal{D}_1 \setminus (\mathcal{D}_{1,1} \cup \mathcal{D}_{1,2})$.

Simulation of packed triple extraction.

- In Step 2.1, for every $\ell \in \{\frac{N+3}{2}, \dots, N\}$, $\mathcal{S}$ computes corrupted parties' shares of $\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}$ and $\llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}$ as linear combinations of their shares of $\{\llbracket \mathbf{F}(\ell') \rrbracket_{t+k-1}\}_{\ell' \in [\frac{N-1}{2}]}$ and $\{\llbracket \mathbf{G}(\ell') \rrbracket_{t+k-1}\}_{\ell' \in [\frac{N-1}{2}]}$, respectively.
- In Step 2.2, for each $\ell \in \{\frac{N+3}{2}, \dots, N\}$, $\mathcal{S}$ simulates the invocation of Π_{mult} according whether the ℓ -th packed triple is known by the adversary. Recall $\mathcal{S}$ learns corrupted parties' shares of $\llbracket \mathbf{X} \rrbracket_{t+k-1}, \llbracket \mathbf{Y} \rrbracket_{t+k-1}$ when simulating Step 2.1. Recall $\mathcal{S}$ learns corrupted parties' shares of $(\llbracket \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{R} \rrbracket_{t+2k-2})$ when emulating $\mathcal{F}_{\text{rand-sharing}}(\Sigma_2, \frac{N-1}{2})$ and $\mathcal{F}_{\text{rand-sharing}}(\Sigma_5, \frac{N-1}{2})$ in Step 0.
 - If the ℓ -th packed triple $(\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{C}_\ell \rrbracket_{t+k-1})$ is known to the adversary, i.e., $\ell \in \mathcal{E}_2$, $\mathcal{S}$ simulates as follows. Recall $\mathcal{S}$ learns the whole sharing of the packed triple $(\llbracket \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{B} \rrbracket_{t+k-1}, \llbracket \mathbf{C} \rrbracket_{t+k-1})$ when simulating Step 3 of $\Pi_{\text{batched-vrfy}}$.
 - (a) In Step 1, $\mathcal{S}$ randomly samples two degree- $(t+k-1, t+k-1)$ packed Shamir sharings as $\llbracket \mathbf{X} \rrbracket_{t+k-1}, \llbracket \mathbf{Y} \rrbracket_{t+k-1}$ based on shares of corrupted parties. $\mathcal{S}$ computes honest parties' shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1}$ as $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{X} \rrbracket_{t+k-1} + \llbracket \mathbf{A} \rrbracket_{t+k-1}$, $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{Y} \rrbracket_{t+k-1} + \llbracket \mathbf{B} \rrbracket_{t+k-1}$, and sends them P_{king} on behalf of honest parties.

- (b) In Step 2, for $U \in \{X + A, Y + B\}$, since $\mathcal{S}$ learns honest parties' shares of $\llbracket U \rrbracket_{t+k-1}$, $\mathcal{S}$ honestly simulates behaviors of honest parties in $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket U \rrbracket_{t+k-1})$.
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
- (c) In Step 3, if P_{king} is honest, $\mathcal{S}$ follows the protocol honestly. $\mathcal{S}$ receives honest parties' shares of $\llbracket X + A \rrbracket_{k-1}$ and $\llbracket Y + B \rrbracket_{k-1}$ from P_{king} , where we write $\bar{U}$, which is distributed by possibly corrupt P_{king} , to distinguish it from the value U , which is computed by the simulator $\mathcal{S}$.
- (d) In Step 4, for $U \in \{X + A, Y + B\}$, since $\mathcal{S}$ learns honest parties' shares of $\llbracket U \rrbracket_{k-1}$, $\mathcal{S}$ faithfully simulates the behaviors of honest parties in $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket U \rrbracket_{k-1})$.
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
- (e) In Step 5, $\mathcal{S}$ computes honest parties' shares of $\llbracket Z \rrbracket_{t+2k-2}$ as $\llbracket Z \rrbracket_{t+2k-2} \leftarrow \llbracket X + A \rrbracket_{k-1} \cdot \llbracket Y + B \rrbracket_{k-1} - \llbracket X + A \rrbracket_{k-1} \cdot \llbracket B \rrbracket_{t+k-1} - \llbracket A \rrbracket_{t+k-1} \cdot \llbracket Y + B \rrbracket_{k-1} + \llbracket C \rrbracket_{t+k-1}$. Then $\mathcal{S}$ randomly samples a pair of double sharings $(\llbracket R \rrbracket_{t+k-1}, \llbracket R \rrbracket_{t+2k-2})$ based on shares of corrupted parties. $\mathcal{S}$ computes honest parties' shares of $\llbracket Z + R \rrbracket_{t+2k-2}$ as $\llbracket Z + R \rrbracket_{t+2k-2} \leftarrow \llbracket Z \rrbracket_{t+2k-2} + \llbracket R \rrbracket_{t+2k-2}$, and sends them to P_{king} on behalf of honest parties.
- (f) In Step 6, since $\mathcal{S}$ learns honest parties' shares of $\llbracket Z + R \rrbracket_{t+2k-2}$, $\mathcal{S}$ faithfully simulates behaviors of honest parties in $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket Z + R \rrbracket_{t+2k-2})$.
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
- (g) In Step 7, if P_{king} is honest, $\mathcal{S}$ follows the protocol honestly. $\mathcal{S}$ receives honest parties' shares of $\llbracket Z + R \rrbracket_{k-1}$, where we write $\tilde{U}$, which is distributed by possibly corrupt P_{king} , to distinguish it from the value $\bar{U}$, which is computed by the simulator $\mathcal{S}$.
- (h) In Step 8, since $\mathcal{S}$ learns honest parties' shares of $\llbracket Z + R \rrbracket_{k-1}$, $\mathcal{S}$ honestly simulates the behaviors of honest parties in $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket Z + R \rrbracket_{k-1})$.
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.

$\mathcal{S}$ computes the whole sharing $\llbracket Z + R \rrbracket_{k-1}$ according to shares of honest parties.
- (i) In Step 9, $\mathcal{S}$ computes the whole sharing $\llbracket \tilde{Z} \rrbracket_{t+k-1}$ as $\llbracket \tilde{Z} \rrbracket_{t+k-1} \leftarrow \llbracket Z + R \rrbracket_{k-1} - \llbracket R \rrbracket_{t+k-1}$.
- If the ℓ -th packed triple $(\llbracket A_\ell \rrbracket_{t+k-1}, \llbracket B_\ell \rrbracket_{t+k-1}, \llbracket C_\ell \rrbracket_{t+k-1})$ is unknown by the adversary, i.e., $\ell \in \mathcal{D}_2$, $\mathcal{S}$ simulates as follows. Recall $\mathcal{S}$ learns corrupted parties' shares of the packed triple $(\llbracket A \rrbracket_{t+k-1}, \llbracket B \rrbracket_{t+k-1}, \llbracket C \rrbracket_{t+k-1})$ when simulating Step 2.4 of $\Pi_{\text{batched-vrfy}}$.
 - (a) In Step 1, $\mathcal{S}$ computes corrupted parties' shares of $\llbracket X + A \rrbracket_{t+k-1}, \llbracket Y + B \rrbracket_{t+k-1}$. $\mathcal{S}$ randomly samples two degree- $(t+k-1, t+k-1)$ packed Shamir sharings as $\llbracket X + A \rrbracket_{t+k-1}, \llbracket Y + B \rrbracket_{t+k-1}$ based on shares of corrupted parties. $\mathcal{S}$ sends honest parties' shares of $\llbracket X + A \rrbracket_{t+k-1}, \llbracket Y + B \rrbracket_{t+k-1}$ to P_{king} on behalf of honest parties.
 - (b) In Step 2, for $U \in \{X + A, Y + B\}$, $\mathcal{S}$ simulates the behaviors of honest parties in $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket U \rrbracket_{t+k-1})$ honestly.
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
 - (c) In Step 3, if P_{king} is honest, $\mathcal{S}$ follows the protocol honestly. $\mathcal{S}$ received honest parties' shares of $\llbracket X + A \rrbracket_{k-1}$ and $\llbracket Y + B \rrbracket_{k-1}$ from P_{king} .
 - (d) In Step 4, for $U \in \{X + A, Y + B\}$, since $\mathcal{S}$ learns honest parties' shares of $\llbracket U \rrbracket_{k-1}$, $\mathcal{S}$ faithfully simulates the behaviors of honest parties in $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \llbracket U \rrbracket_{k-1})$.
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.

$\mathcal{S}$ computes the whole sharings of $\llbracket X + A \rrbracket_{k-1}, \llbracket Y + B \rrbracket_{k-1}$ according to shares of honest parties.
 - (e) In Step 5, $\mathcal{S}$ computes corrupted parties' shares of $\llbracket Z + R \rrbracket_{t+2k-2} \leftarrow \llbracket X + A \rrbracket_{k-1} \cdot \llbracket Y + B \rrbracket_{k-1} - \llbracket X + A \rrbracket_{k-1} \cdot \llbracket B \rrbracket_{t+k-1} - \llbracket A \rrbracket_{t+k-1} \cdot \llbracket Y + B \rrbracket_{k-1} + \llbracket C \rrbracket_{t+k-1} + \llbracket R \rrbracket_{t+2k-2}$. $\mathcal{S}$ randomly samples a degree- $(t+2k-2, t+2k-2)$ packed Shamir sharing as $\llbracket Z + R \rrbracket_{t+2k-2}$ based on shares of corrupted parties, and sends shares of honest parties to P_{king} on behalf of honest parties.
 - (f) In Step 6, $\mathcal{S}$ faithfully simulates behaviors of honest parties in $\pi_{\text{check-consistency-one}}(P_{\text{king}}, \llbracket Z + R \rrbracket_{t+2k-2})$.
 - If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
 - (g) In Step 7, if P_{king} is honest, $\mathcal{S}$ follows the protocol honestly. $\mathcal{S}$ receives honest parties' shares of $\llbracket Z + R \rrbracket_{k-1}$ from P_{king} .

- (h) In Step 8, since $\mathcal{S}$ learns honest parties' shares of $\widetilde{\llbracket \mathbf{Z} + \mathbf{R} \rrbracket}_{k-1}$, $\mathcal{S}$ honestly simulates the behaviors of honest parties in $\pi_{\text{check-consistency-all}}(P_{\text{king}}, \widetilde{\llbracket \mathbf{Z} + \mathbf{R} \rrbracket}_{k-1})$.
- If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
- $\mathcal{S}$ computes the whole sharing $\widetilde{\llbracket \mathbf{Z} + \mathbf{R} \rrbracket}_{k-1}$ according to shares of honest parties.
- (i) In Step 9, $\mathcal{S}$ computes corrupted parties' shares of $\llbracket \tilde{\mathbf{Z}} \rrbracket_{t+k-1}$ as $\llbracket \tilde{\mathbf{Z}} \rrbracket_{t+k-1} \leftarrow \widetilde{\llbracket \mathbf{Z} + \mathbf{R} \rrbracket}_{k-1} - \llbracket \mathbf{R} \rrbracket_{t+k-1}$.
8. In Step 2.3, recall $\mathcal{S}$ learns honest parties' shares of $\{\llbracket \mathbf{V}_\ell \rrbracket_{k-1}, \llbracket \mathbf{V}_\ell \rrbracket_{t+k-1}\}_{\ell \in [N]}$ and $\{\llbracket \tilde{\mathbf{U}}_\ell \rrbracket_{k-1}, \llbracket \tilde{\mathbf{U}}_\ell \rrbracket_{t+2k-2}\}_{\ell \in [\frac{N-1}{2}]}$ when simulating Π_{mult} . $\mathcal{S}$ emulates $\mathcal{F}_{\text{batched-vrfy}}(P_{\text{king}}, \Sigma)$ as follows. $\mathcal{S}$ sends the honest parties' shares of $\{\llbracket \mathbf{V}_\ell \rrbracket_{k-1}, \llbracket \mathbf{V}_\ell \rrbracket_{t+k-1}\}_{\ell \in [N]}$ and $\{\llbracket \tilde{\mathbf{U}}_\ell \rrbracket_{k-1}, \llbracket \tilde{\mathbf{U}}_\ell \rrbracket_{t+2k-2}\}_{\ell \in [\frac{N-1}{2}]}$ to the adversary on behalf of $\mathcal{F}_{\text{batched-vrfy}}$. Then $\mathcal{S}$ checks whether $\overline{\mathbf{V}}_\ell = \mathbf{V}_\ell$ for every $\ell \in [N-1]$ and $\overline{\mathbf{U}}_\ell = \mathbf{U}_\ell$ for every $\ell \in [\frac{N-1}{2}]$.
- If all checks pass, i.e., $\overline{\mathbf{V}}_\ell = \mathbf{V}_\ell$ for every $\ell \in [N-1]$ and $\overline{\mathbf{U}}_\ell = \mathbf{U}_\ell$ for every $\ell \in [\frac{N-1}{2}]$, $\mathcal{S}$ waits for an instruction from the adversary.
 - If $\mathcal{S}$ receives **accept** from the adversary, $\mathcal{S}$ outputs **accept** to the adversary on behalf of $\mathcal{F}_{\text{batched-vrfy}}$, and continues the simulation.
 - * In this case, $\mathcal{S}$ knows for every $\ell \in \{\frac{N+3}{2}, \dots, N\}$, after invoking Π_{mult} , all parties hold their shares of $\llbracket \mathbf{F}(\ell) * \mathbf{G}(\ell) - \mathbf{A}_\ell * \mathbf{B}_\ell + \mathbf{C}_\ell \rrbracket_{t+k-1}$.
 - If $\mathcal{S}$ receives **reject** and a disputed pair $E \subset \mathcal{P}_{\text{active}}$ from the adversary, $\mathcal{S}$ outputs **abort** and E on behalf of $\mathcal{F}_{\text{batched-vrfy}}$, and sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
 - If one of the checks fails, $\mathcal{S}$ receives a disputed pair $E \subset \mathcal{P}_{\text{active}}$ from the adversary and outputs E on behalf of $\mathcal{F}_{\text{batched-vrfy}}$, and sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
9. In Step 2.4, recall for every $\ell \in [\frac{N+1}{2}]$, $\mathcal{S}$ learns corrupted parties' shares of $\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}$ when simulating $\Pi_{\text{batched-vrfy}}$, and for every $\ell \in \{\frac{N+3}{2}, \dots, N\}$, $\mathcal{S}$ learns corrupted parties' shares of $\llbracket \mathbf{F}(\ell) * \mathbf{G}(\ell) - \mathbf{A}_\ell * \mathbf{B}_\ell + \mathbf{C}_\ell \rrbracket_{t+k-1}$ when simulating Π_{mult} . Then $\mathcal{S}$ learns corrupted parties' shares of $\llbracket \overline{\mathbf{H}}(\cdot) \rrbracket_{t+k-1}$, where $\overline{\mathbf{H}}$ is a $k \times k$ matrix of k^2 degree- $(N-1)$ polynomials defined as $\overline{\mathbf{H}}(\ell) = \mathbf{C}_\ell$ for every $\ell \in [\frac{N+1}{2}]$ and $\overline{\mathbf{H}}(\ell) = \mathbf{F}(\ell) * \mathbf{G}(\ell) - \mathbf{A}_\ell * \mathbf{B}_\ell + \mathbf{C}_\ell$ for every $\ell \in \{\frac{N+3}{2}, \dots, N\}$.
- Simulation of triple verification.**
10. In Step 3.1, for $\ell \in [N]$ such that $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \geq 2$, i.e., $\ell \in \mathcal{E}$, recall $\mathcal{S}$ learns $(\mathbf{A}_\ell, \mathbf{B}_\ell, \mathbf{C}_\ell)$ when simulating Step 2.4 of $\Pi_{\text{batched-vrfy}}$. Then $\mathcal{S}$ checks whether $\mathbf{A}_\ell * \mathbf{B}_\ell = \mathbf{C}_\ell$ holds. Denote the set of packed triples which are checked by corrupted parties by $\mathcal{E}_3$. Then $|\mathcal{E}_3| = \frac{3.5N}{27}$.
- If one of the checks fails, $\mathcal{S}$ computes honest parties' shares of $\llbracket \mathbf{F}(\cdot) \rrbracket_{t+k-1}$, $\llbracket \mathbf{G}(\cdot) \rrbracket_{t+k-1}$, and $\llbracket \overline{\mathbf{H}}(\cdot) \rrbracket_{t+k-1}$ as follows.
 - For each $\ell \in \mathcal{D}_{1,0} \cup \mathcal{D}_{1,2}$, $\mathcal{S}$ randomly samples honest parties' shares of $\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}$ and $\llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}$ based on shares of corrupted parties.
 - For $\ell \in \mathcal{D}_{1,1}$, $\mathcal{S}$ computes honest parties' shares of $\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}$, $\llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}$ as linear combinations of their shares of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{D}_{1,0} \cup \mathcal{D}_{1,2} \cup \mathcal{E}_2}$.
 - For each $\ell \in \mathcal{D}_1$, $\mathcal{S}$ randomly samples honest parties' shares of $\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}$ based on shares of corrupted parties and the constraint $\mathbf{C}_\ell = \mathbf{A}_\ell * \mathbf{B}_\ell$. Notice that $\llbracket \overline{\mathbf{H}}(\ell) \rrbracket_{t+k-1} = \llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}$ for $\ell \in \mathcal{D}_1$.
 - For $\ell \in \mathcal{D}_2$, $\mathcal{S}$ computes honest parties' shares of $\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}$, $\llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}$ and their shares of $\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}$, $\llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}$. $\mathcal{S}$ samples honest parties' shares of $\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}$ based on shares of corrupted parties and $\mathbf{C}_\ell = \mathbf{A}_\ell * \mathbf{B}_\ell$, and samples a pair of double sharings in the form of $(\llbracket \mathbf{R} \rrbracket_{t+k-1}, \llbracket \mathbf{R} \rrbracket_{t+2k-2})$ based on shares of corrupted parties. In doing this, $\mathcal{S}$ can faithfully compute honest parties' shares of $\llbracket \overline{\mathbf{H}}(\ell) \rrbracket_{t+k-1}$ for $\ell \in \mathcal{D}_2$.
 - For each $\ell \in \mathcal{E}$, recall $\mathcal{S}$ learns honest parties' shares of $\llbracket \overline{\mathbf{H}}(\ell) \rrbracket_{t+k-1}$ when simulating Step 2.4 of $\Pi_{\text{batched-vrfy}}$ and Step 9 of Π_{mult} .
- $\mathcal{S}$ then faithfully follows the protocol to identify a disputed pair $E \subset \mathcal{P}_{\text{active}}$. Then $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.

- If all checks pass, $\mathcal{S}$ learns $\overline{\mathbf{H}} = \mathbf{F} * \mathbf{G}$. For $\ell \in [\frac{10.5N}{27}]$, $\mathcal{S}$ computes corrupted parties' shares of $(\llbracket \mathbf{F}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N + \ell) \rrbracket_{t+k-1})$ as linear combinations of their shares of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in [N]}$. Then $\mathcal{S}$ simulates according to whether $P_i := P_{(\ell \bmod n')}$ is corrupted.
 - If P_i is corrupted, $\mathcal{S}$ samples honest parties' shares of $(\llbracket \mathbf{F}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N + \ell) \rrbracket_{t+k-1})$ based on shares of corrupted parties and the constraint $\mathbf{H}(N + \ell) = \mathbf{F}(N + \ell) * \mathbf{G}(N + \ell)$. Then $\mathcal{S}$ sends shares of honest parties to the adversary on behalf of honest parties. Then in Step 3.1.a, $\mathcal{S}$ faithfully follows the protocol.
 - * If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
 - If P_i is honest, $\mathcal{S}$ receives corrupted parties' shares of $(\llbracket \mathbf{F}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N + \ell) \rrbracket_{t+k-1})$ from the adversary. In Step 3.1.a, for $\mathbf{U} \in \{\mathbf{F}(N + \ell), \mathbf{G}(N + \ell), \mathbf{H}(N + \ell)\}$, $\mathcal{S}$ computes the evaluation points joint held by an honest party and a corrupted party from corrupted parties' shares of $\llbracket \mathbf{U} \rrbracket_{t+k-1}$ computed previously. In particular, for every $j_1 \in \mathcal{C}_{\text{active}}$ and $j_2 \in \mathcal{H}_{\text{active}}$, $\mathcal{S}$ computes $f_{j_2}^{\mathbf{U}}(\alpha_{j_1}) := g_{j_1}^{\mathbf{U}}(\alpha_{j_2})$ and $g_{j_2}^{\mathbf{U}}(\alpha_{j_1}) := f_{j_1}^{\mathbf{U}}(\alpha_{j_2})$. $\mathcal{S}$ uses these values to simulate the behaviors of honest parties.
 - * If a disputed pair E is output, $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
 - In Step 3.1.b, $\mathcal{S}$ broadcasts **accept** on behalf of honest P_i .
- In Step 3.2,
- If all active parties broadcast **accept**, for every $\ell \in \{\frac{10.5N}{27} + 1, \dots, \frac{N+1}{2}\}$, $\mathcal{S}$ computes shares of corrupted parties of $(\llbracket \mathbf{F}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N + \ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N + \ell) \rrbracket_{t+k-1})$ as linear combinations of their shares of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in [N]}$, sends them and **accept** to $\mathcal{F}_{\text{packed-triple}}$.
 - If party P_i broadcasts **(Incorrect, ℓ, P_i)**, then $\mathcal{S}$ simulates $\Pi_{\text{fault-localize-triple}}(P_i)$ as follows.
 - * In Step 1, $\mathcal{S}$ simulates the behaviors of honest parties faithfully. Since the triple checked by P_i is correct, either P_i is identified as a corrupted party, or a disputed pair $\{P_i, P_{i'}\}$ is identified.
- $\mathcal{S}$ sends **reject** and the disputed pair E to $\mathcal{F}_{\text{packed-triple}}$.
11. $\mathcal{S}$ outputs what the adversary outputs and honest parties output their outputs received from $\mathcal{F}_{\text{packed-triple}}$.

Before showing the distribution of the output in the ideal world is identical to that in the real world, we first prove two claims regarding the effectiveness of **Consistency check** and **Triple verification** as follows.

Claim. If all parties accept in **Consistency check** when invoking $\Pi_{\text{batched-vrfy}}$, then for each $\ell \in [N]$, the secrets of sharings $\llbracket \mathbf{U}_\ell(1) \rrbracket_{t+k-1}, \llbracket \mathbf{U}_\ell(2) \rrbracket_{t+k-1}, \llbracket \mathbf{U}_\ell(3) \rrbracket_{t+k-1}$ distributed by $P_{\ell_1}, P_{\ell_2}, P_{\ell_3}$ (cf. Step 3 of the simulation), respectively, constitute a $k \times k$ matrix of degree-1 polynomials.

Proof. If all parties accept their sharings, then $n' - t'$ tuples of sharings $\{\llbracket \epsilon^{(\ell)} \rrbracket_{t+k-1}\}_{\ell \in \mathcal{P}_{\text{active}}}$ checked by $n' - t'$ active honest parties are guaranteed to be correct, i.e., their secrets are $\mathbf{0} \in \mathbb{F}^{k \times k}$. By property of Vandermonde matrix, there is a one-to-one correspondence between $n' - t'$ sharings $\{\llbracket \epsilon^{(\ell)} \rrbracket_{t+k-1}\}_{\ell \in \mathcal{P}_{\text{active}}}$ checked by active honest parties and original $n' - t'$ sharings $\{\llbracket \epsilon_\ell \rrbracket_{t+k-1}\}_{\ell \in [n' - t']}$.

Furthermore, by functionality of $\mathcal{F}_{\text{rand-sharing}}$, all honest parties' shares of $\{\llbracket \mathbf{O}_\ell \rrbracket_{t+k-1}\}$ form degree- $(t + k - 1, t + k - 1)$ packed Shamir sharings with secret being $\mathbf{0} \in \mathbb{F}^{k \times k}$. Therefore, all honest parties' shares of $\{\llbracket \theta_\ell \rrbracket_{t+k-1}\}_N$ form degree- $(t + k - 1, t + k - 1)$ packed Shamir sharings with secret being $\mathbf{0} \in \mathbb{F}^{k \times k}$. As $\mathbf{0} = \theta_\ell = \lambda_{1,1}(3) \cdot \Delta \mathbf{U}_\ell(1) + \lambda_{1,2}(3) \cdot \Delta \mathbf{U}_\ell(2) + \Delta \mathbf{U}_\ell(3)$, this in turn implies the secrets of sharings $\llbracket \mathbf{U}_\ell(1) \rrbracket_{t+k-1}, \llbracket \mathbf{U}_\ell(2) \rrbracket_{t+k-1}, \llbracket \mathbf{U}_\ell(3) \rrbracket_{t+k-1}$ constitute a $k \times k$ matrix of degree-1 polynomials. $\square$

Claim. If all parties accept in **Triple verification**, supposing they hold shares of $\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}$, then it holds that $\overline{\mathbf{H}}(\ell) = \mathbf{F}(\ell) * \mathbf{G}(\ell)$ for every $\ell \in [N]$, i.e., $\overline{\mathbf{H}} = \mathbf{F} * \mathbf{G}$.

Proof. By the claim above, if all parties accept the sharings in **Consistency check**, then for each $U \in \{A_\ell, B_\ell, C_\ell\}$, three sharings $\llbracket U_\ell(1) \rrbracket_{t+k-1}$, $\llbracket U_\ell(2) \rrbracket_{t+k-1}$, $\llbracket U_\ell(3) \rrbracket_{t+k-1}$ distributed by the ℓ -th committee constitute a $k \times k$ matrix of degree-1 polynomials. Hence, if they are distributed by a committee containing at least two honest parties, the secrets are fully determined by sharings distributed by honest parties, and therefore guaranteed to satisfy the multiplication relations. This implies there are a number of $(1 - (\frac{1}{3})^3 - 3 \cdot (\frac{1}{3})^2 \cdot \frac{2}{3}) \cdot N = \frac{20N}{27}$ packed triples are correct.

In **Triple verification**, there are a number of $\frac{10.5N}{27} \cdot \frac{2}{3} = \frac{7N}{27}$ triples checked by honest parties. Therefore, there are $\frac{20N}{27} + \frac{7N}{27} = N$ packed triples guaranteed to be correct, which means the $k \times k$ matrix of degree- $(N-1)$ polynomials $F * G - H$ has at least N roots. Hence, $F * G - H \equiv 0$. $\square$

We now show that the distribution of the output in the ideal world is identical to that in the real world using the following hybrid arguments.

Hyb₀: In this hybrid, we consider the execution in the real world.

Hyb₁: In this hybrid, in Step 0, instead of invoking $\mathcal{F}_{\text{rand-sharing}}(\Sigma_2, \frac{N-1}{2})$ and $\mathcal{F}_{\text{rand-sharing}}(\Sigma_5, \frac{N-1}{2})$, $\mathcal{S}$ emulates by itself and records shares of corrupted parties. $\mathcal{S}$ delays the sampling of honest parties' shares. **Hyb₁** and **Hyb₀** have identical distributions.

Hyb₂: In this hybrid, in Step 1.1.a, instead of invoking $\mathcal{F}_{3\text{-triple}}$, $\mathcal{S}$ emulates by itself and records shares of corrupted party. For $\ell \in [N]$ such that $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \leq 1$, $\mathcal{S}$ delays the sampling of honest parties' shares. **Hyb₂** and **Hyb₁** have identical distributions.

Hyb₃: In this hybrid, in Step 1.1.b, when an honest party distributes degree- $(t+k-1, t+k-1)$ packed Shamir sharings, $\mathcal{S}$ samples random shares subject to degree- $(t+k-1, t+k-1)$ packed Shamir sharings as shares of corrupted parties. When invoking $\pi_{\text{check-consistency-all}}$, $\mathcal{S}$ computes the evaluation points held by an honest party and a corrupted party using shares of corrupted parties. **Hyb₃** and **Hyb₂** have identical distributions.

Hyb₄: In this hybrid, in Step 1.1.b, when a corrupted party P_{ℓ_v} distributes degree- $(t+k-1, t+k-1)$ packed Shamir sharing $\llbracket U_\ell(v) \rrbracket_{t+k-1}$. If all parties accept in $\pi_{\text{check-consistency-all}}$, $\mathcal{S}$ reconstructs the secret $\widetilde{U_\ell(v)}$, and additionally computes difference $\Delta U_\ell(v) \leftarrow \Delta U_\ell(v) - U_\ell(v)$, where $U_\ell(v)$ is computed using shares of P_{ℓ_v} received when $\mathcal{F}_{3\text{-triple}}$. **Hyb₄** and **Hyb₃** have identical distributions.

Hyb₅: In this hybrid, when simulating Step 1 of $\Pi_{\text{batched-vrfy}}$, instead of invoking $\mathcal{F}_{\text{rand-sharing}}(\Sigma_6, N)$, $\mathcal{S}$ emulates it by itself and records shares of corrupted parties. **Hyb₅** and **Hyb₄** have identical distributions.

Hyb₆: In this hybrid, when simulating Step 2.1 of $\Pi_{\text{batched-vrfy}}$, we change the computation of honest parties' shares of $\llbracket \epsilon_\ell \rrbracket_{t+k-1}$ as follows. We first compute corrupted parties' shares of $\llbracket \epsilon_\ell \rrbracket_{t+k-1}$ and the secrets as $\epsilon_\ell = \lambda_{1,1}(3) \cdot \Delta U_\ell(1) + \lambda_{1,2}(3) \cdot \Delta U_\ell(2) - \Delta U_\ell(3)$, then randomly samples a degree- $(t+k-1, t+k-1)$ packed Shamir sharing $\llbracket \theta_\ell \rrbracket_{t+k-1}$ based on shares of corrupted parties and the secrets. Then we compute honest parties' shares of $\llbracket O_\ell \rrbracket_{t+k-1}$ as $\llbracket O_\ell \rrbracket_{t+k-1} \leftarrow \llbracket \theta_\ell \rrbracket_{t+k-1} - \llbracket \epsilon_\ell \rrbracket_{t+k-1}$. **Hyb₆** and **Hyb₅** have identical distributions due to the randomness of $\llbracket O_\ell \rrbracket_{t+k-1}$.

Hyb₇: In this hybrid, when simulating $\Pi_{\text{batched-vrfy}}$, if all parties broadcast **accept**, we do not compute honest parties' shares of $\llbracket O_\ell \rrbracket_{t+k-1}$, which is feasible since they not used in **Hyb₆**. **Hyb₇** and **Hyb₆** have identical distributions.

Hyb₈: In this hybrid, when simulating $\Pi_{\text{batched-vrfy}}$, if all parties broadcast **accept**, we compute corrupted parties' shares of $\llbracket A_\ell \rrbracket_{t+k-1}$, $\llbracket B_\ell \rrbracket_{t+k-1}$, $\llbracket C \rrbracket_{t+k-1}$ for each $\ell \in [N]$. Additionally, for each $\ell \in [N]$ such that $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \geq 2$, we compute the whole sharings $\llbracket A_\ell \rrbracket_{t+k-1}$, $\llbracket B_\ell \rrbracket_{t+k-1}$, $\llbracket C \rrbracket_{t+k-1}$. **Hyb₈** and **Hyb₇** have identical distributions.

Hyb₉: In this hybrid, when simulating Π_{mult} , when all parties use a packed triple known to the adversary, we change the computation of honest parties' shares of $\llbracket X + A \rrbracket_{t+k-1}$, $\llbracket Y + B \rrbracket_{t+k-1}$ as follows. We first compute corrupted parties' shares of $\llbracket X \rrbracket_{t+k-1}$, $\llbracket Y \rrbracket_{t+k-1}$ and randomly sample two sharings as $\llbracket X \rrbracket_{t+k-1}$, $\llbracket Y \rrbracket_{t+k-1}$ based on shares of corrupted parties. Then we compute honest parties' shares of $\{\llbracket A_\ell \rrbracket_{t+k-1}, \llbracket B_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,1}}$ as linear combinations of their shares of $\{\llbracket F(\ell) \rrbracket_{t+k-1}, \llbracket G(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{D}_{1,0} \cup \mathcal{D}_{1,2}}$, then sample honest parties' shares of $\llbracket C_\ell \rrbracket_{t+k-1}$ based on shares of corrupted parties and the secret $C_\ell = A_\ell * B_\ell$ for each $\ell \in \mathcal{D}_{1,1}$. **Hyb₉** and **Hyb₈** have identical distributions because there is a one-to-one

correspondence between $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,1}}$ and $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_2}$ given values of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{D}_{1,0} \cup \mathcal{D}_{1,2}}$, and $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,1}}$ are random in **Hyb**₈.

Hyb₁₀: In this hybrid, when simulating $\mathcal{I}_{\text{mult}}$, when all parties use a packed triple unknown to the adversary, we change the computation of honest parties' shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1}$ as follows. We first compute corrupted parties' shares of $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1}$, randomly sample two degree- $(t+k-1, t+k-1)$ packed Shamir sharings as $\llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1}$ based on shares of corrupted parties. Then we compute honest parties' shares of $\llbracket \mathbf{X} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} \rrbracket_{t+k-1}$ as $\llbracket \mathbf{X} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{X} + \mathbf{A} \rrbracket_{t+k-1} - \llbracket \mathbf{A} \rrbracket_{t+k-1}$ and $\llbracket \mathbf{Y} \rrbracket_{t+k-1} \leftarrow \llbracket \mathbf{Y} + \mathbf{B} \rrbracket_{t+k-1} - \llbracket \mathbf{B} \rrbracket_{t+k-1}$, respectively. **Hyb**₁₀ and **Hyb**₉ have identical distributions because of the randomness of $\llbracket \mathbf{A} \rrbracket_{t+k-1}, \llbracket \mathbf{B} \rrbracket_{t+k-1}$.

Hyb₁₁: In this hybrid, instead of invoking $\mathcal{F}_{\text{batched-vrfy}}(P_{\text{king}}, \Sigma_8)$, $\mathcal{S}$ emulates itself as follows. $\mathcal{S}$ sends honest parties' shares of $\{\llbracket \mathbf{V}_\ell \rrbracket_{k-1}, \llbracket \mathbf{V}_\ell \rrbracket_{t+k-1}\}_{\ell \in [N]}$ and $\{\llbracket \widetilde{\mathbf{U}}_\ell \rrbracket_{k-1}, \llbracket \widetilde{\mathbf{U}}_\ell \rrbracket_{t+2k-2}\}_{\ell \in \frac{N-1}{2}}$ to the adversary and checks whether $\widetilde{\mathbf{V}}_\ell = \mathbf{V}_\ell$ for every $\ell \in [N]$ and $\widetilde{\mathbf{U}}_\ell = \mathbf{U}_\ell$ for every $\ell \in [\frac{N-1}{2}]$. **Hyb**₁₁ and **Hyb**₁₀ have identical distributions.

Hyb₁₂: In this hybrid, when simulating Step 3.1, $\mathcal{S}$ additionally checks for $\ell \in [N]$ such that $|\mathcal{C} \cap \{\ell_1, \ell_2, \ell_3\}| \geq 2$, i.e., $\ell \in \mathcal{E}$, whether it holds that $\mathbf{A}_\ell * \mathbf{B}_\ell = \mathbf{C}_\ell$. If all checks pass, we change the computation of honest parties' shares of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_3}$ as follows. For each $\ell \in \mathcal{E}_3$, we first compute corrupted parties' shares of $(\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1})$, and randomly sample two degree- $(t+k-1, t+k-1)$ packed Shamir sharings as $\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}$ based on shares of corrupted parties. Then we compute $\mathbf{F}(\ell), \mathbf{G}(\ell)$ and $\mathbf{H}(\ell) \leftarrow \mathbf{F}(\ell) * \mathbf{G}(\ell)$, and randomly sample a degree- $(t+k-1, t+k-1)$ packed Shamir sharing as $\llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}$ based on shares of corrupted parties and the secret. Then we compute honest parties' shares of $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,2}}$ as linear combinations of their shares of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{E}_3 \cup \mathcal{D}_{1,0}}$, then compute honest parties' shares of $\{\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,2}}$ as linear combinations of their shares of $\{\llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{E}_3 \cup \mathcal{D}_{1,0} \cup \mathcal{D}_{1,1}}$. **Hyb**₁₂ and **Hyb**₁₁ have identical distributions because

- by the second claim above, if all checks pass, we have $\mathbf{F} * \mathbf{G} = \mathbf{H}$;
- there is a one-to-one correspondence between $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_3}$ and $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,2}}$ given values in $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{D}_{1,0}}$, and $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,2}}$ are random in **Hyb**₁₁;
- there is a one-to-one correspondence between $\{\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,2}}$ and $\{\llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_3}$ given the values in $\{\llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{D}_{1,0} \cup \mathcal{D}_{1,1}}$ and $\{\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,2}}$ are random subject to shares of corrupted parties and the secrets $\mathbf{C}_\ell = \mathbf{A}_\ell * \mathbf{B}_\ell$ in **Hyb**₁₁.

Hyb₁₃: In this hybrid, we change the way of simulating honest parties' outputs and let honest parties receive outputs from $\mathcal{F}_{\text{packed-triple}}$.

- If one of checks in **Hyb**₁₂ fails, a disputed pair E has been identified. $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
- If all checks in **Hyb**₁₂ pass,
 - if some party broadcasts **Incorrect** in **triple verification**, a disputed pair E has been identified. $\mathcal{S}$ sends **reject** together with E to $\mathcal{F}_{\text{packed-triple}}$.
 - if all parties broadcast **accept** in **triple verification**, for each $\ell \in \{\frac{10.5N}{27} + 1, \dots, \frac{N+1}{2}\}$, we compute corrupted parties' shares of $(\llbracket \mathbf{F}(N+\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(N+\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{H}(N+\ell) \rrbracket_{t+k-1})$ and sends them to $\mathcal{F}_{\text{packed-triple}}$.

We compute honest parties' shares of $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,0}}$ as linear combinations of their shares of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{E}_3 \cup \{N + \frac{10.5N}{27} + 1, \dots, N + \frac{N+1}{2}\}}$. We then compute their shares of $\{\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,0}}$ as linear combinations of their shares of $\{\llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{E}_3 \cup \mathcal{D}_{1,0} \cup \mathcal{D}_{1,1} \cup \mathcal{D}_{1,2}}$ and $\{\llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\{N + \frac{10.5N}{27} + 1, \dots, N + \frac{N+1}{2}\}}$.

Hyb₁₃ and **Hyb**₁₂ have identical distributions because

- there is a one-to-one correspondence between $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \{N + \frac{10.5N}{27} + 1, \dots, N + \frac{N+1}{2}\}}$ and $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,0}}$ given values in $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{E}_3}$, and $\{\llbracket \mathbf{A}_\ell \rrbracket_{t+k-1}, \llbracket \mathbf{B}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,0}}$ are random in **Hyb**₁₂;

- there is a one-to-one correspondence between $\{\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,0}}$ and $\{\llbracket \overline{\mathbf{H}}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \{N + \frac{10.5N}{N} + 1, \dots, N + \frac{N+1}{2}\}}$ given values in $\{\llbracket \overline{\mathbf{H}}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{E}_1 \cup \mathcal{E}_2 \cup \mathcal{E}_3 \cup \mathcal{D}_2 \cup \mathcal{D}_{1,1} \cup \mathcal{D}_{1,2}}$, and $\{\llbracket \mathbf{C}_\ell \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,0}}$ are random subject to shares of corrupted parties and the secrets $\mathbf{C}_\ell = \mathbf{A}_\ell * \mathbf{B}_\ell$ in **Hyb**₁₂.

Hyb₁₄: In this hybrid, if all checks in **Hyb**₁₂ pass and all parties broadcast **accept** in **triple verification**, we do not compute honest parties' shares of $\{\llbracket \mathbf{F}(\ell) \rrbracket_{t+k-1}, \llbracket \mathbf{G}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_{1,0} \cup \mathcal{D}_{1,1} \cup \mathcal{D}_{1,2}}$ and their shares of $\{\llbracket \mathbf{H}(\ell) \rrbracket_{t+k-1}\}_{\ell \in \mathcal{D}_2 \cup \mathcal{D}_{1,0} \cup \mathcal{D}_{1,1} \cup \mathcal{D}_{1,2}}$, which is feasible as they are not used in the simulation. **Hyb**₁₄ and **Hyb**₁₃ have identical distributions.

Since **Hyb**₁₄ corresponds to the ideal world, $\Pi_{\text{packed-triple}}$ securely computes $\mathcal{F}_{\text{packed-triple}}$ in the $\mathcal{F}_{3\text{-triple}}$, $\mathcal{F}_{\text{rand-sharing}}$ -hybrid model. $\square$

Acknowledgments

Y. Song was supported in part by the National Basic Research Program of China Grant 2011CBA00300, 2011CBA00301, the National Natural Science Foundation of China Grant 61033001, 61361136003, and the Shanghai Qi Zhi Institute Innovation Program SQZ202313. X. Ye was supported by the National Key R&D Program of China (Grant No. 2023YFA1011200) and the National Natural Science Foundation of China (Grant No. 62372262).

References

- AAPP23. Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Detect, pack and batch: Perfectly-secure mpc with linear communication and constant expected time. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology – EUROCRYPT 2023*, pages 251–281, Cham, 2023. Springer Nature Switzerland.
- AC24. Gilad Asharov and Anirudh Chandramouli. Perfect (parallel) broadcast in constant expected rounds via statistical vss. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 310–339, Cham, 2024. Springer Nature Switzerland.
- BGP92. Piotr Berman, Juan A. Garay, and Kenneth J. Perry. *Bit optimal distributed consensus*, page 313–321. Plenum Press, USA, 1992.
- BGW88. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, pages 1–10. ACM, 1988.
- BH08. Zuzana Beerliová-Trubíniová and Martin Hirt. Perfectly-secure mpc with linear communication complexity. In Ran Canetti, editor, *Theory of Cryptography*, pages 213–230, Berlin, Heidelberg, 2008. Springer Berlin Heidelberg.
- Bra87. Gabriel Bracha. An $o(\log n)$ expected rounds randomized byzantine generals protocol. *J. ACM*, 34(4):910–920, oct 1987.
- Can00. Ran Canetti. Security and composition of multiparty cryptographic protocols. *Journal of Cryptology*, 13:143–202, 2000.
- CCD88. David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, pages 11–19. ACM, 1988.
- CP17. Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Transactions on Information Theory*, 63:428–468, 2017.
- CW92. Brian A. Coan and Jennifer L. Welch. Modular construction of a byzantine agreement protocol with optimal message bit complexity. *Information and Computation*, 97(1):61–85, 1992.
- DEN24. Anders Dalskov, Daniel Escudero, and Ariel Nof. Fully secure mpc and zk-flip over rings: New constructions, improvements and extensions. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 136–169, Cham, 2024. Springer Nature Switzerland.
- DLN19. Ivan Damgård, Kasper Green Larsen, and Jesper Buus Nielsen. Communication lower bounds for statistically secure mpc, with or without preprocessing. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 61–84, Cham, 2019. Springer International Publishing.
- DLS21. Ivan Bjerre Damgård, Boyang Li, and Nikolaj Ignatieff Schwartzbach. More Communication Lower Bounds for Information-Theoretic MPC. In Stefano Tessaro, editor, *2nd Conference on Information-Theoretic Cryptography (ITC 2021)*, volume 199 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 2:1–2:18, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

- DPPR25. Ivan Damgård, Shravani Patil, Arpita Patra, and Lawrence Roy. New upper and lower bounds for perfectly secure MPC. Cryptology ePrint Archive, Paper 2025/1206, 2025.
- EGPS22. Daniel Escudero, Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. TurboPack: Honest majority MPC with constant online communication. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 951–964, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
- GBO⁺23. Mariana Gama, Emad Heydari Beni, Emmanuela Orsini, Nigel P. Smart, and Oliver Zajonc. MPC with delayed parties over star-like networks. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 172–203, Singapore, 2023. Springer Nature Singapore.
- GLS19. Vipul Goyal, Yanyi Liu, and Yifan Song. Communication-efficient unconditional mpc with guaranteed output delivery. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology – CRYPTO 2019*, pages 85–114, Cham, 2019. Springer International Publishing.
- GLS24. Vipul Goyal, Chen-Da Liu-Zhang, and Yifan Song. Towards achieving asynchronous mpc with linear communication and optimal resilience. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology – CRYPTO 2024*, pages 170–206, Cham, 2024. Springer Nature Switzerland.
- GMW87. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*, pages 218–229. ACM, 1987.
- GPS21. Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Unconditional communication-efficient mpc via hall’s marriage theorem. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology – CRYPTO 2021*, pages 275–304, Cham, 2021. Springer International Publishing.
- GPS22. Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Sharing transformation and dishonest majority mpc with packed secret sharing. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022*, pages 3–32, Cham, 2022. Springer Nature Switzerland.
- HMP00. Martin Hirt, Ueli Maurer, and Bartosz Przydatek. Efficient secure multi-party computation. In Tatsuaki Okamoto, editor, *Advances in Cryptology — ASIACRYPT 2000*, pages 143–161, Berlin, Heidelberg, 2000. Springer Berlin Heidelberg.
- PP25. Shravani Patil and Arpita Patra. Perfectly-secure network-agnostic mpc with optimal resiliency. In *Proceedings of the ACM Symposium on Principles of Distributed Computing*, PODC ’25, page 121–130, New York, NY, USA, 2025. Association for Computing Machinery.
- SY25a. Yifan Song and Xiayi Ye. Honest majority mpc with $\tilde{O}(|c|)$ communication in minicrypt. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 453–483, Cham, 2025. Springer Nature Switzerland.
- SY25b. Yifan Song and Xiayi Ye. Perfectly-secure mpc with constant online communication complexity. In Elette Boyle and Mohammad Mahmoody, editors, *Theory of Cryptography*, pages 329–361, Cham, 2025. Springer Nature Switzerland.
- Yao82. Andrew C Yao. Protocols for secure computations. In *Foundations of Computer Science, 1982. SFCS’08. 23rd Annual Symposium on*, pages 160–164. IEEE, 1982.